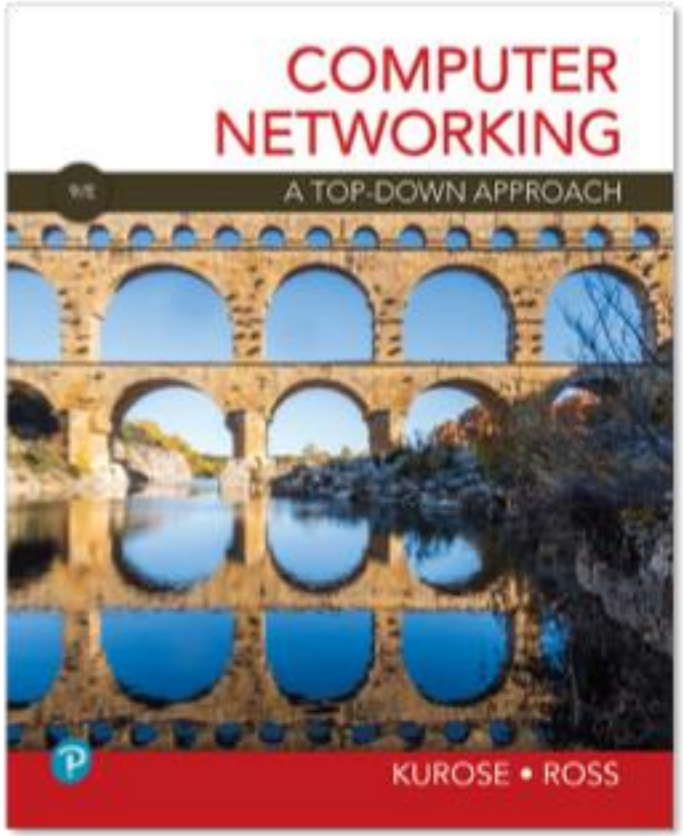
Chapter 4
Network Layer : Data Plane

第00讲-课程简介_哔哩哔哩_bilibili

https://www.bilibili.com/video/BV1Ut411d7RE/?p=1&vd_source=5fbc4920b6c246ce2d9126818c78b17b







Computer Networking: A Top-Down Approach

9th edition

Jim Kurose, Keith Ross

Pearson, 2025

Network layer: our goals

- understand principles behind network layer services, focusing on data plane:
  - network layer service models
  - forwarding versus routing
  - how a router works
  - addressing
  - generalized forwarding
  - Internet architecture
- instantiation, implementation in the Internet
  - IP protocol
  - NAT, middleboxes



Network layer: “data plane” roadmap

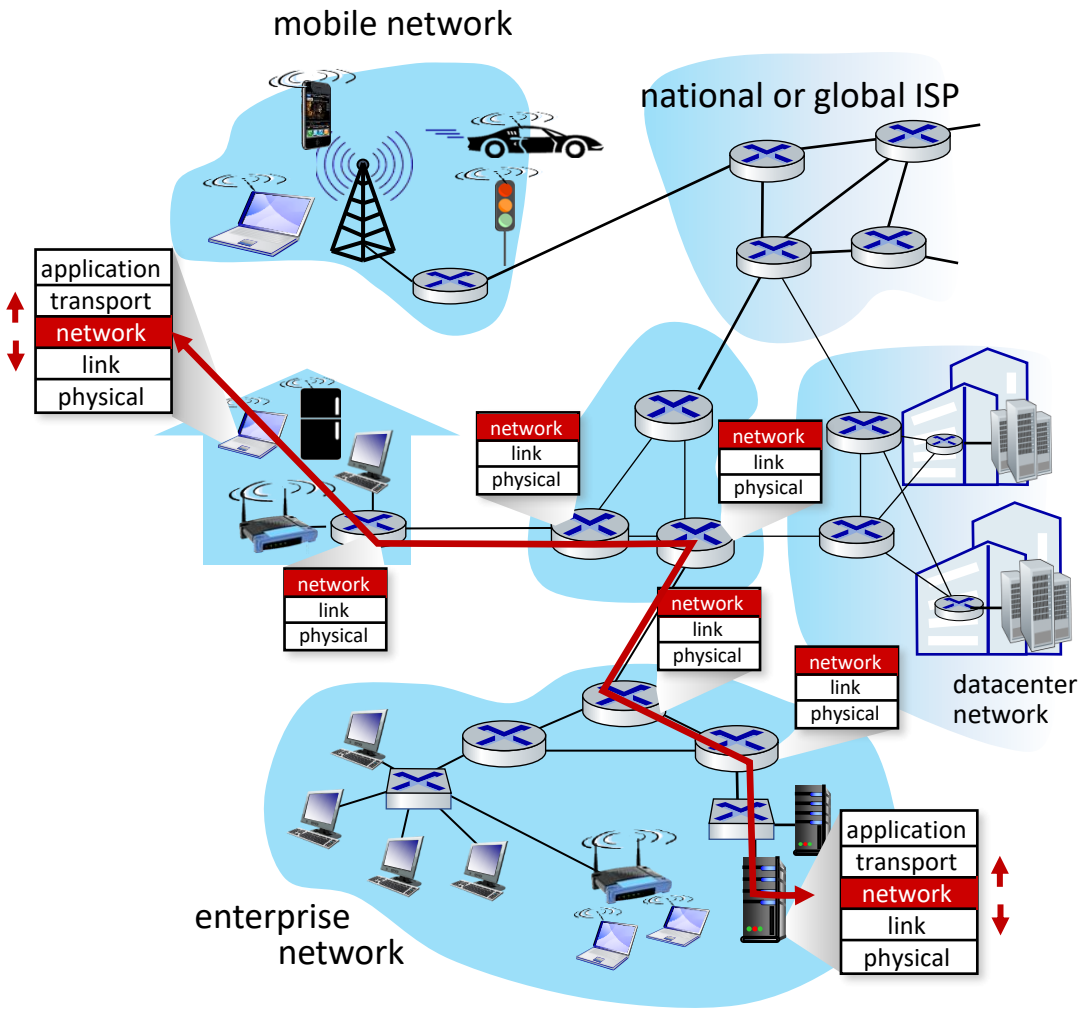
- Network layer: overview
  - data plane
  - control plane
- What’s inside a router
  - input/output ports, switching
  - buffer management
  - scheduling
- IP: the Internet Protocol
  - datagram format, addressing
  - network address translation (NAT)
  - IPv6



- Generalized Forwarding
  - Match+action
  - OpenFlow
  - Middleboxes
- Internet architecture

Network-layer services and protocols

- transport segment from sending to receiving host
  - sender: encapsulates segments into datagrams, passes to link layer
  - receiver: delivers segments to transport layer protocol
- network layer protocols in *every Internet device*: hosts, routers
- routers:
  - examines header fields in all IP datagrams passing through it
  - moves datagrams from input ports to output ports to transfer datagrams along end-end path



Two key network-layer functions

network-layer functions:

- *forwarding:* move packets from a router's input link to appropriate router output link
- *routing:* determine route taken by packets from source to destination
  - *routing algorithms*

analogy: taking a trip

- *forwarding:* process of getting through single interchange
- *routing:* process of planning trip from source to destination



forwarding

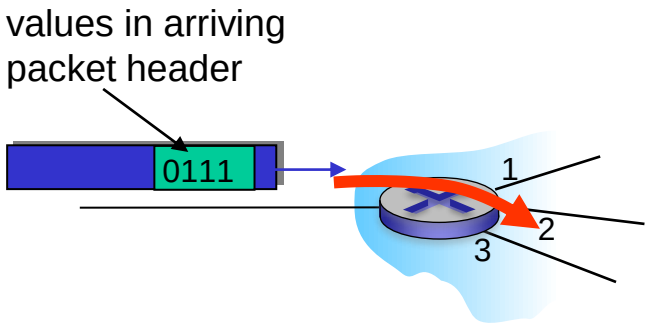


routing

Network layer: data plane, control plane

Data plane:

- *local*, per-router function
- determines how datagram arriving on router input port is forwarded to router output port

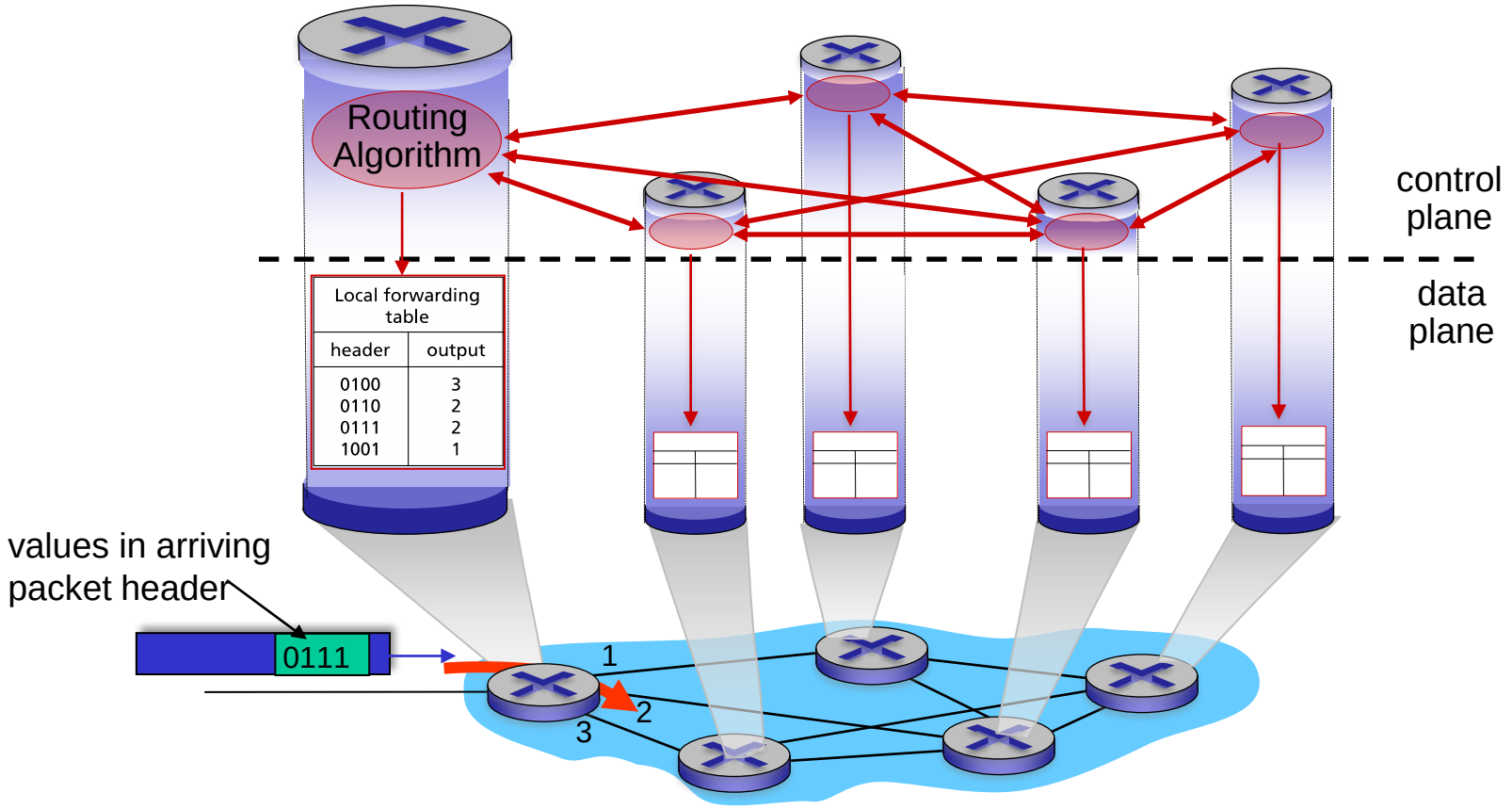


Control plane

- *network-wide* logic
- determines how datagram is routed among routers along end-to-end path from source host to destination host
- two control-plane approaches:
  - *traditional routing algorithms:* implemented in routers
  - *software-defined networking (SDN):* implemented in (remote) servers

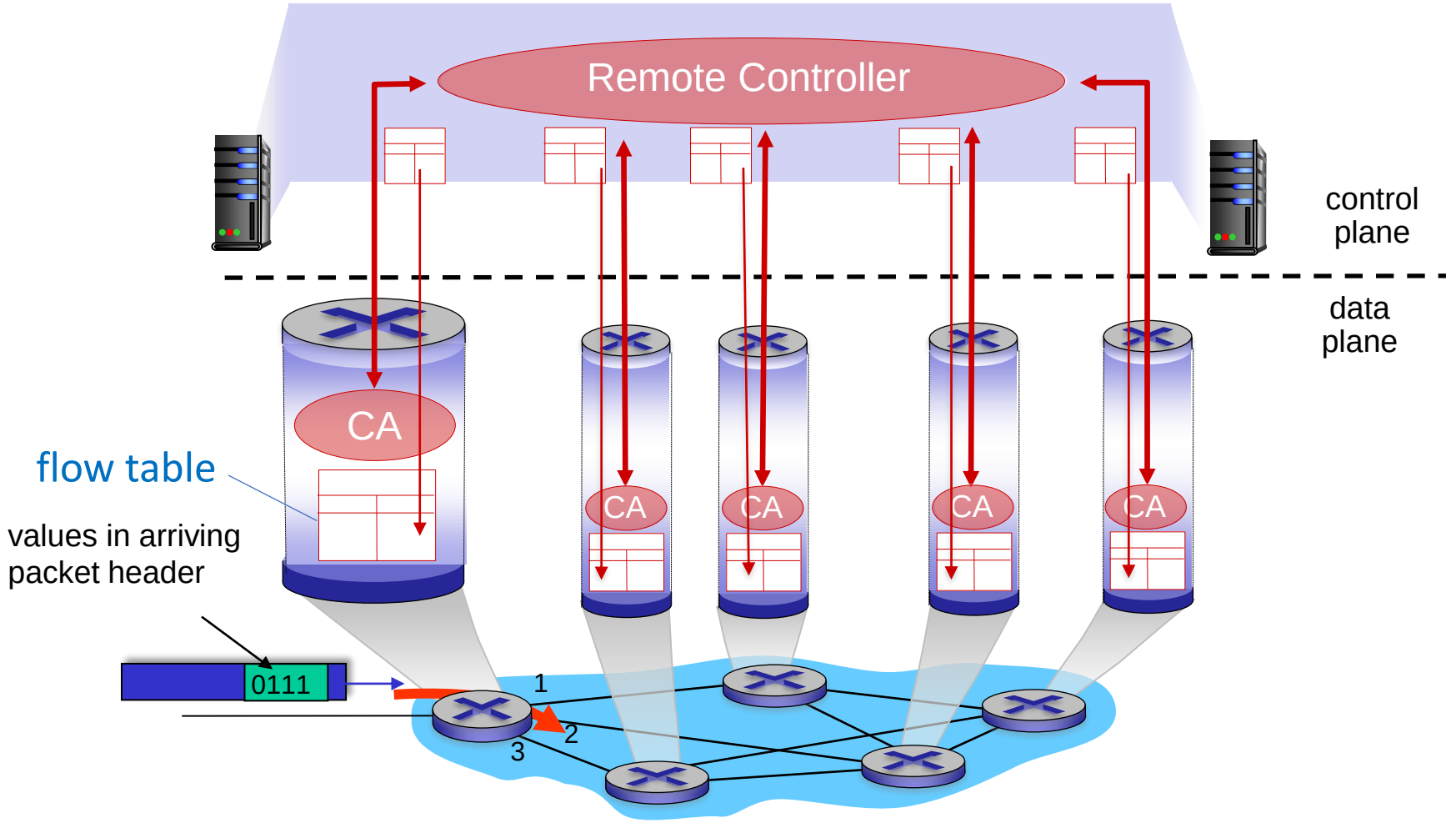
Per-router control plane

Individual routing algorithm components *in each and every router* interact in the control plane



Software-Defined Networking (SDN) control plane

Remote controller computes, installs forwarding tables in routers



A key distinction between per-router control and logically centralized control:

- The CAs (Control Agency) do not directly interact with each other nor do they actively take part in computing the forwarding table.
- Typically, the CA has minimum functionality (communicate with the controller)

Network service model

Q: What *service model* for "channel" transporting datagrams from sender to receiver?

possible services for *individual* datagrams:

- guaranteed delivery
- guaranteed delivery with bounded delay (e.g., less than 40 msec)

possible services for a *flow* of datagrams:

- in-order datagram delivery
- guaranteed minimum bandwidth to flow
- security

This is only a partial list of services that a network layer could provide —there are countless variations possible.

Network-layer service model (However, in the Internet)

Network Architecture	Service Model	Quality of Service (QoS) Guarantees ?			
		Bandwidth	Loss	Order	Timing
Internet	best effort	none	no	no	no

Internet "best effort" service model

No guarantees on:

i. successful datagram delivery to destination
ii. timing or order of delivery
iii. bandwidth available to end-end flow



Network-layer service model



Network Architecture	Service Model	Quality of Service (QoS) Guarantees ?			
		Bandwidth	Loss	Order	Timing
Internet	best effort	none	no	no	no
ATM	Constant Bit Rate	Constant rate	yes	yes	yes
ATM	Available Bit Rate	Guaranteed min	no	yes	no
Internet	Intserv Guaranteed (RFC 1633, 1994)	yes	yes	yes	yes
Internet	Diffserv (RFC 2475, 1998)	possible	possibly	possibly	no

Reflections on best-effort service:

More than 

- simplicity of mechanism has allowed Internet to be widely deployed
- sufficient provisioning of bandwidth allows performance of real-time applications (e.g., interactive voice, video) to be "good enough" for "most of the time"
- replicated, application-layer distributed services (datacenters, CDNs) connecting close to clients' networks, allow services to be provided from multiple locations
- congestion control of "elastic" services helps

It's hard to argue with success of best-effort service model

Network layer: "data plane" roadmap

- Network layer: overview
  - data plane
  - control plane
- What's inside a router
  - input/output ports, switching
  - buffer management
  - scheduling
- IP: the Internet Protocol
  - datagram format, addressing
  - network address translation (NAT)
  - IPv6



- Generalized Forwarding
  - Match+action
  - OpenFlow
  - Middleboxes
- Internet architecture

Router architecture overview



high-level view of generic router architecture:

Routing processor performs control-plane functions.

avoid a centralized processing bottleneck

shadow copy

routing processor

high-speed switching fabric

router input ports

router output ports

routing, management control plane (**software**) operates in millisecond time frame

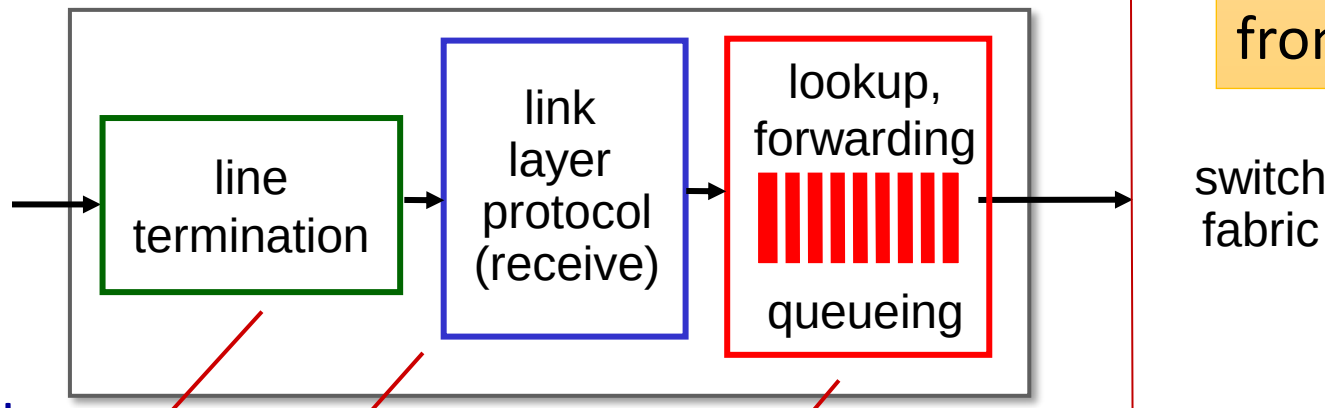
forwarding data plane (**hardware**) operates in nanosecond timeframe

Switching fabric is a network inside a router, connecting input ports to output ports

Shadow copy:
The forwarding table is **copied** from the routing processor to the line cards **over a separate bus**

Input port functions

interface



The forwarding table is either computed and updated by the routing processor or is received from a remote SDN controller.

physical layer:
bit-level reception

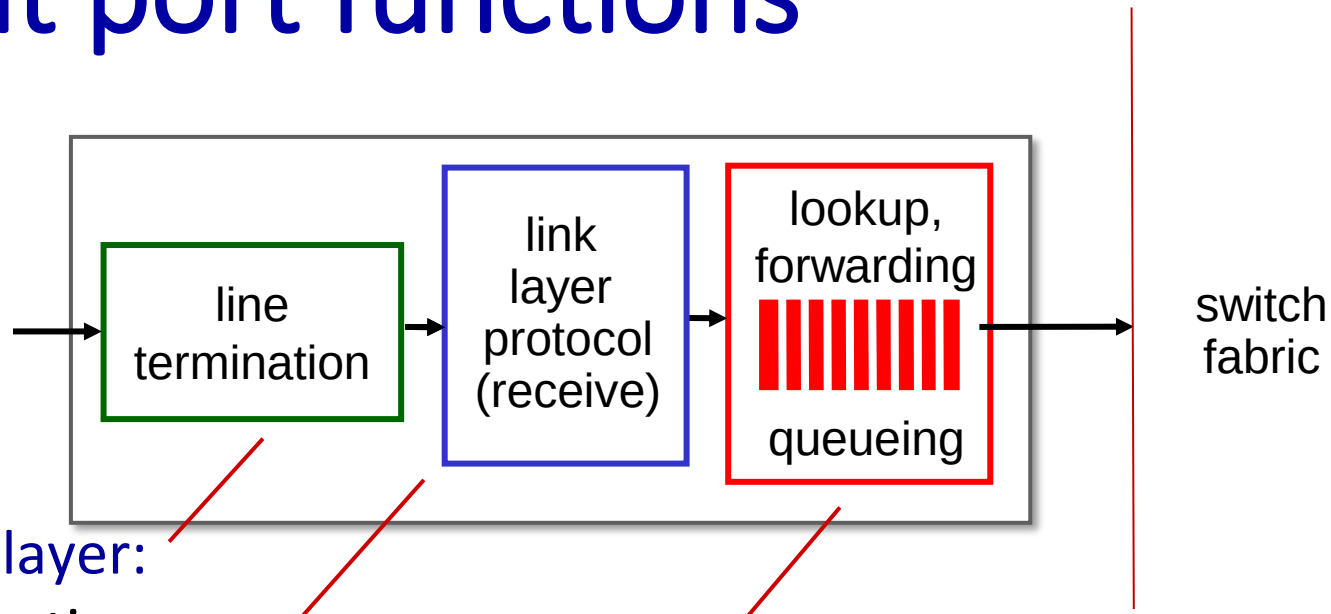
link layer:
e.g., Ethernet

decentralized switching:

- using header field values, lookup output port using forwarding table in input port memory (*"match plus action"*)
- **GOAL**: complete input port processing at 'line speed'
- input port queuing: if datagrams arrive faster than forwarding rate into switch fabric

$T_{lookup} < T_{packet\text{-}receiving}$

Input port functions



physical layer:
bit-level reception

link layer:
e.g., Ethernet

decentralized switching:

- using header field values, lookup output port using forwarding table in input port memory *("match plus action")*
- destination-based forwarding: forward based only on destination IP address (traditional)
- generalized forwarding: forward based on any set of header field values

Destination-based forwarding

forwarding table

Destination Address Range	Link Interface
11001000 00010111 00010000 00000000 through	0
11001000 00010111 00010000 00000100 through 11001000 00010111 00010000 00000111	3
11001000 00010111 00011000 11111111	
11001000 00010111 00011001 00000000 through 11001000 00010111 00011111 11111111	2
otherwise	3

split

Q: but what happens if ranges don't divide up so nicely?

Longest prefix matching

longest prefix match

when looking for forwarding table entry for given destination address, use *longest* address prefix that matches destination address.

Destination Address Range				Link interface
11001000	00010111	00010***	********	0
11001000	00010111	00011000	********	1
11001000	00010111	00011***	********	2
otherwise				3

examples:

11001000	00010111	00010110	10100001	which interface?
11001000	00010111	00011000	10101010	which interface?

Longest prefix matching

longest prefix match

when looking for forwarding table entry for given destination address, use *longest* address prefix that matches destination address.

Destination Address Range	Link interface
11001000 00010111 00010*** ********	0
11001000 00010111 00011000 ********	1
11001000 00010111 00011*** ********	2
otherwise	3

match!

examples:

11001000 00010111 00010110 10100001 which interface?

11001000 00010111 00011000 10101010 which interface?

Longest prefix matching

longest prefix match

when looking for forwarding table entry for given destination address, use *longest* address prefix that matches destination address.

Destination Address Range	Link interface
11001000 00010111 00010*** ********	0
11001000 00010111 00011000 ********	1
11001000 00010111 00011*** ********	2
otherwise	3

match!

examples:

11001000 00010111 00010110 10100001 which interface?

11001000 00010111 00011000 10101010 which interface?

Longest prefix matching

longest prefix match

when looking for forwarding table entry for given destination address, use *longest* address prefix that matches destination address.

Destination Address Range	Link interface
11001000 00010111 00010*** ********	0
11001000 00010111 00011000 ********	1
11001000 00010111 00011*** ********	2
otherwise	3

match!

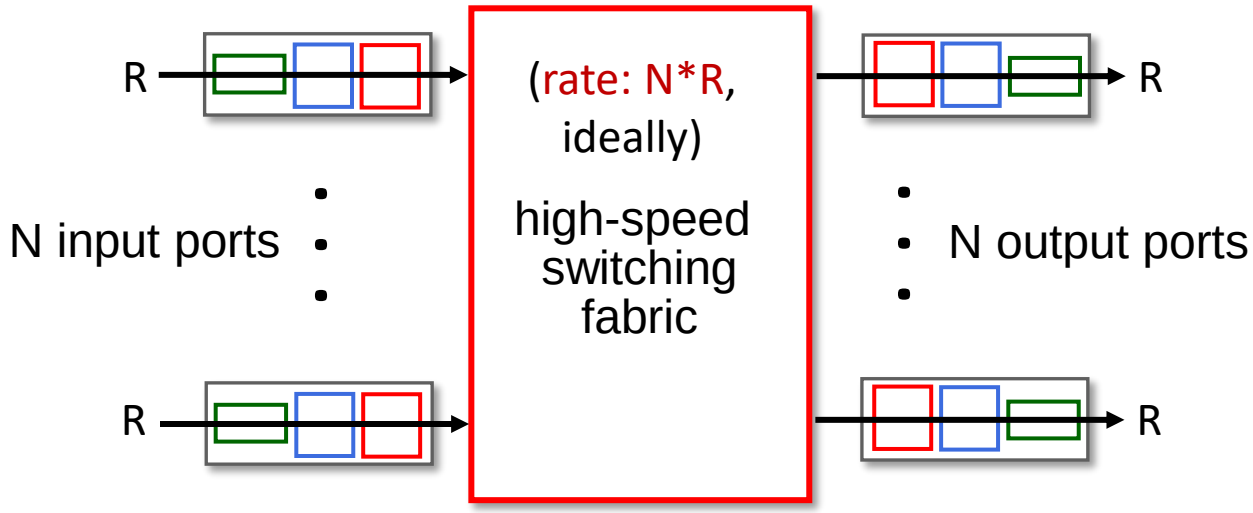
examples:

11001000 00010111 00010110 10100001 which interface?

11001000 00010111 00011000 10101010 which interface?

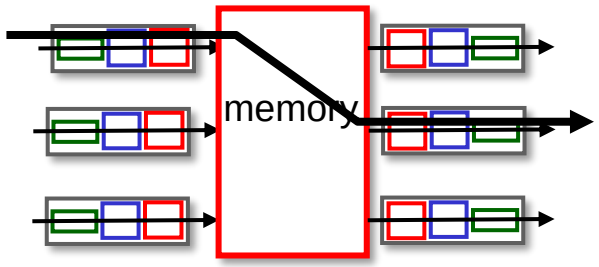
Switching fabrics

- transfer packet from input link to appropriate output link
- switching rate: rate at which packets can be transfered from inputs to outputs
  - often measured as multiple of input/output line rate
  - N inputs: switching rate N times line rate desirable



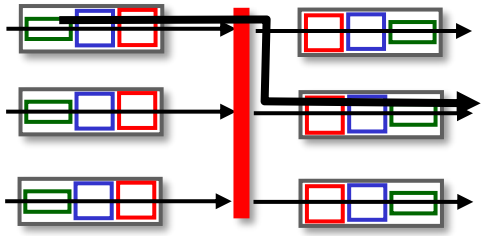
Switching fabrics

- transfer packet from input link to appropriate output link
- switching rate: rate at which packets can be transfer from inputs to outputs
  - often measured as multiple of input/output line rate
  - **N inputs**: switching rate N times line rate desirable
- three major types of switching fabrics:



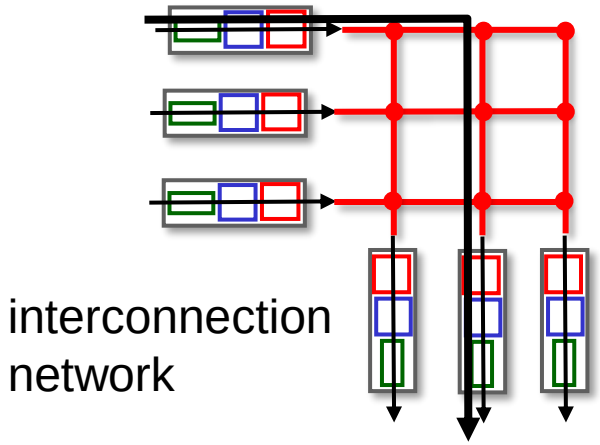
memory

Obsolete as a primary fabric



bus

Mostly obsolete for high-end routers



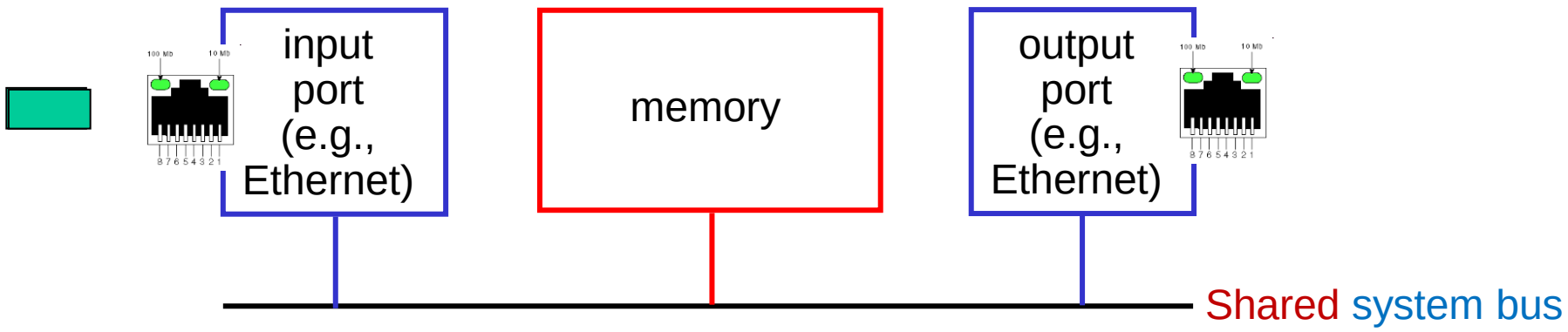
The dominant foundation for modern high-performance routers.

Switching via memory

While "switching via memory" is dead, **memory is more important than ever** for **buffering and queuing**.

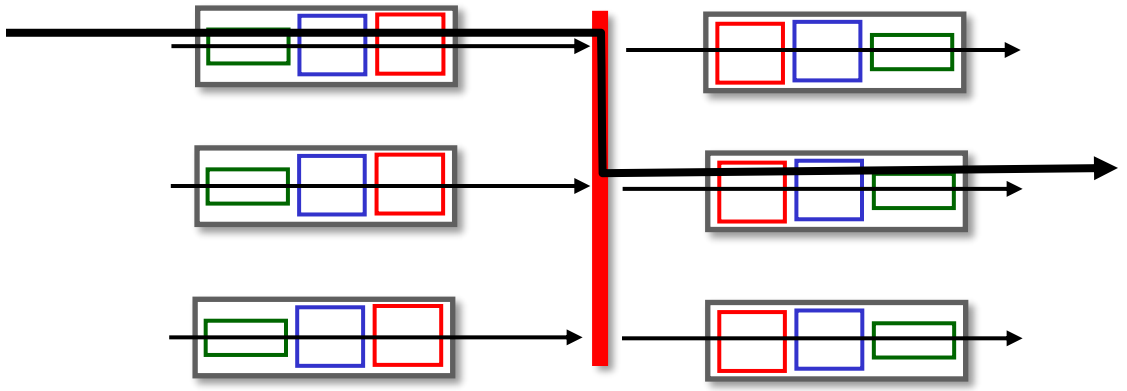
first generation routers:

- The simplest, earliest routers were traditional computers with switching under direct control of CPU
- packet copied to system's memory
- speed limited by memory bandwidth (2 bus crossings per datagram)
- Two packets CANNOT be forwarded at the same time,
  - even if they have different destination ports, since only one memory read/write can be done at a time over the shared system bus



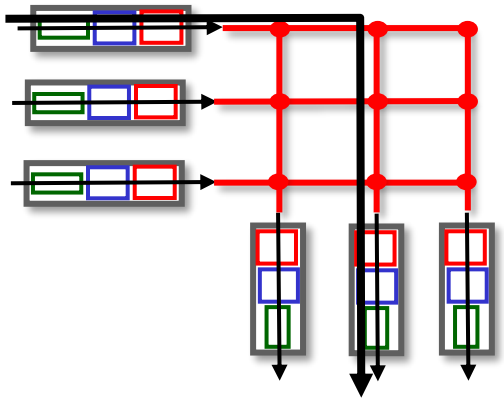
Switching via a bus

- datagram from input port memory to output port memory via a shared bus
- ***bus contention:*** switching speed limited by bus bandwidth
- 32 Gbps bus, Cisco 6500: sufficient speed for access routers
- If multiple packets arrive to the router at the same time, each at a different input port, all but one must wait since only one packet can cross the bus at a time.
- Switching via a bus is often **sufficient** for routers that operate in small local area and enterprise networks.



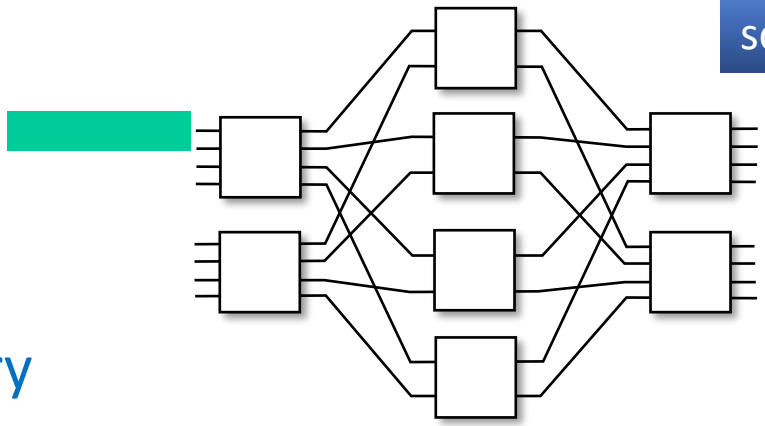
Switching via interconnection network

- **A pure crossbar** can have its own issues (e.g., **head-of-line blocking**).
- Crossbar, Clos networks, other interconnection nets initially developed to connect processors in multiprocessor
- multistage switch: *nxn* switch from multiple stages of smaller switches
- exploiting parallelism (**non-blocking**):
  - fragment datagram into fixed length cells on entry
  - switch cells through the fabric, reassemble datagram at exit



3x3 crossbar

More sophisticated

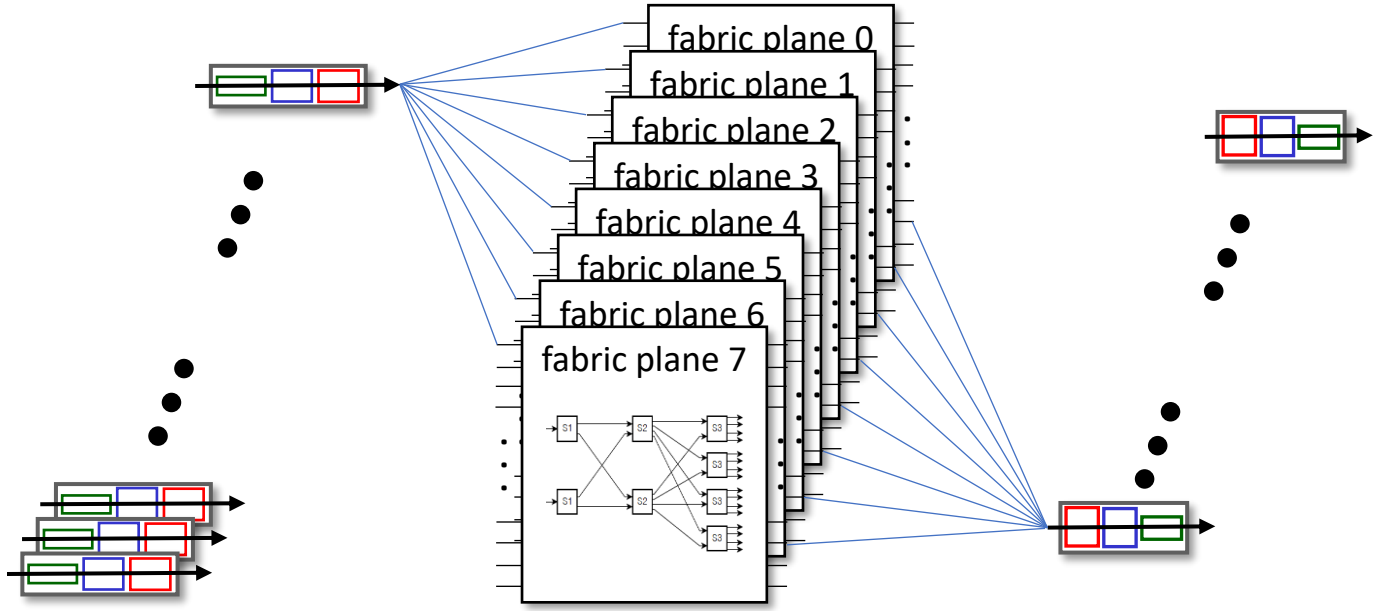


8x8 multistage switch
built from smaller-sized switches

Switching via interconnection network

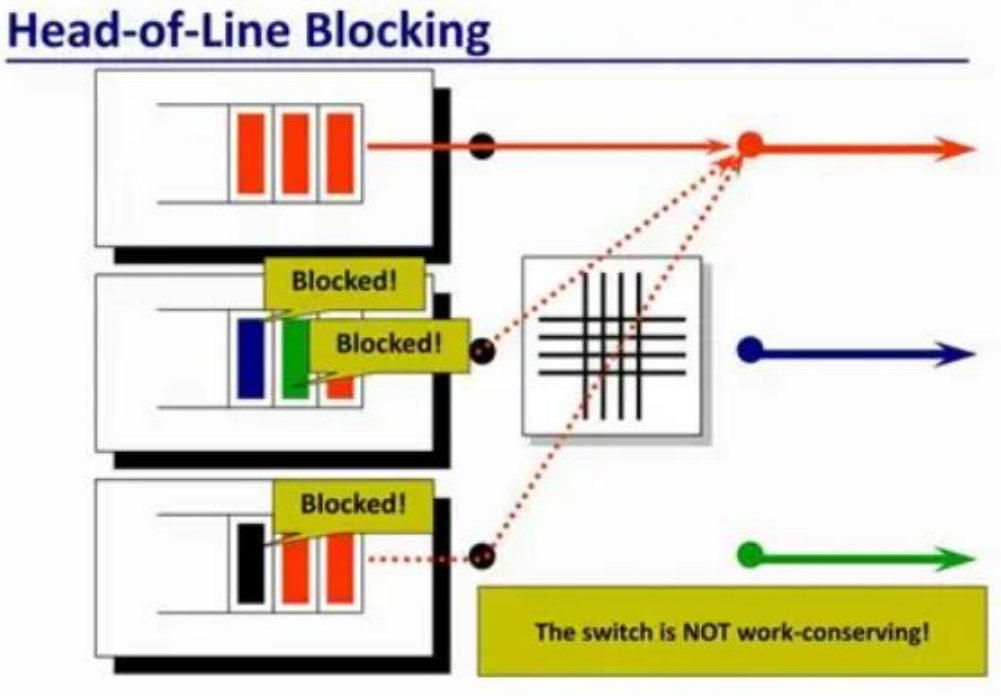
- scaling, using multiple switching "planes" in parallel:
  - speedup, scaleup via parallelism

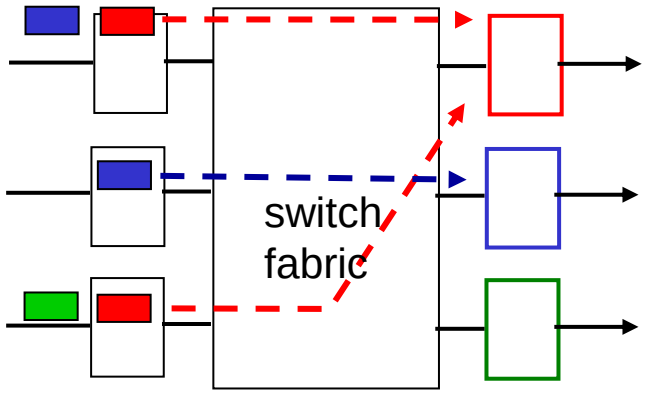
- Cisco CRS router:
  - basic unit: 8 switching planes
  - each plane: 3-stage interconnection network
  - up to 100's Tbps switching capacity



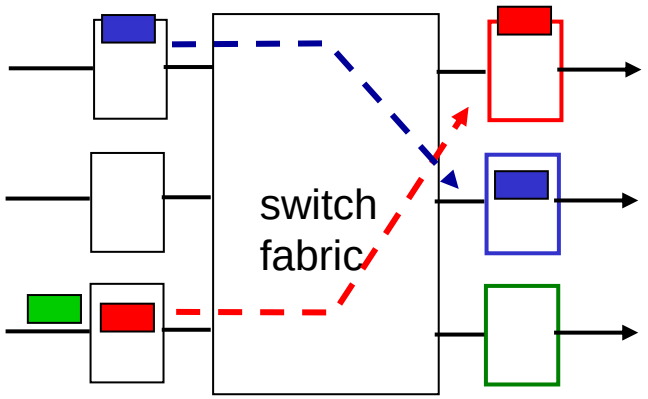
Input port queuing

- If switch fabric slower than input ports combined → queueing may occur at input queues
  - queueing delay and loss due to input buffer overflow!
- Head-of-the-Line (HOL) blocking: queued datagram at front of queue prevents others in queue from moving forward



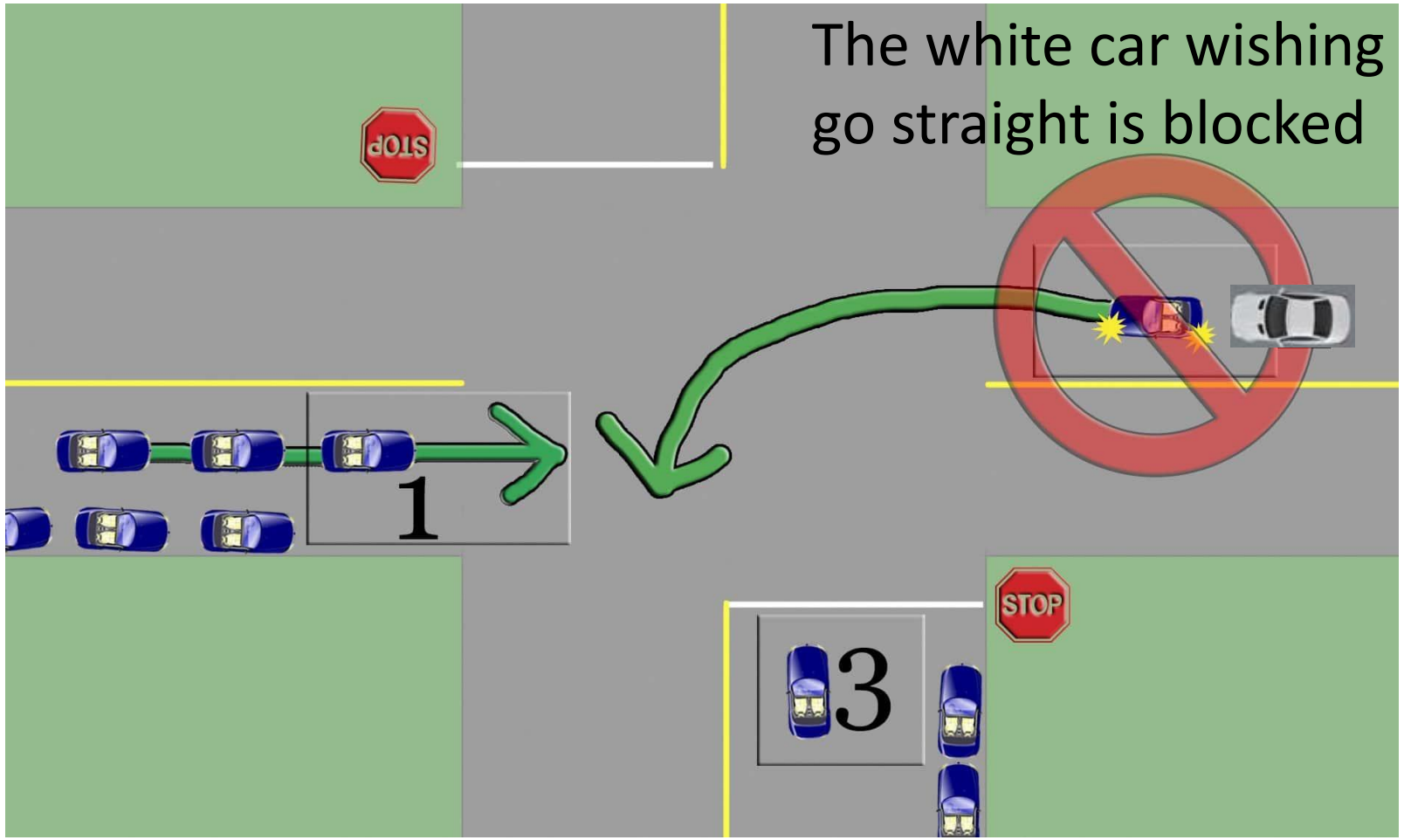


output port contention:
only one red datagram can be transferred.
lower red packet is *blocked*



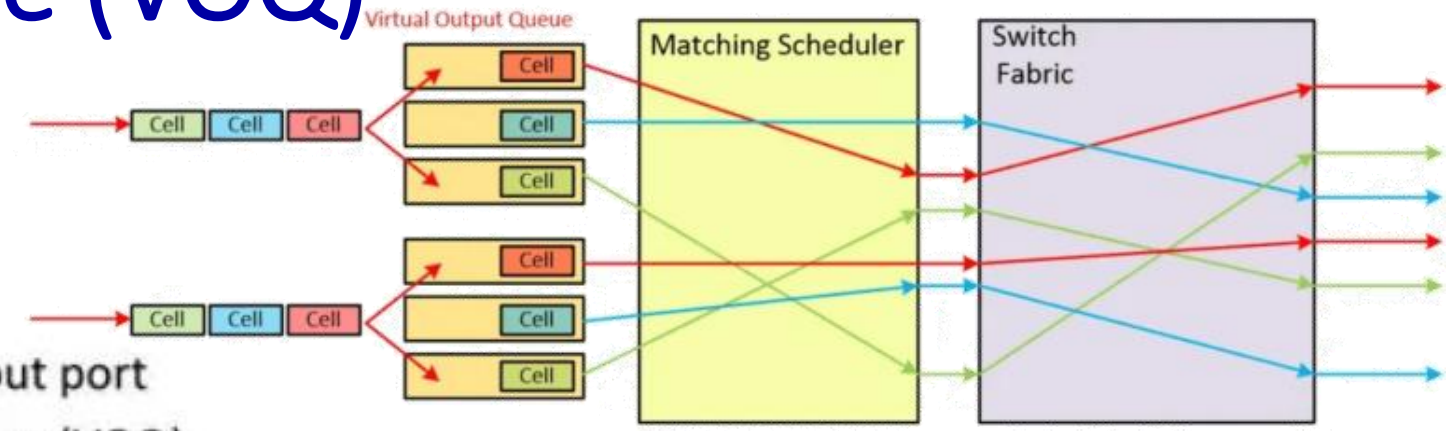
one packet time later: green packet experiences HOL blocking

Analog of HOL blocking



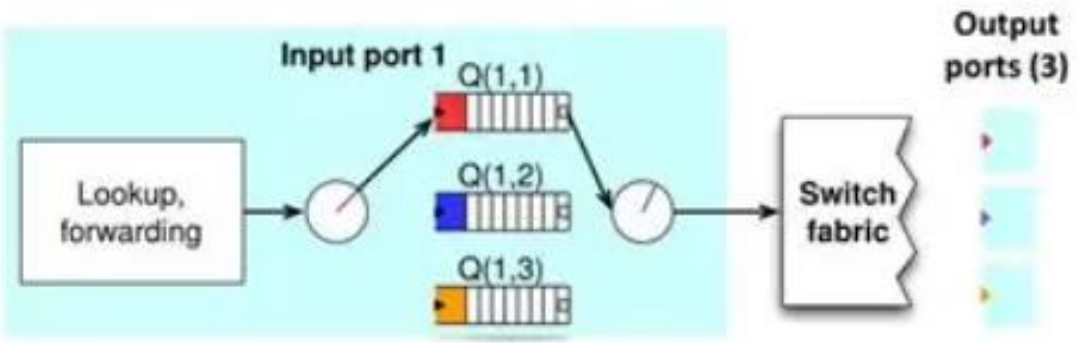
The consequence of HOL blocking:
The switch's throughput is dramatically reduced, preventing the switch from making efficient use of all its available output ports.

The solution of HOL blocking: Virtual Output Queue (VOQ)

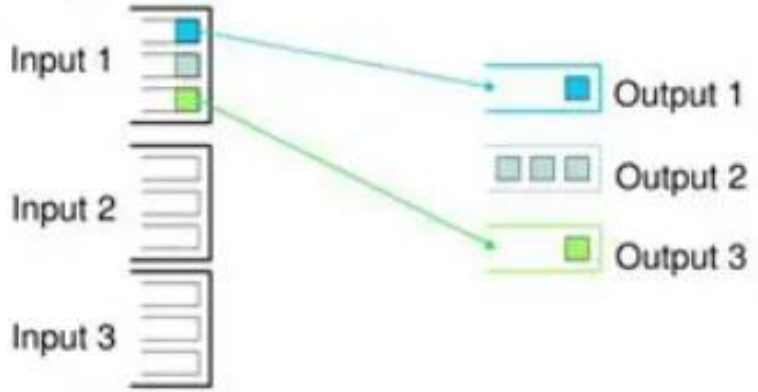


Virtual output queuing

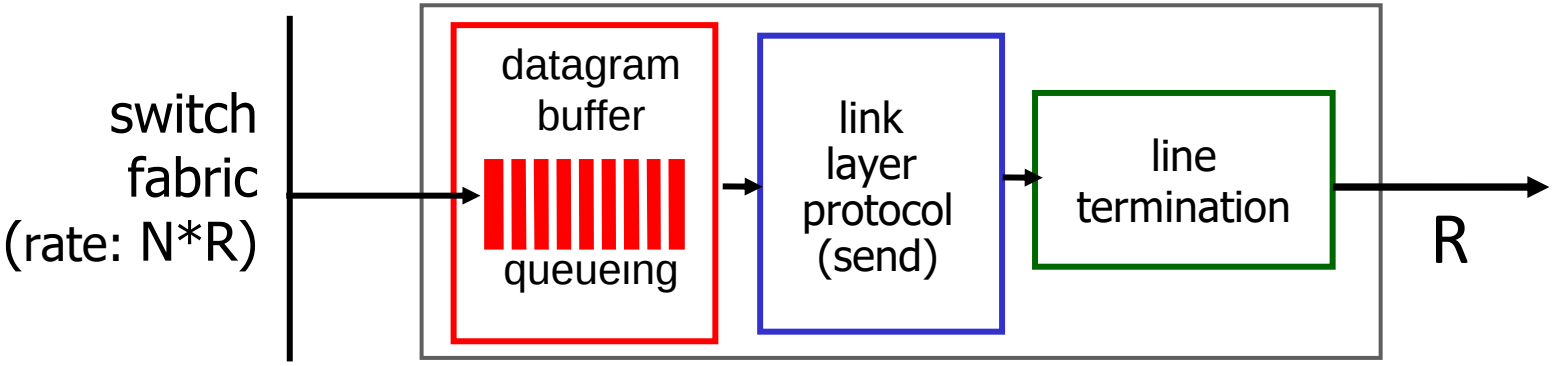
- On each input port, one input queue per output port
- Input port places packet in virtual output queue (VOQ) corresponding to output port of forwarding decision
  - ✓ No head-of-line blocking, no output queuing
  - – Need to schedule fabric



- Maintain N virtual queues at each input
  - – one per output



Output port queuing





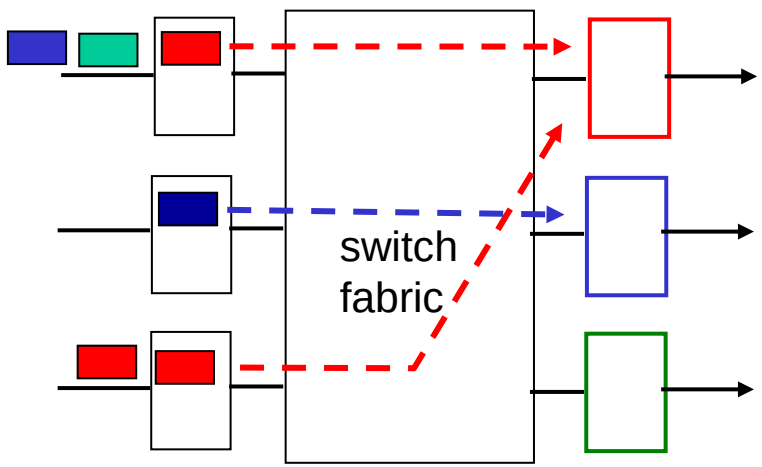
This is a really important slide

- *Buffering* required when datagrams arrive from fabric faster than link transmission rate.
- *Drop policy:* which datagrams to drop if no free buffers?
- *Scheduling discipline* chooses among queued datagrams for transmission

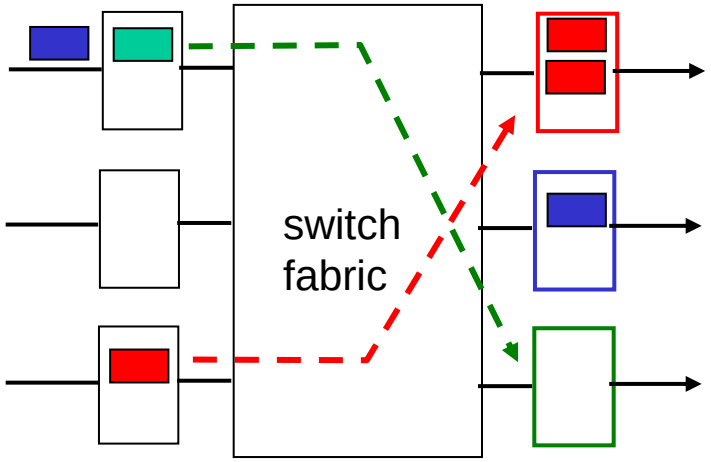
→ Datagrams can be lost due to congestion, lack of buffers

→ Priority scheduling – who gets best performance, network neutrality

Output port queuing



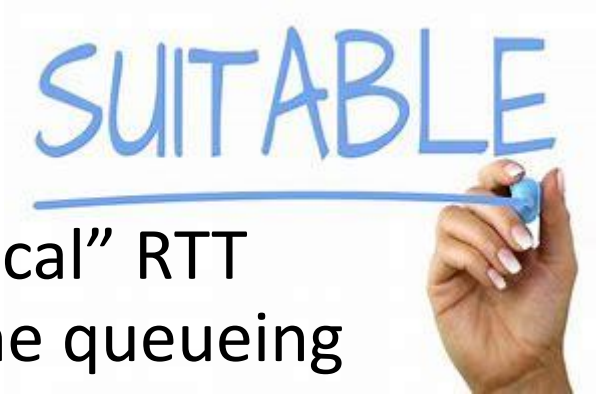
at *t*, packets more from input to output



one packet time later

- buffering when arrival rate via switch exceeds output line speed
- *queueing (delay) and loss due to output port buffer overflow!*

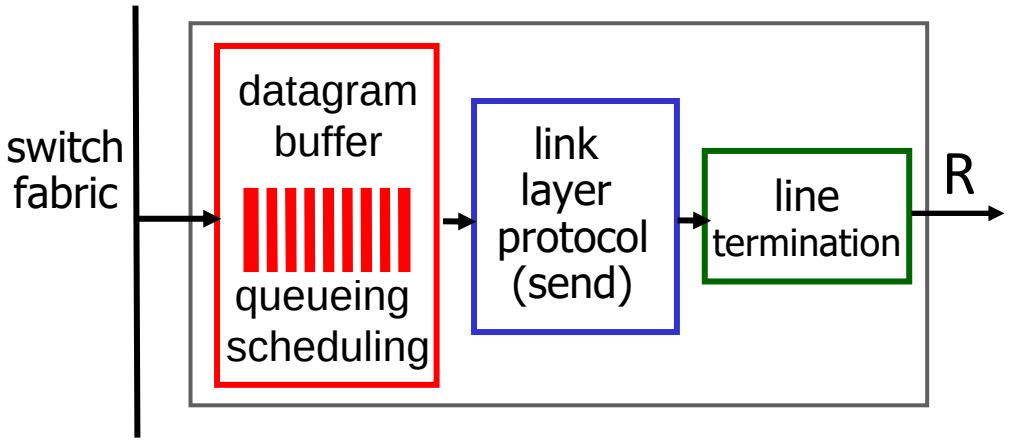
How much buffering?

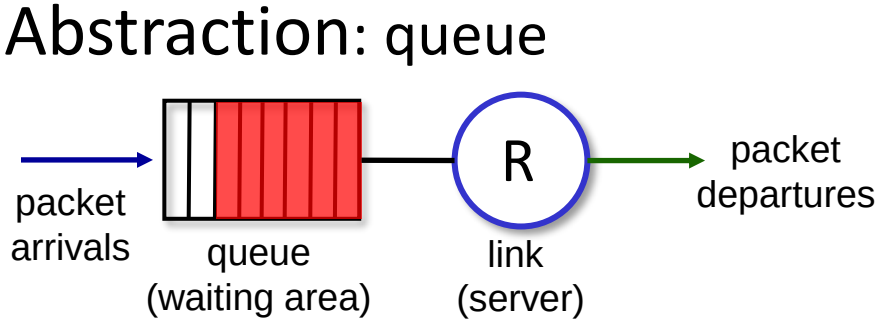


- RFC 3439 rule of thumb: average buffering equal to "typical" RTT (say 250 msec) times link capacity C（on an analysis of the queueing dynamics of a relatively small number of TCP flows.）
  - e.g., C = 10 Gbps link: 2.5 Gbit buffer
- more recent recommendation: with a large number of *N* TCP flows, buffering equal to $\frac{RTT \cdot C}{\sqrt{N}}$
- but *too* much buffering can increase delays (particularly in home routers)
  - long RTTs: poor performance for real-time apps, sluggish TCP response
  - recall delay-based congestion control: "keep bottleneck link just full enough (busy) but no fuller"



Buffer Management





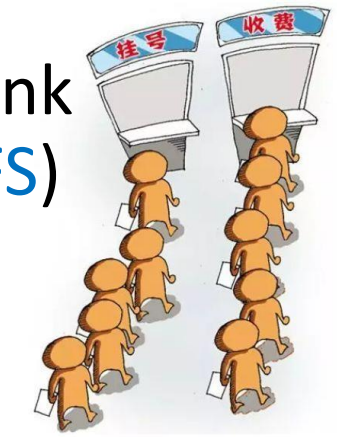
buffer management:

- drop: which packet to add, drop when buffers are full
  - tail drop: drop arriving packet
  - priority: drop/remove on priority basis
- marking: which packets to mark to signal congestion (ECN, RED)

Packet Scheduling: FCFS

packet scheduling: deciding which packet to send next on link

- first come, first served (FCFS)
- priority
- round robin
- weighted fair queueing

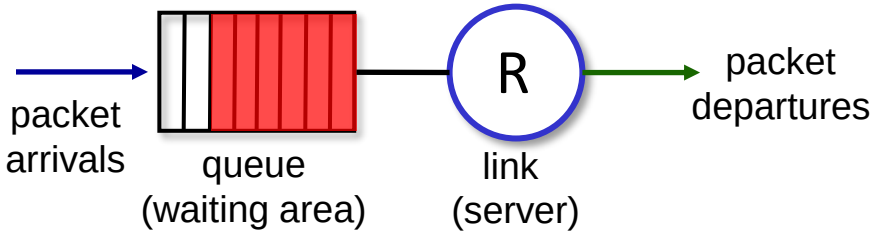


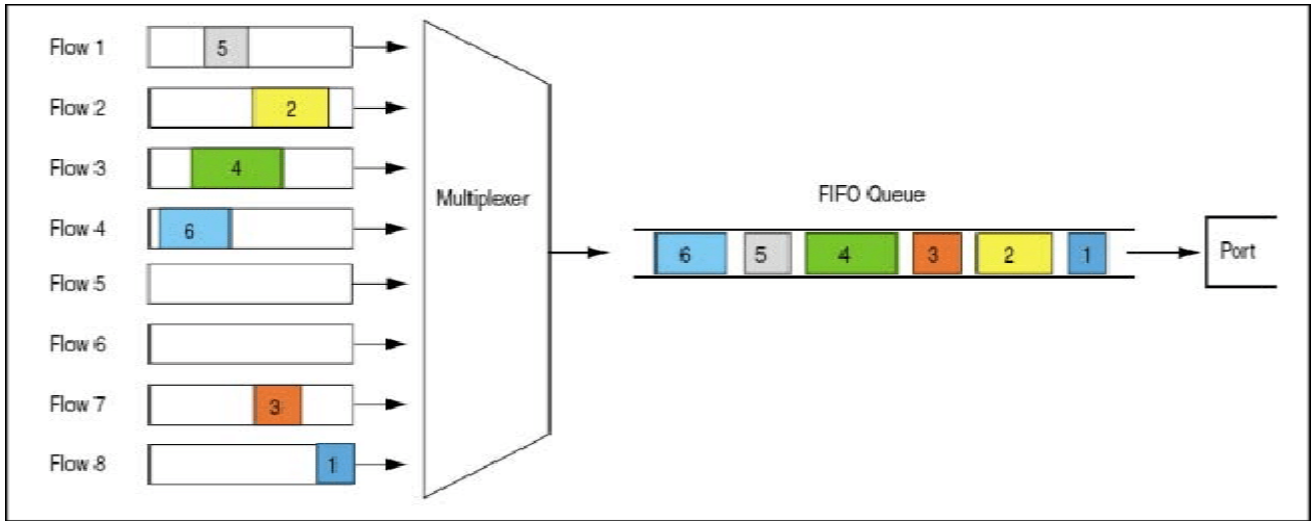
FCFS: packets transmitted in order of arrival to output port

- also known as: First-in-first-out (FIFO)
- real world examples?



Abstraction: queue

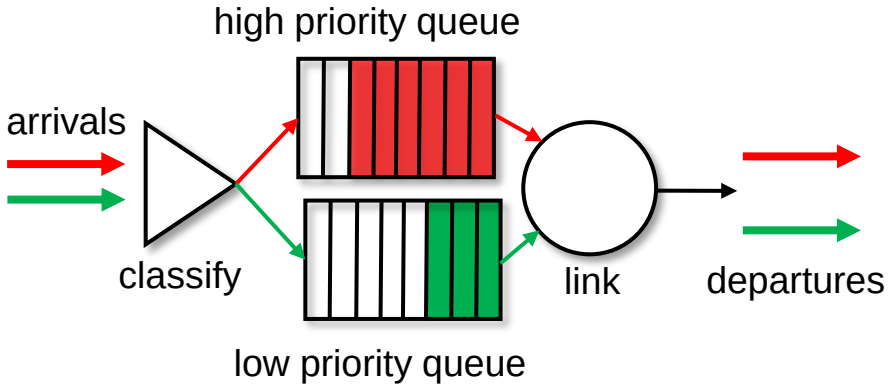


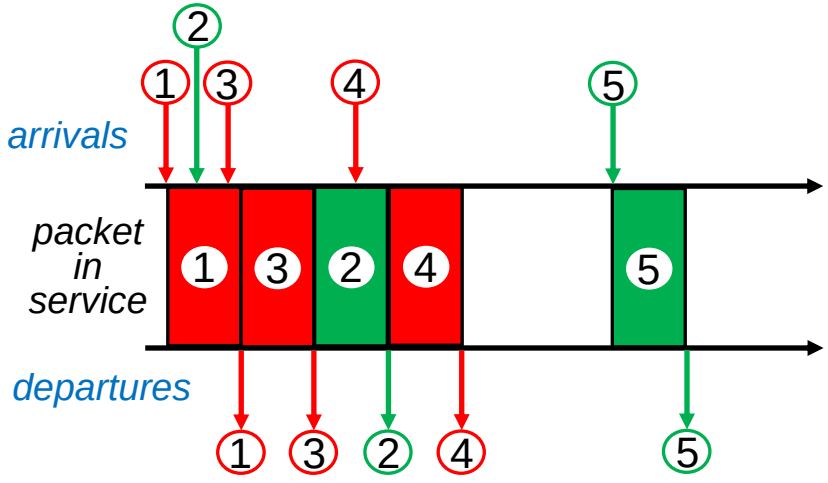


Scheduling policies: priority

Priority scheduling:

- arriving traffic classified, queued by class
  - any header fields can be used for classification
- send packet from highest priority queue that has buffered packets
  - FCFS within priority class

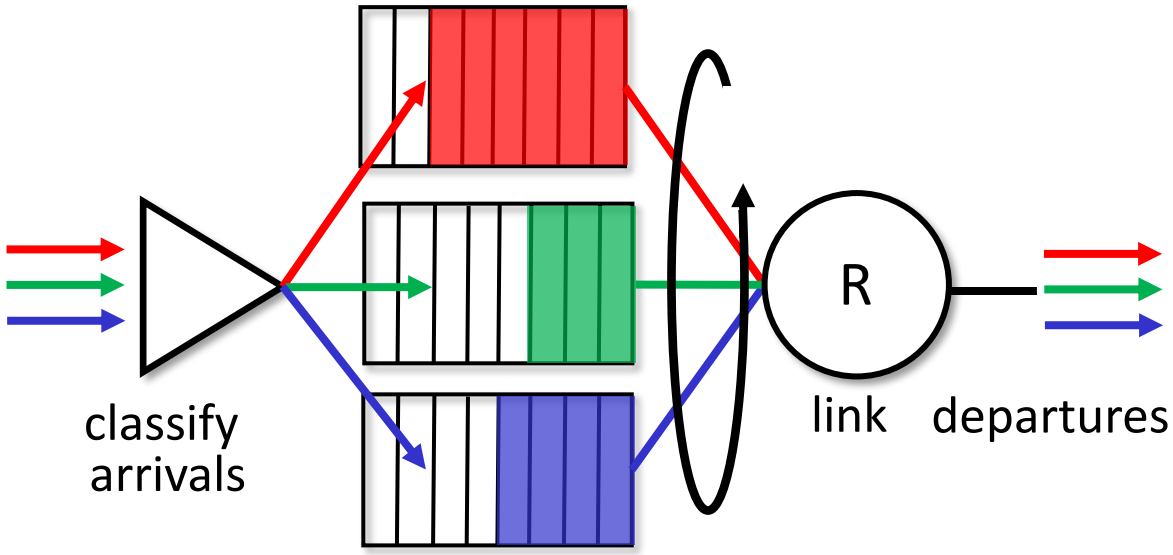


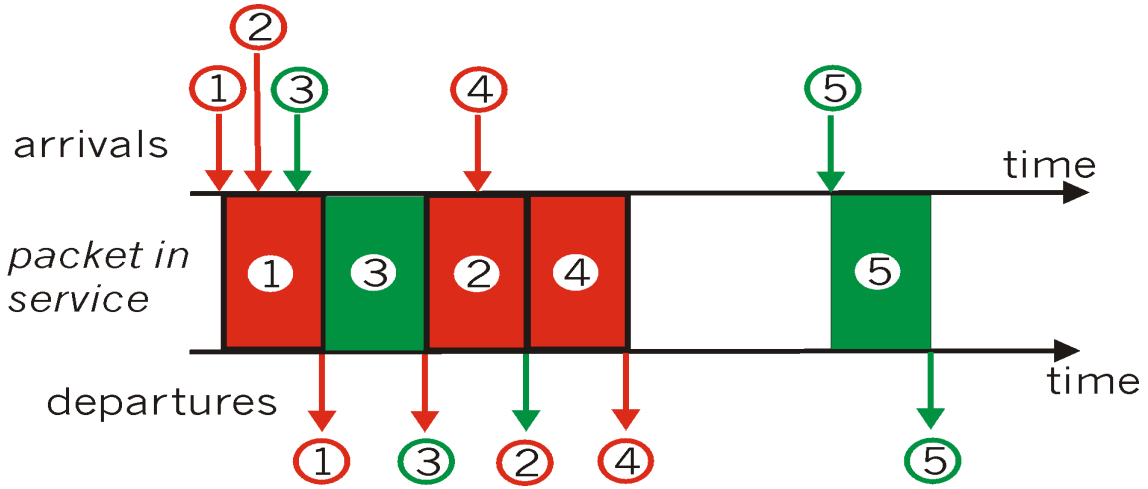


Scheduling policies: round robin

Round Robin (RR) scheduling:

- arriving traffic classified, queued by class
  - any header fields can be used for classification
- server cyclically, repeatedly scans class queues, sending one complete packet from each class (if available) in turn
- if one class to be served is found none, immediately check the next class
- unfair to short packets and has no priority

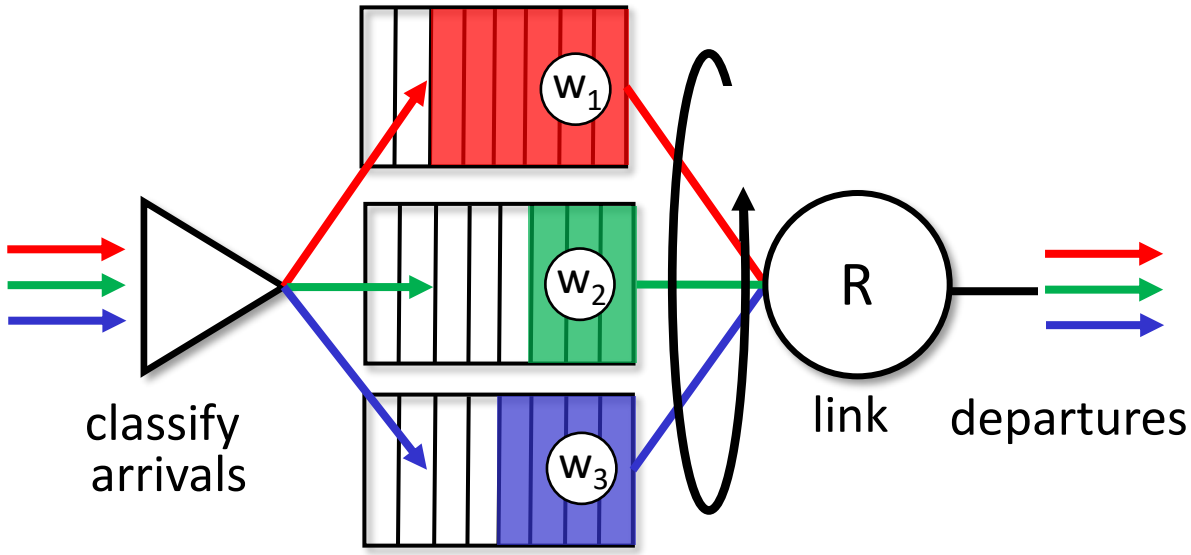




Scheduling policies: weighted fair queueing

Weighted Fair Queuing (WFQ):

- generalized Round Robin
- each class, *i*, has weight, w_i, and gets weighted amount of service in each cycle: $\frac{w_i}{\sum_j w_j}$



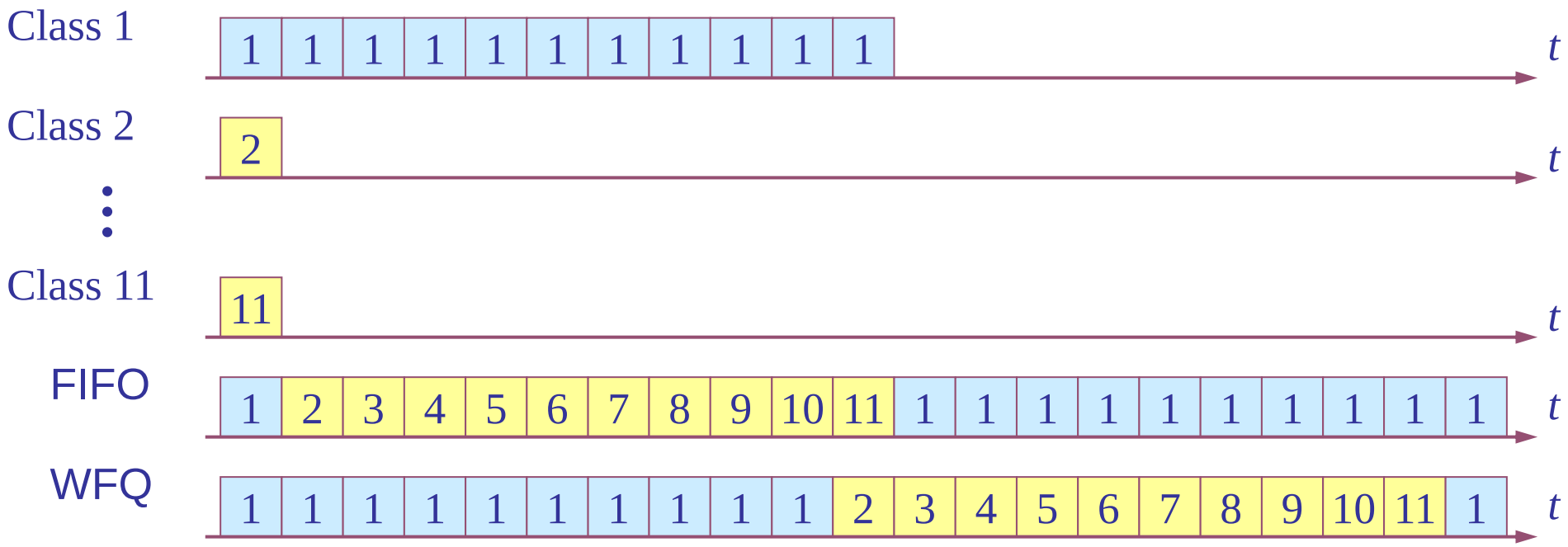
- minimum bandwidth guarantee (per-traffic-class)
  - guaranteed bandwidth R_i for class i

$$R_i = \frac{R \times w_i}{\sum w_j}$$

Comparison between WFQ and FIFO (1)

(a) Packets of class 1 consecutively arrive:

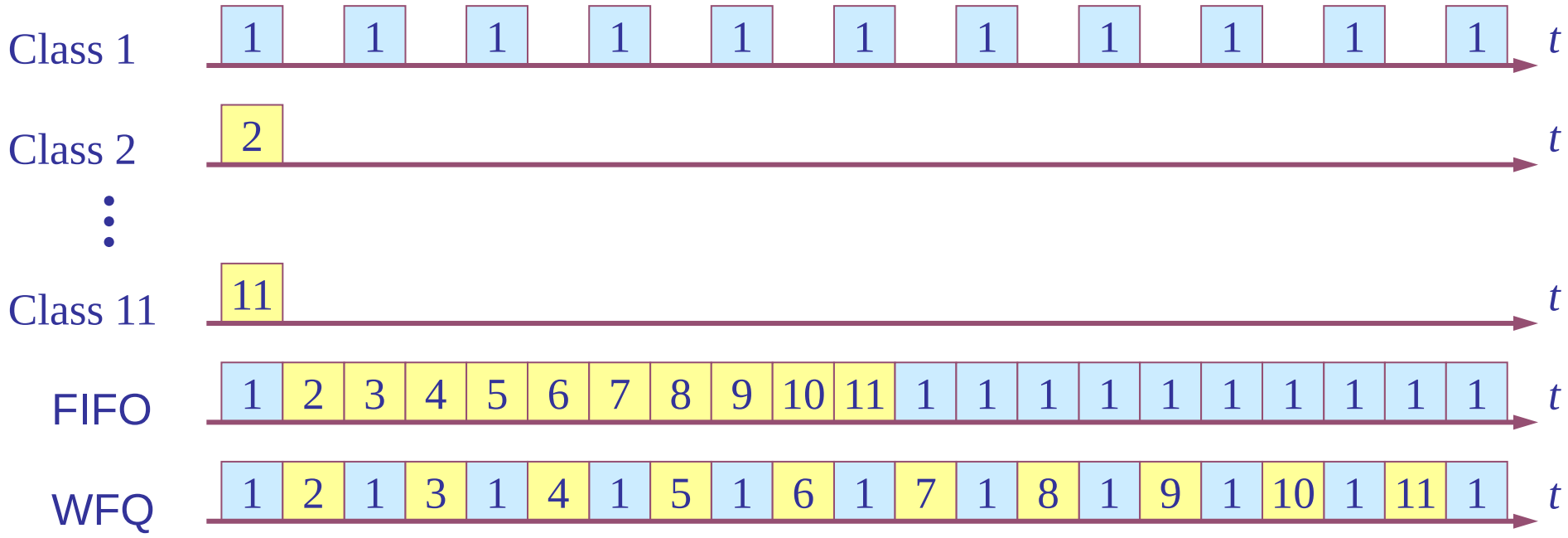
w_1=0.5, w_i = 0.05 (i=2 ~ 11)



Comparison between WFQ and FIFO (2)

(b) Packets of class 1 arrive in a discrete manner:

w_1=0.5, w_i = 0.05 (i=2 ~ 11)



Summary: in either case (a) or (b), FIFO results are the same; WFQ scheduler can always guarantee that packets in class 1 receive more services.

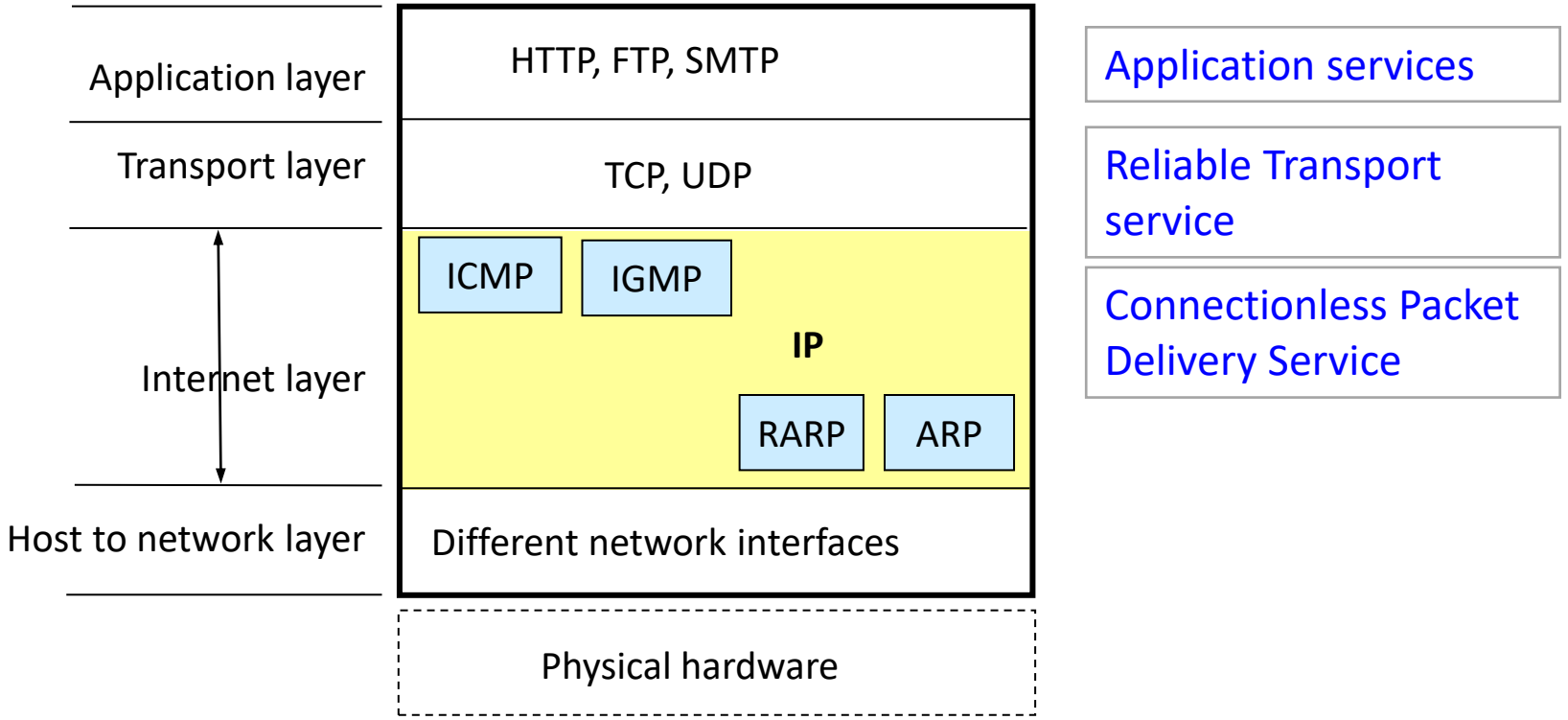
Network layer: "data plane" roadmap

- Network layer: overview
  - data plane
  - control plane
- What's inside a router
  - input/output ports, switching
  - buffer management
  - scheduling
- IP: the Internet Protocol
  - datagram format, addressing
  - network address translation (NAT)
  - IPv6

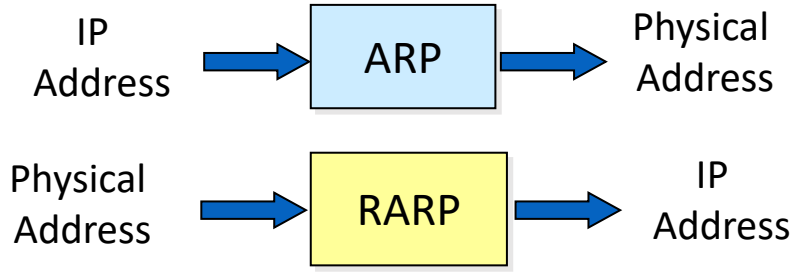


- Generalized Forwarding
  - Match+action
  - OpenFlow
  - Middleboxes
- Internet architecture

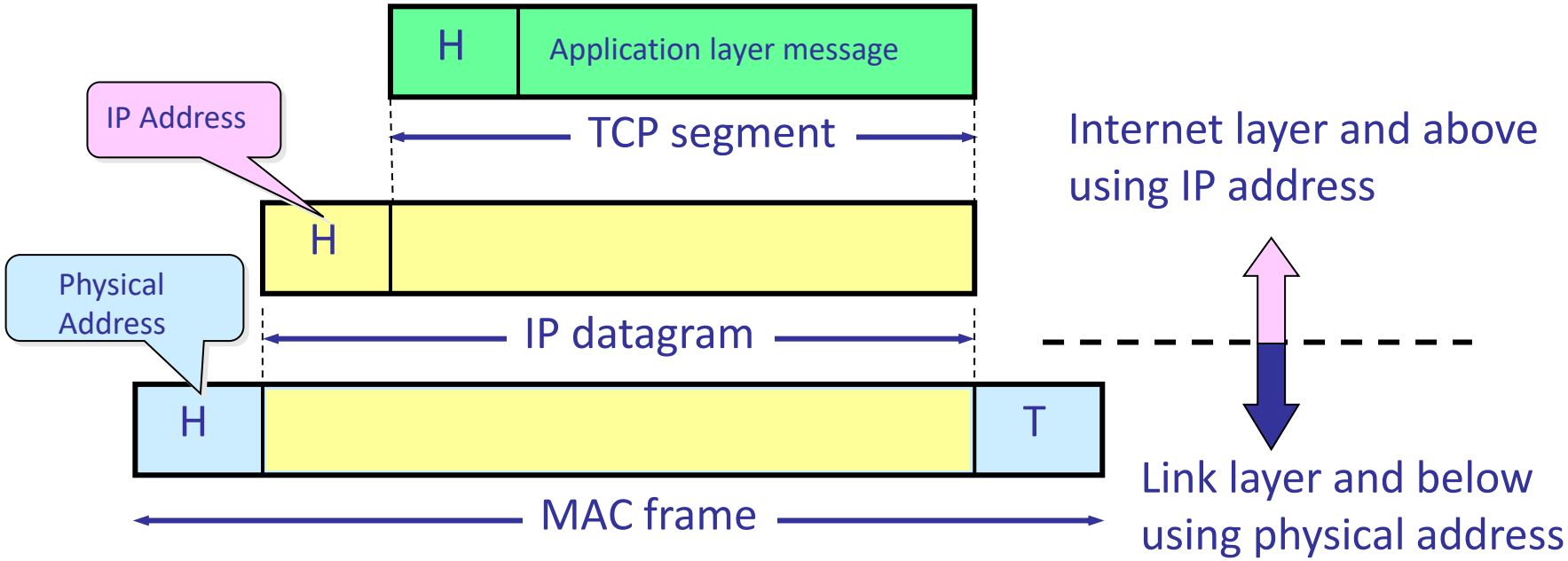
Internet Protocol (IP)



ARP (Address Resolution Protocol)
RARP (Reverse Address Resolution Protocol)
ICMP (Internet Control Message Protocol)
IGMP (Internet Group Management Protocol)



IP Address and Physical Address



Two machines on a given physical network can communicate only if they know each other's physical network address.



is to devise software that ❶hides physical addresses and ❷allows higher-level programs to work only with Internet addresses.

MAC Addresses

- 32-bit IP address:
  - *network-layer* address
  - used to get datagram to destination IP subnet
- MAC (Media Access Control) address, or LAN or physical or Ethernet address:
  - function: *get frame from one interface to another physically-connected interface (same network)*
  - 48 bit MAC address (for most LANs)
    - Each adapter on LAN has unique LAN address
    - Burned in Network Interface Card (NIC) ROM, also sometimes software settable

MAC Address (more)

- MAC address allocation administered by IEEE
- manufacturer buys portion of MAC address space (to assure uniqueness)
- analogy:

  (a) MAC address: like Social Security Number

  (b) IP address: like postal address





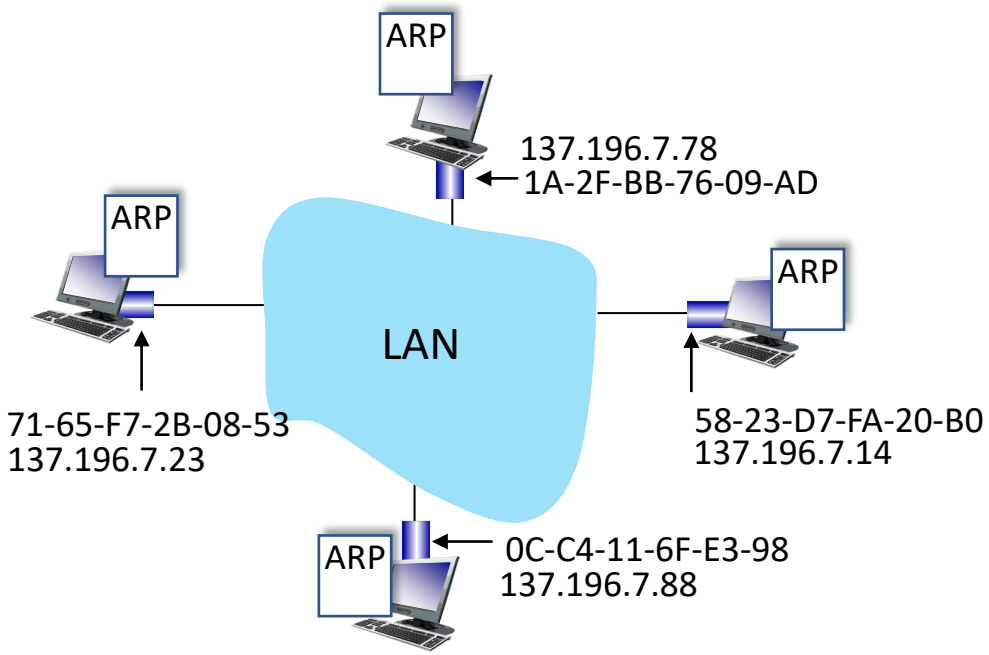
- MAC flat address ➜ portability
  - can move LAN card from one LAN to another
- IP hierarchical address NOT portable
  - address depends on IP subnet to which node is attached

ARP: Address Resolution Protocol (1)

- ARP permits machines to resolve addresses without keeping a permanent record of bindings.
- To avoid maintaining a centralized database, the designers chose to use a low-level protocol (ARP) to bind addresses dynamically.

ARP: address resolution protocol (2)

Question: how to determine interface's MAC address, knowing its IP address?



ARP table: each IP node (e.g., host, router,…) on LAN has table

- IP/MAC address mappings for some LAN nodes:

  < IP address; MAC address; TTL>

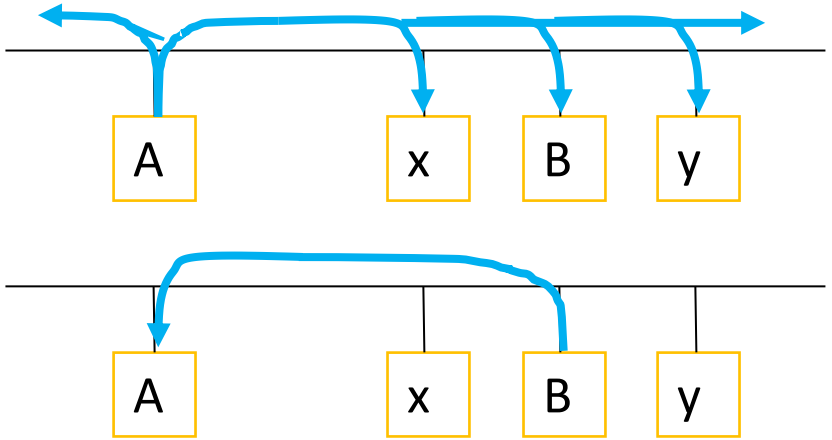
- TTL (Time To Live): time after which address mapping will be forgotten (typically 20 min)

ARP protocol: Same LAN (network)

- A wants to send datagram to B, and B's MAC address NOT in A's ARP table.
- A broadcasts ARP query packet, containing B's IP address
  - Dest. MAC address = FF-FF-FF-FF-FF-FF
  - all machines on LAN receive ARP query
- B receives ARP packet, replies to A with its (B's) MAC address
  - frame sent to A's MAC address (unicast)



- A caches (saves) IP-to-MAC address pair in its ARP table until information becomes old (times out)
  - soft state: information that times out (goes away) unless refreshed
- ARP is "plug-and-play":
  - nodes create their ARP tables *without intervention from net administrator*



ARP protocol in action

example: A wants to send datagram to B

- B's MAC address not in A's ARP table, so A uses ARP to find B's MAC address

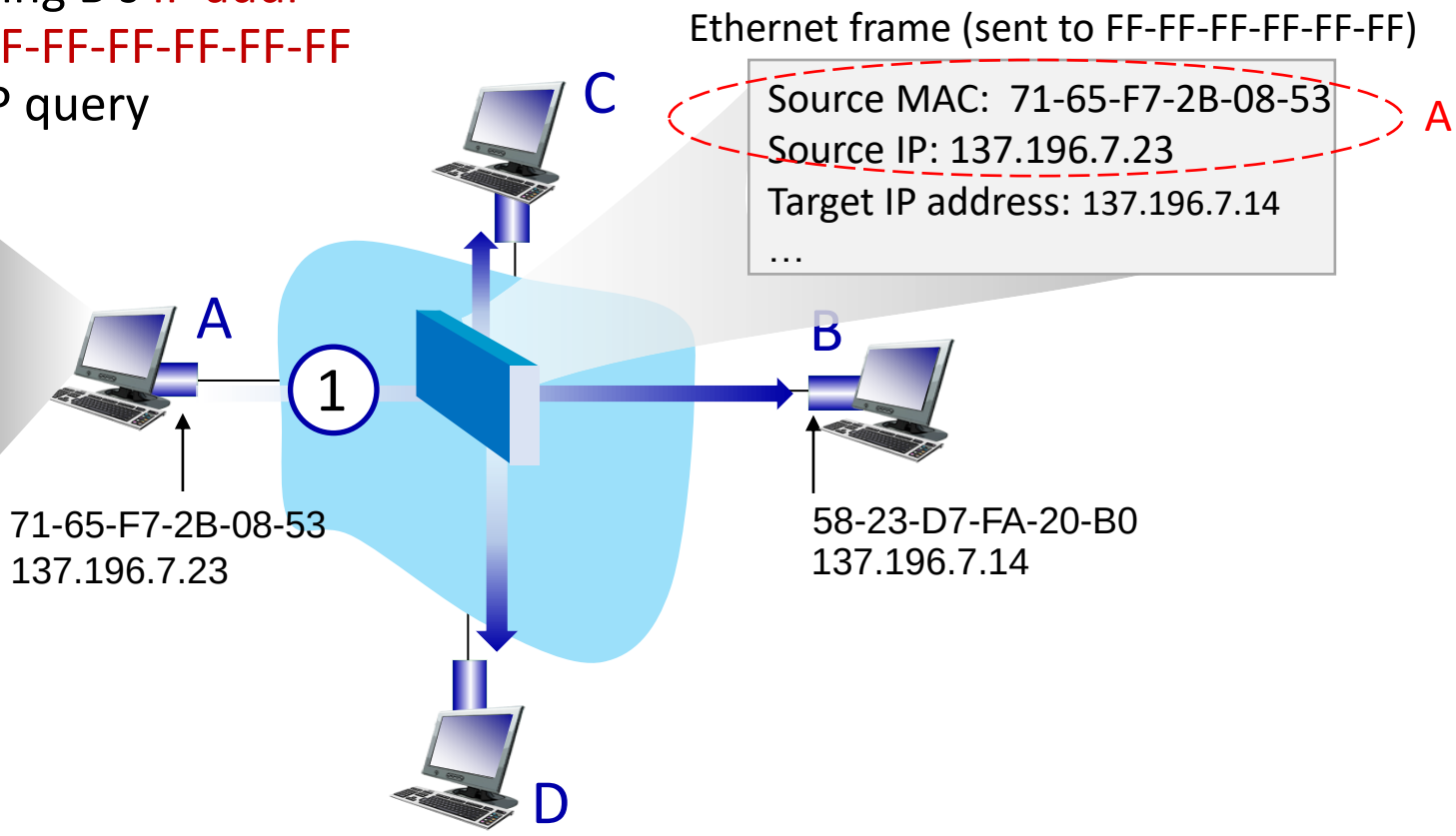
A broadcasts ARP query, containing B's IP addr

1

- destination MAC address = FF-FF-FF-FF-FF-FF
- all nodes on LAN receive ARP query

ARP table in A

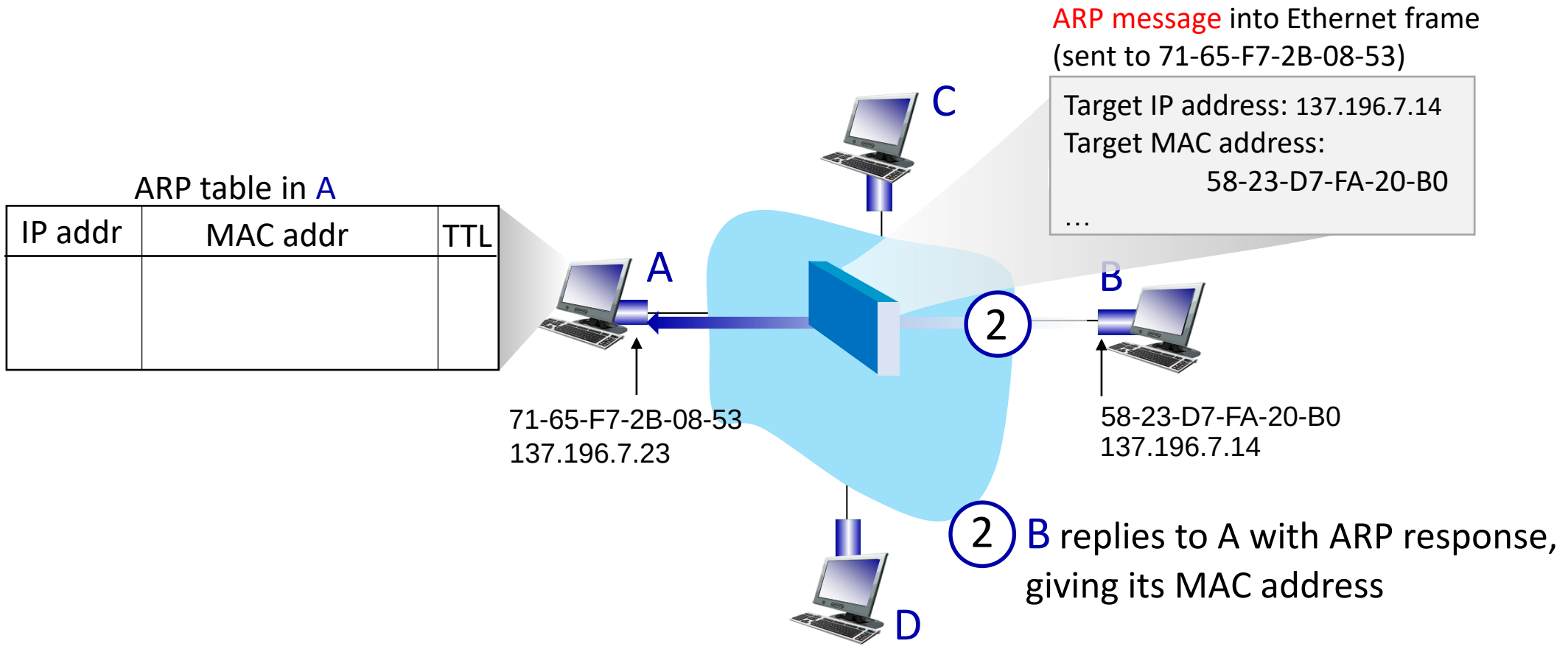
IP addr	MAC addr	TTL



ARP protocol in action

example: A wants to send datagram to B

- B's MAC address not in A's ARP table, so A uses ARP to find B's MAC address



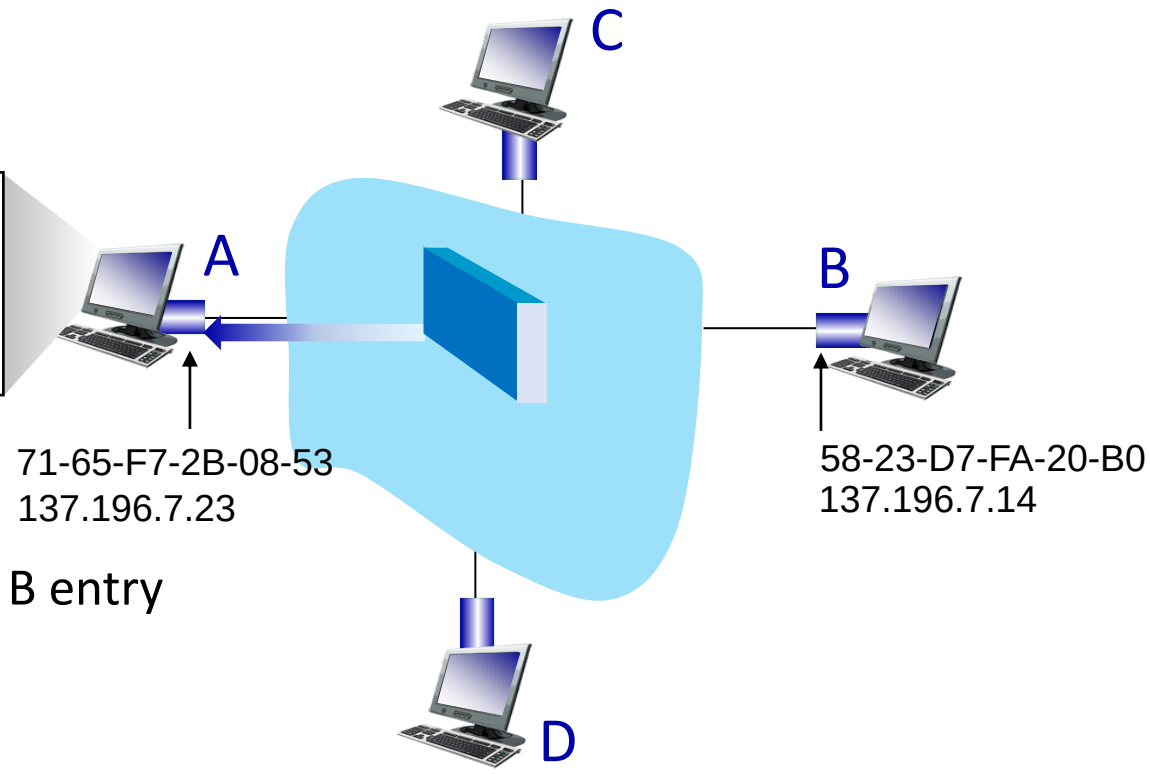
ARP protocol in action

example: A wants to send datagram to B

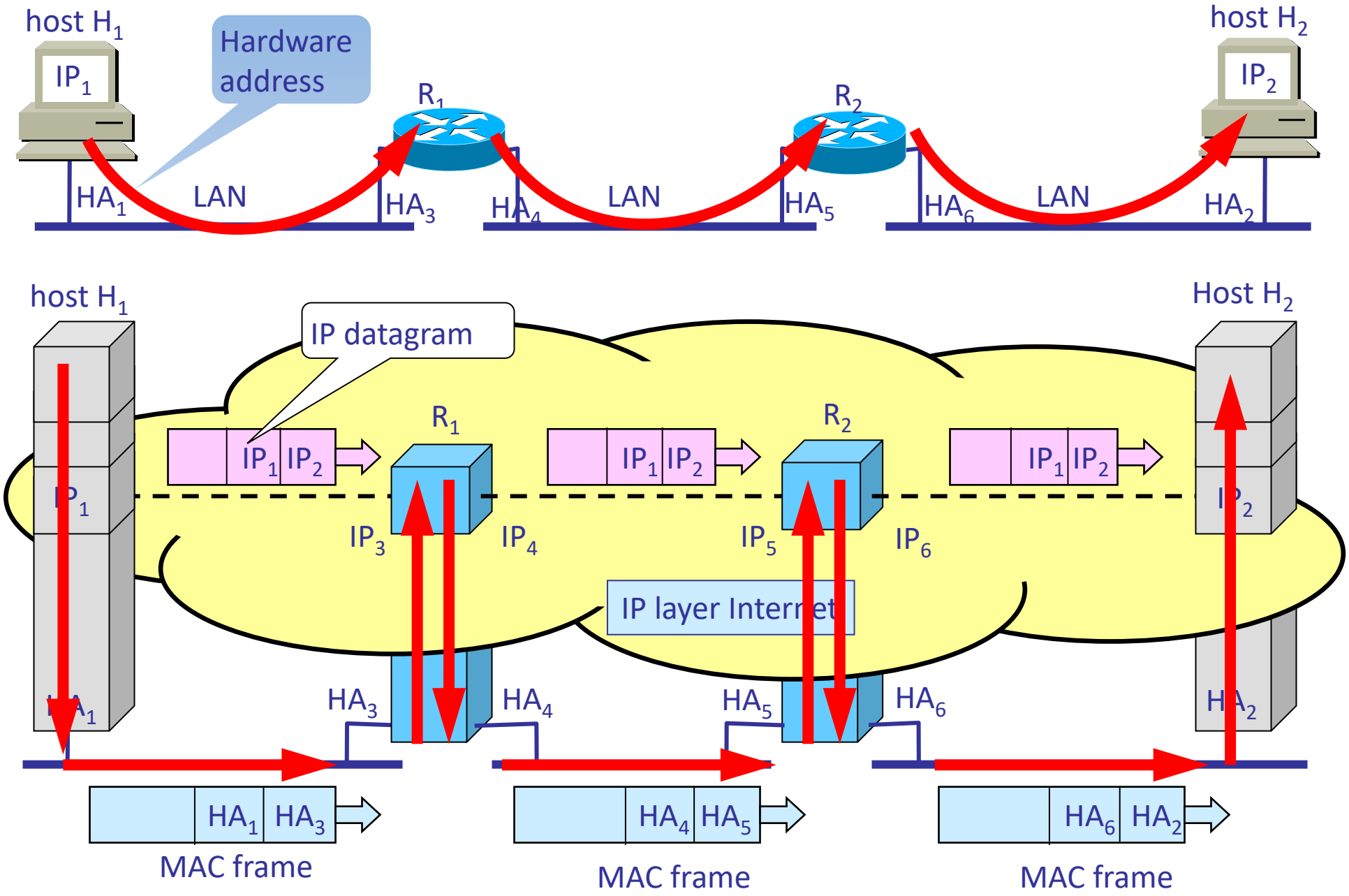
- B's MAC address not in A's ARP table, so A uses ARP to find B's MAC address

ARP table in A

IP addr	MAC addr	TTL
137.196.7.14	58-23-D7-FA-20-B0	500



③ A receives B's reply, adds B entry into its local ARP table

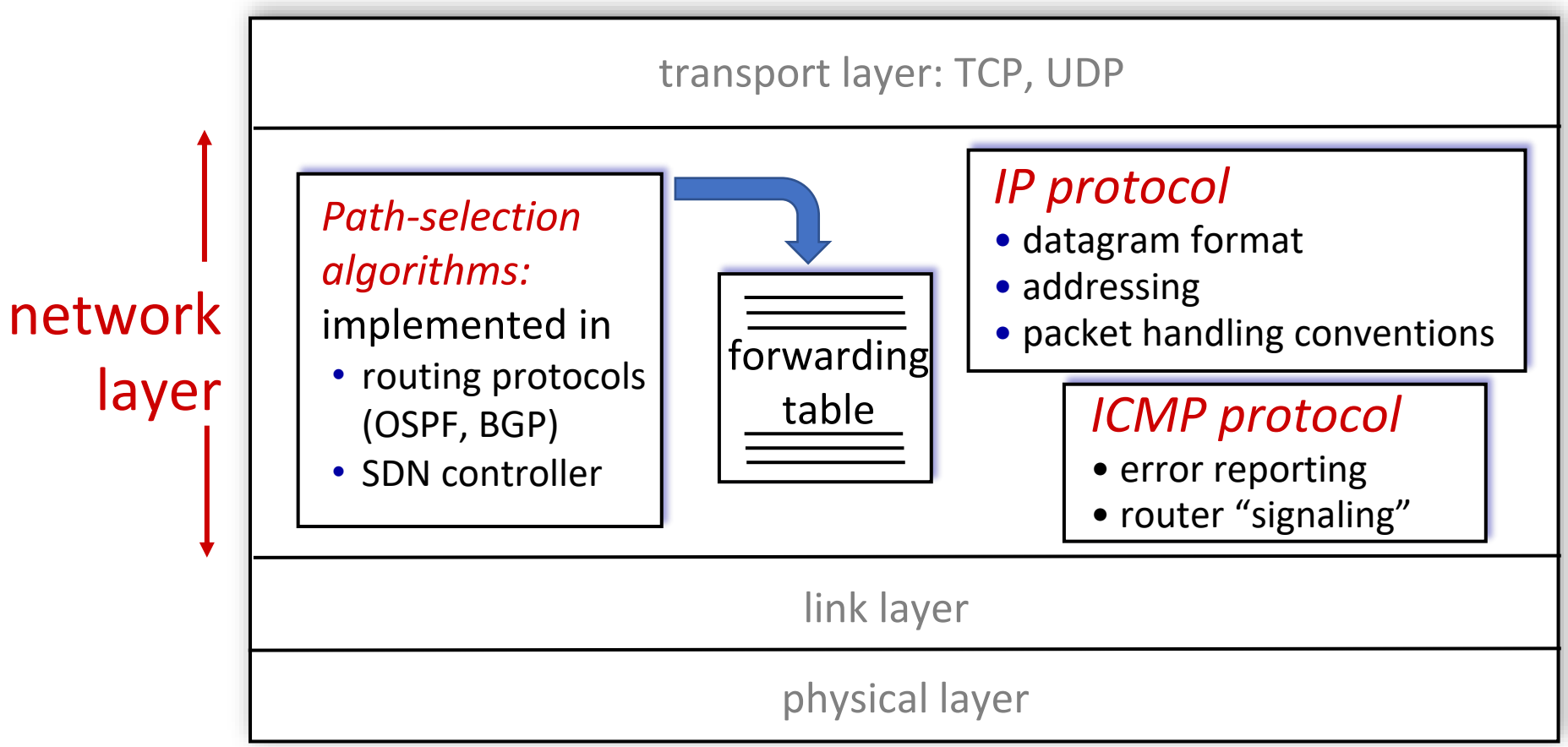


The address field in a **datagram** network is **end-to-end addressing**.

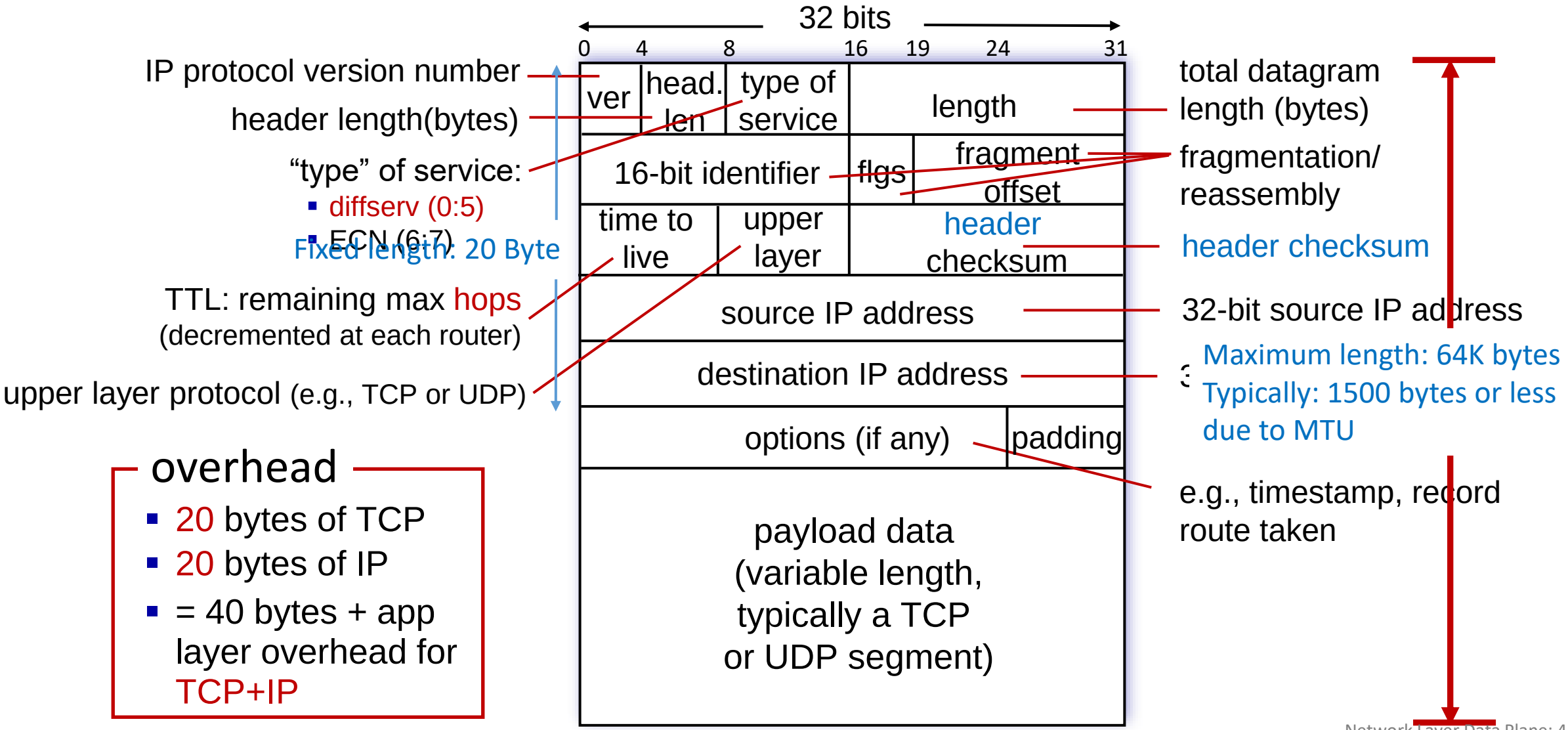
At each hop, the sender's and receiver's MAC are changed along the way.

Network Layer: Internet

host, router network layer functions:



IPv4 Datagram format

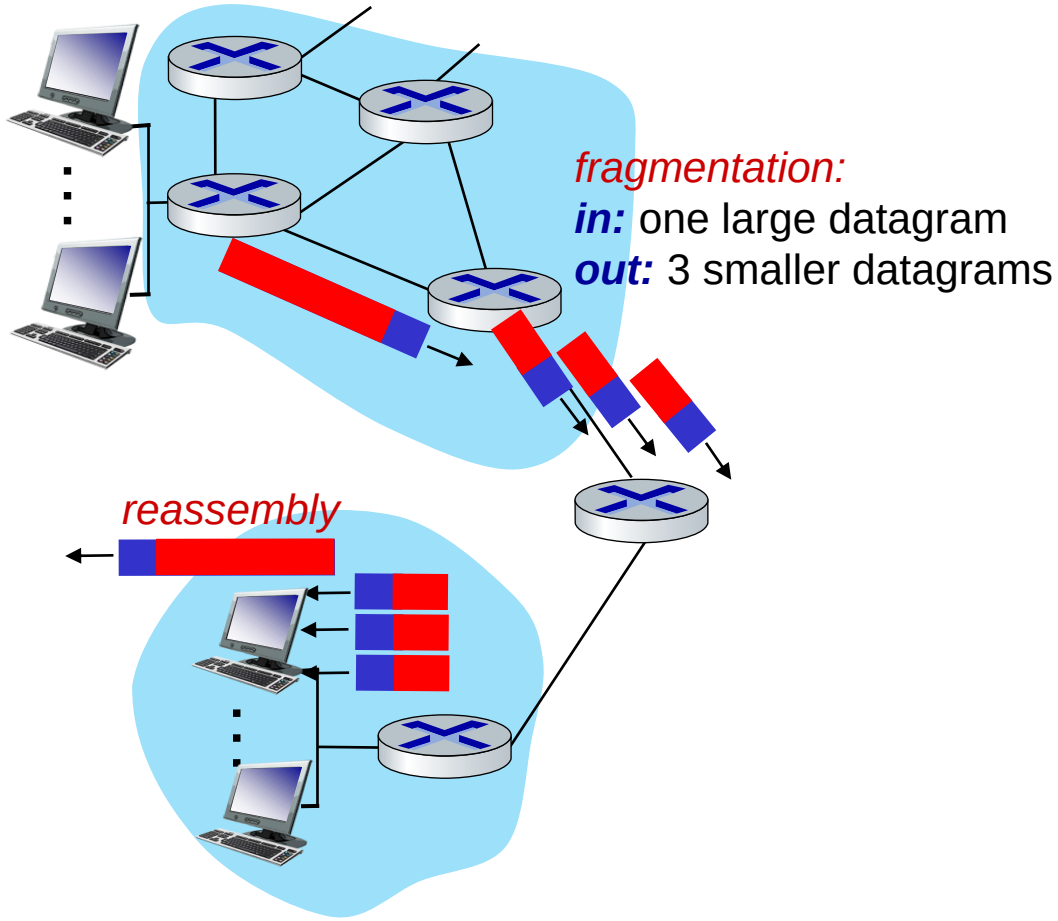


overhead

- 20 bytes of TCP
- 20 bytes of IP
- = 40 bytes + app layer overhead for TCP+IP

IP fragmentation/reassembly

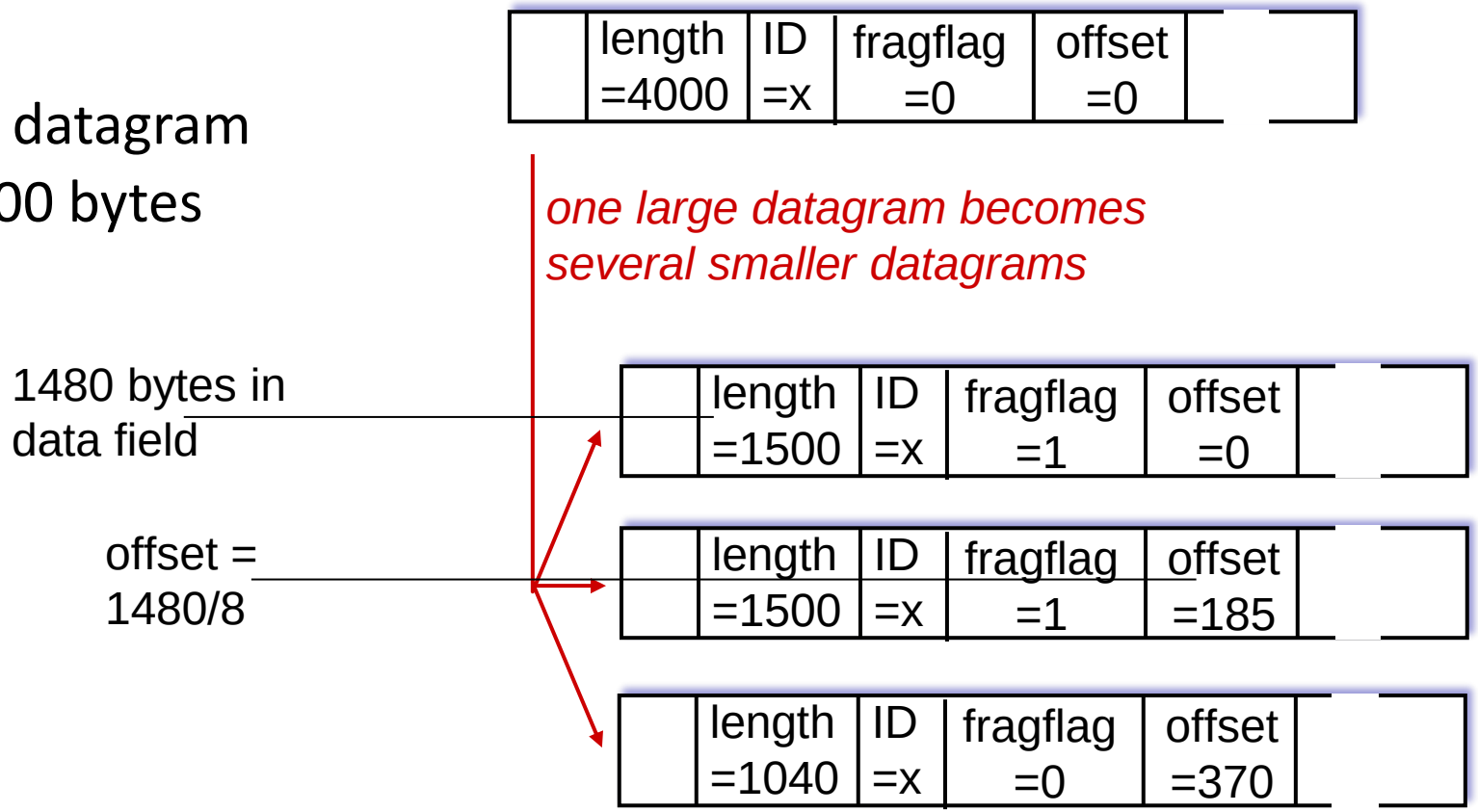
- network links have MTU (maximum transmission unit) - largest possible link-level frame
  - different link types, different MTUs
- large IP datagram divided ("fragmented") within net
  - one datagram becomes several datagrams
  - "reassembled" only at *destination*
  - IP header bits used to identify, order related fragments



IP fragmentation/reassembly

example:

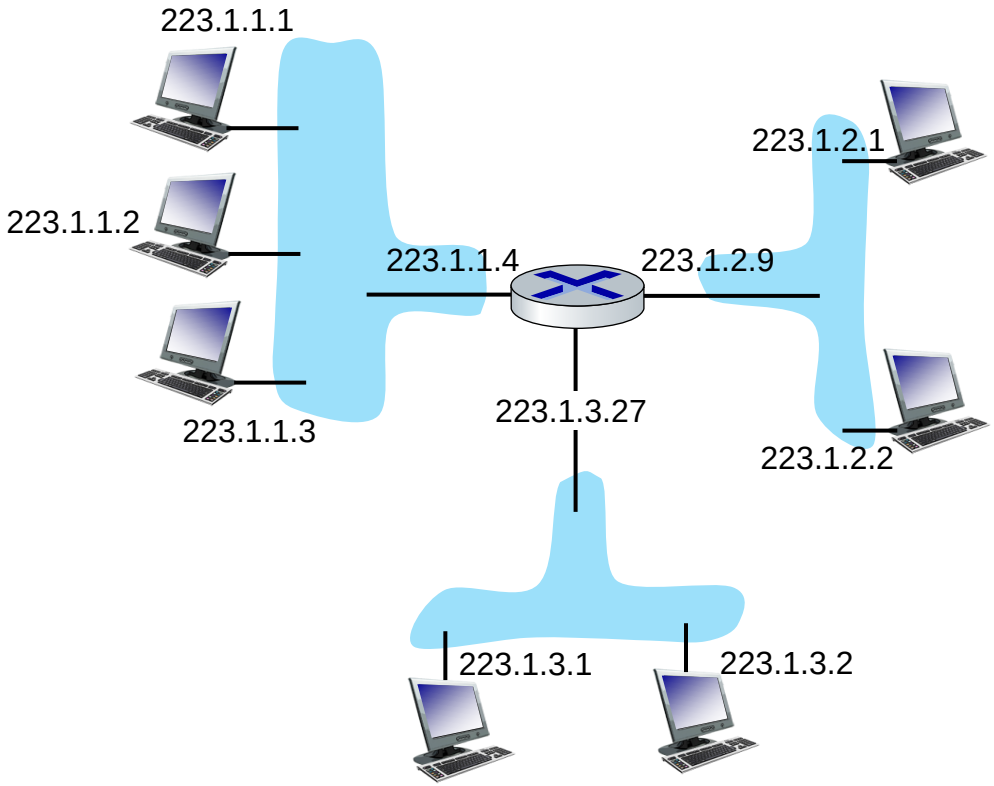
- 4000 byte datagram
- MTU = 1500 bytes



1480+1480+1020 = 3980 = 4000-20

IP addressing: introduction

- IP address: 32-bit identifier associated with each host or router *interface*
- interface: connection between host/router and physical link
  - router's typically have multiple interfaces
  - host typically has one or two interfaces (e.g., wired Ethernet, wireless 802.11)



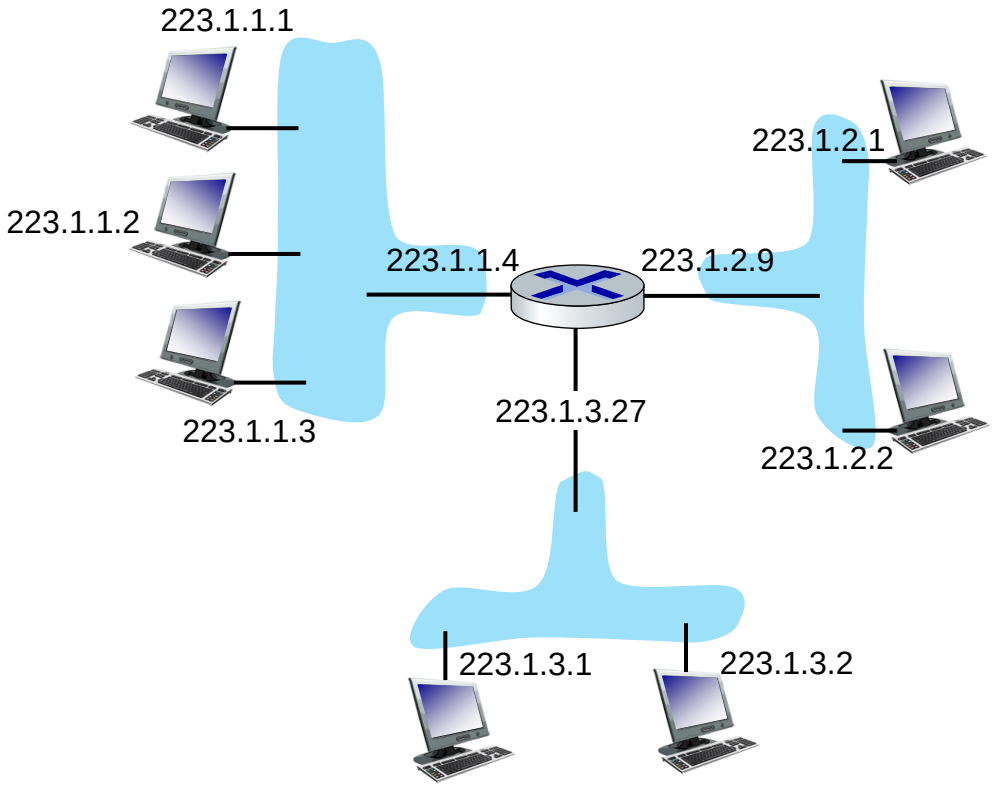
dotted-decimal IP address notation:

223.1.1.1 = 11011111 00000001 00000001 00000001

223 1 1 1

IP addressing: introduction

- IP address: 32-bit identifier associated with each host or router *interface*
- interface: connection between host/router and physical link
  - router's typically have multiple interfaces
  - host typically has one or two interfaces (e.g., wired Ethernet, wireless 802.11)



dotted-decimal IP address notation:

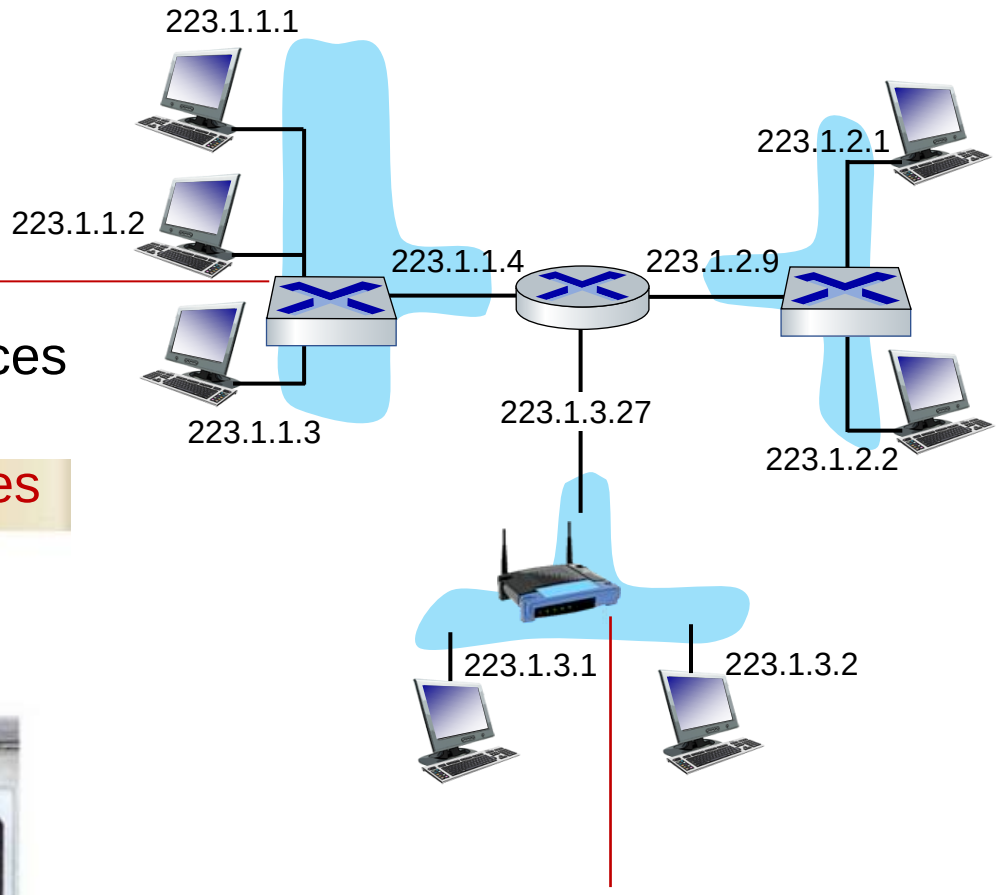
223.1.1.1 = 11011111 00000001 00000001 00000001

223 1 1 1

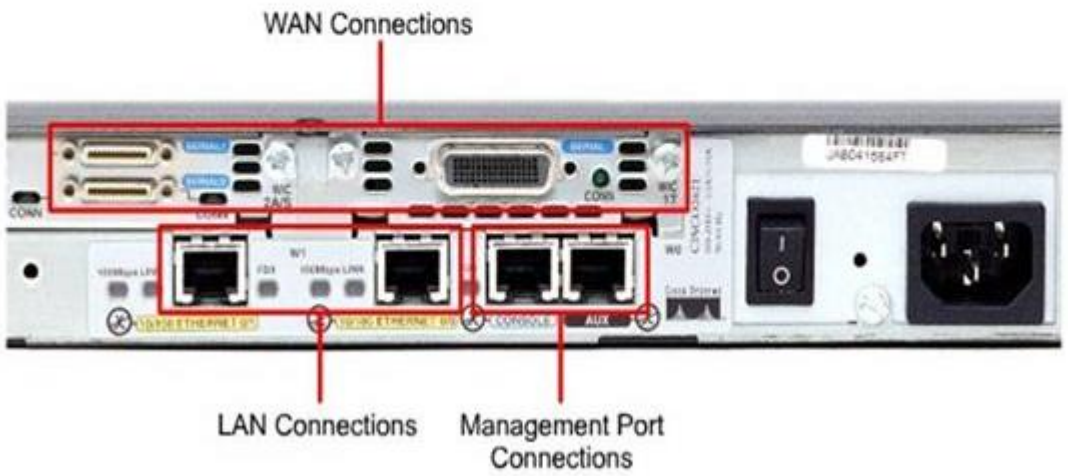
IP addressing: introduction

Q: how are interfaces actually connected?

A: wired Ethernet interfaces connected by Ethernet switches

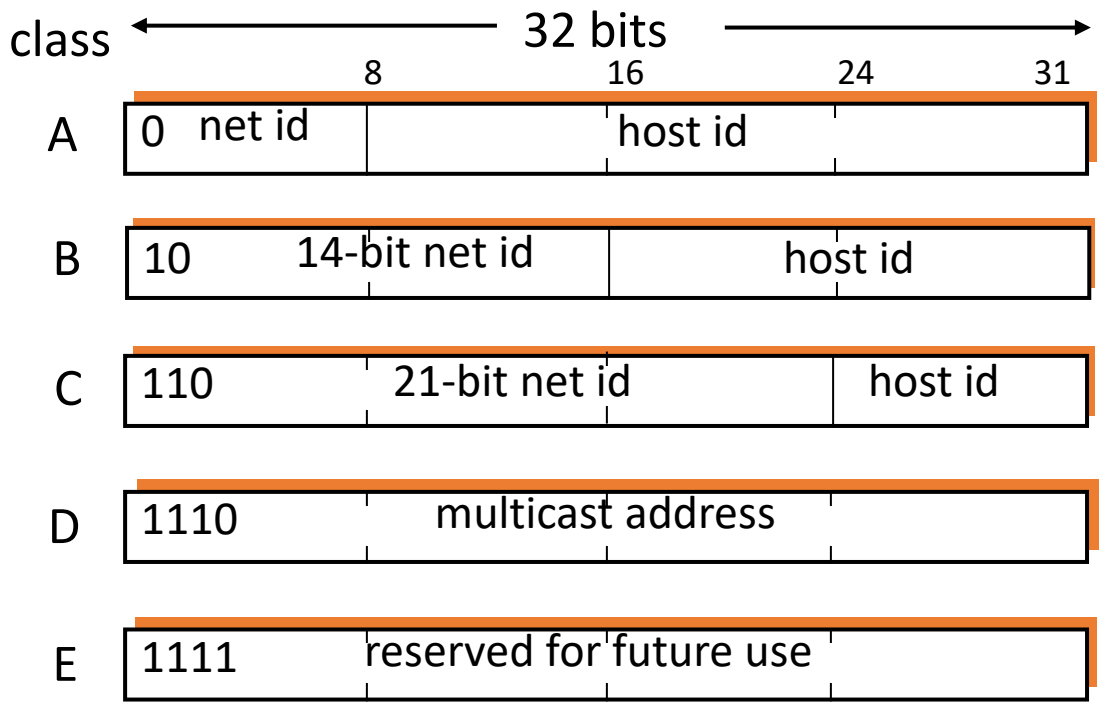


Router Interfaces



A: wireless WiFi interfaces connected by WiFi base station

Classful IP Addresses



Dotted Decimal Notation

1.0.0.0. to 127.0.0.0

128.0.0.0 to 191.255.0.0

192.0.0.0 to 223.255.255.0

224.0.0.0 to 239.255.255.255

240.0.0.0 to 255.255.255.254

The range of dotted decimal values that correspond to each IP address class.

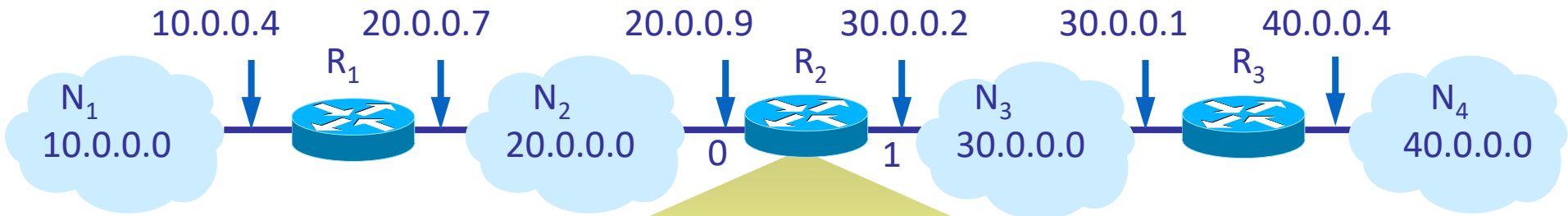
Some values are reserved for special purposes.

Special forms of IP address

net id	host id	valid source	valid destination	representation/comments
0	0	yes	no	Startup source address (this host)
0	host-id	yes	no	A host in this network
all 1s	all 1s	no	yes	Limited/ local network broadcast
net-id	all 1s	no	yes	Directed broadcast for the network net-id
net-id	all 0s	no	yes	The network itself/directed broadcast
127	Not all 0s or all 1s	yes	yes	Loopback test for the host (127 is not a network address)

Note: Routers along the path use ONLY the net-id portion of the address when deciding how to forward the datagram.

Direct and Indirect Delivery

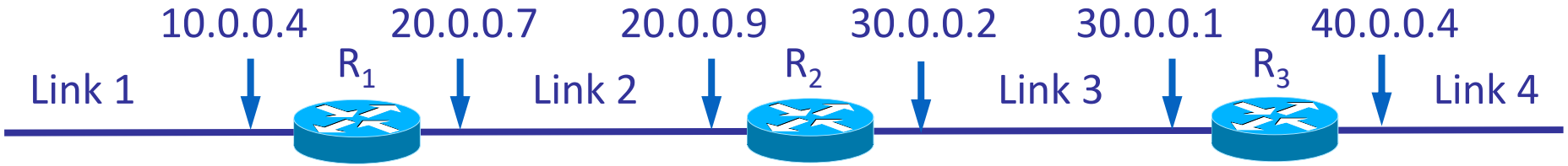


Routing table in R_2

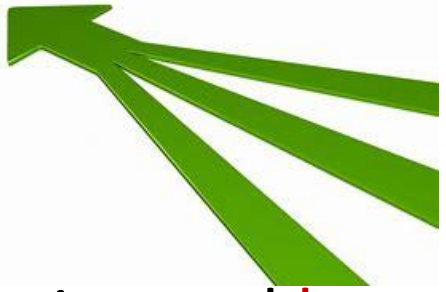
To reach hosts on network	Next hop
20.0.0.0	Direct delivery, interface 0
30.0.0.0	Direct delivery, interface 1
10.0.0.0	20.0.0.7
40.0.0.0	30.0.0.1

The final router deliver datagram directly

Q: Can the intermediate IP address be written into the datagram?



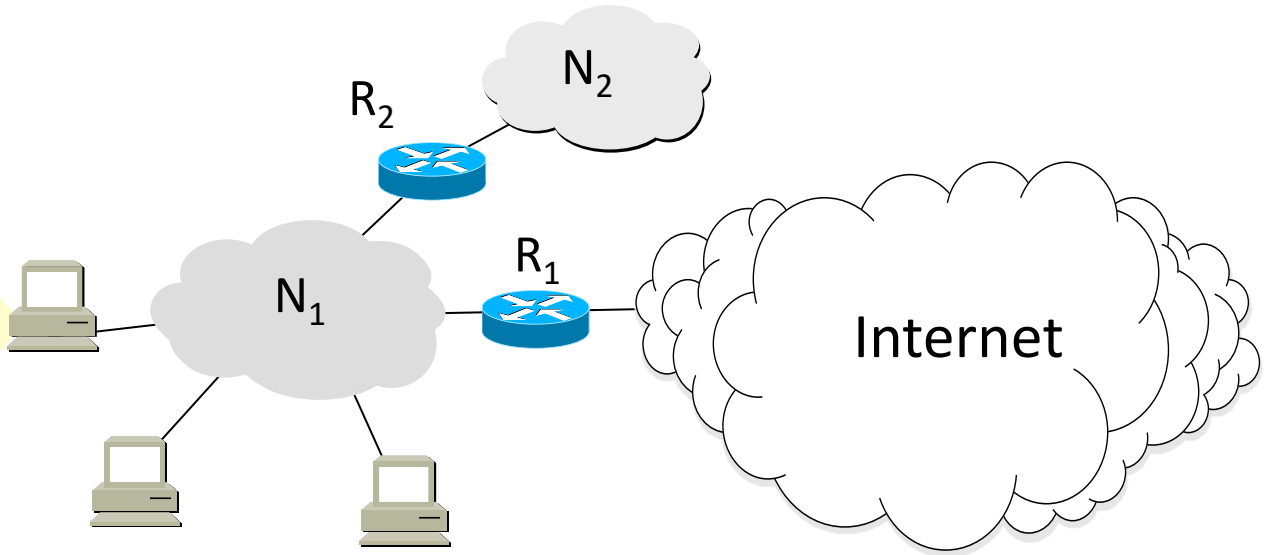
Default Routes



- Another technique, used to hide information and keep routing table sizes small, consolidates multiple entries into a default case.
- Default forwarding is especially useful when a site has a small set of local addresses and only one connection to the rest of the Internet.

Routing table in any host of network N_1

Destination Network	Next hop
N_1	Direct
N_2	R_2
Default	R_1



Original Classful IP Addressing scheme



A router can keep one routing entry per network instead of per destination host.

- to determine the boundary between prefix and suffix



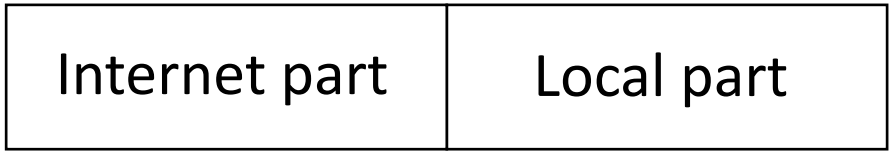
Weakness: Requiring a unique prefix for each physical network would **exhaust** the address space quickly as the Internet proliferates.

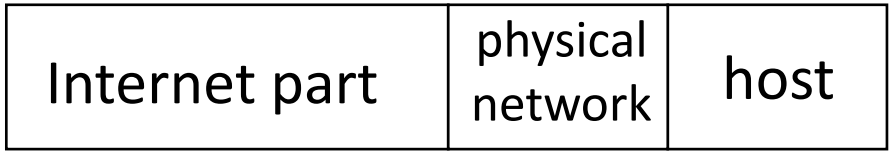
- In order to reduce the number of network prefixes used, **three survival solutions** are:
  1. unnumbered point-to-point links,
  2. proxy ARP
  3. subnet addressing.



Subnet Addressing / Subnetting

- Conceptually, adding subnets only changes the interpretation of IP addresses slightly.
- Routers only examine the network prefix. The interpretation of the local portion of an address is left up to the site.





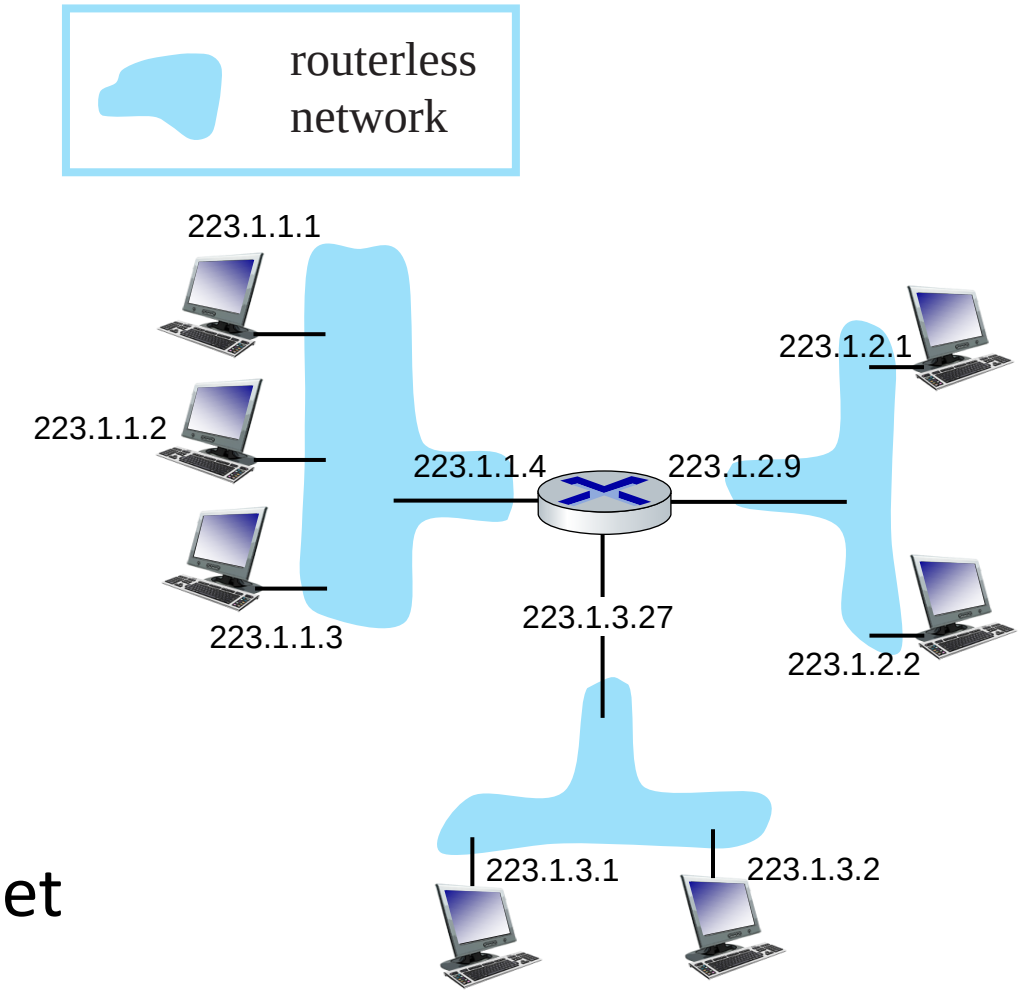
Subnet mask ("AND" operation):

Network Identifier Host Identifier

11111111 11111111 11111111 00000000 255.255.255.0

Subnets

- *What's a subnet ?*
  - device interfaces that can physically reach each other without passing through an intervening router
- IP addresses have structure:
  - subnet part: devices in same subnet have common high order bits
  - host part: remaining low order bits

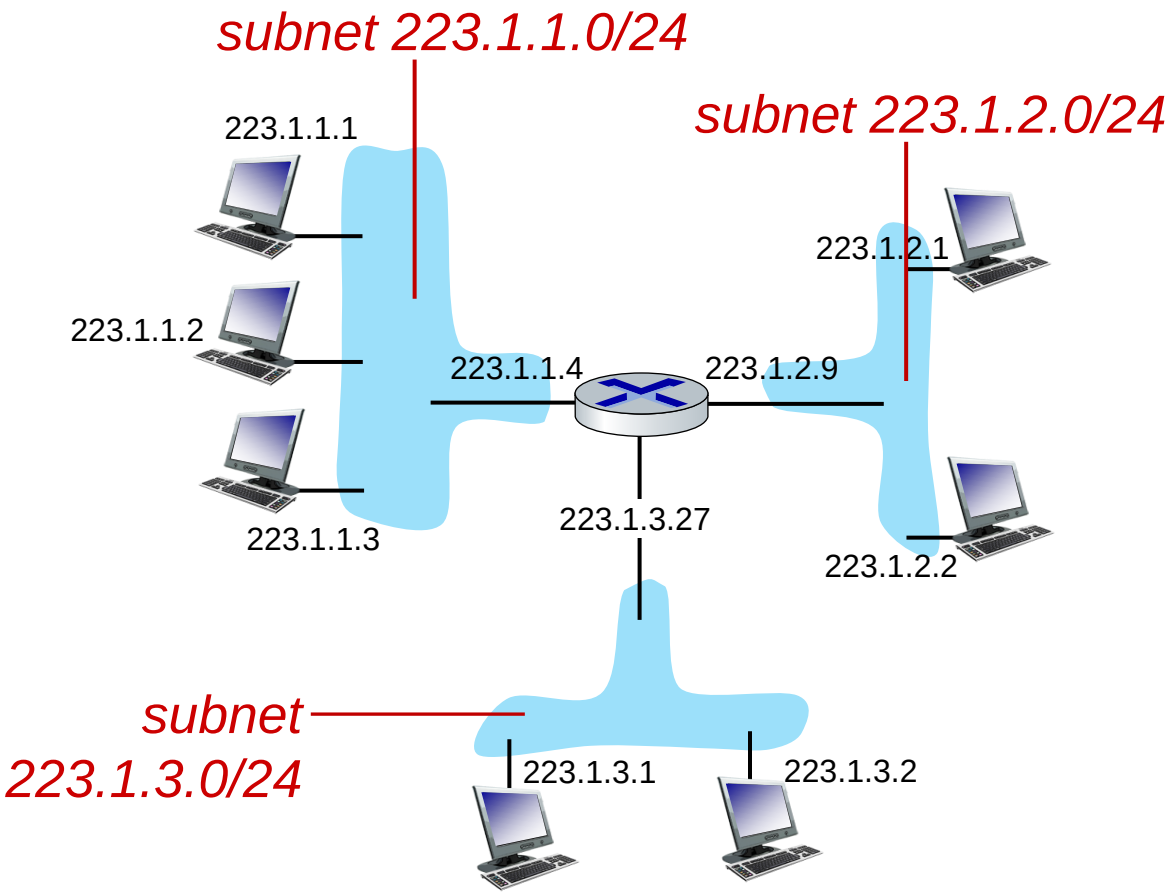


network consisting of 3 subnets

Subnets

Recipe for defining subnets:

- detach each interface from its host or router, creating "islands" of isolated networks
- each isolated network is called a *subnet*

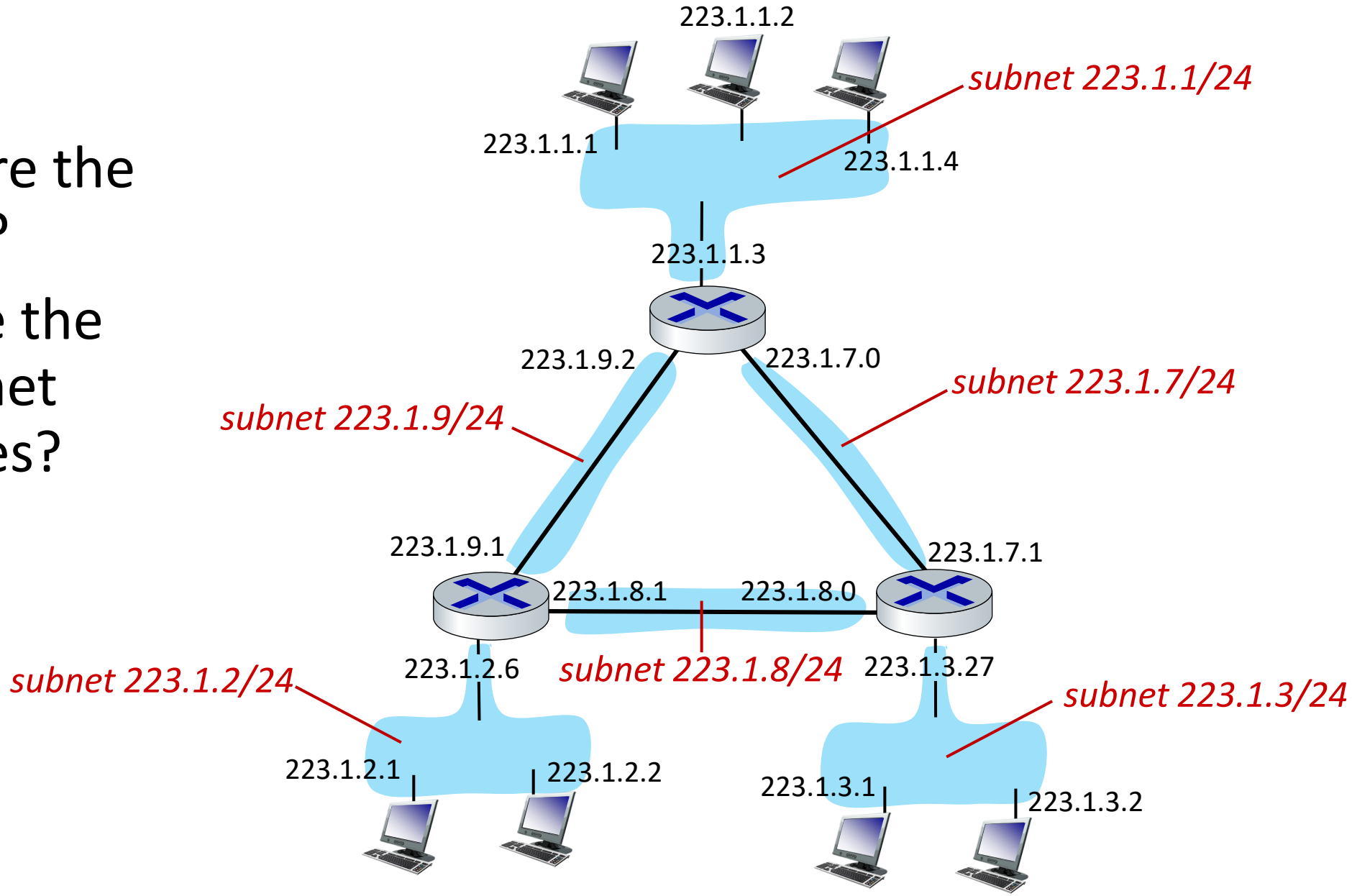


subnet mask: /24

(high-order 24 bits: subnet part of IP address)

Subnets

- where are the subnets?
- what are the /24 subnet addresses?

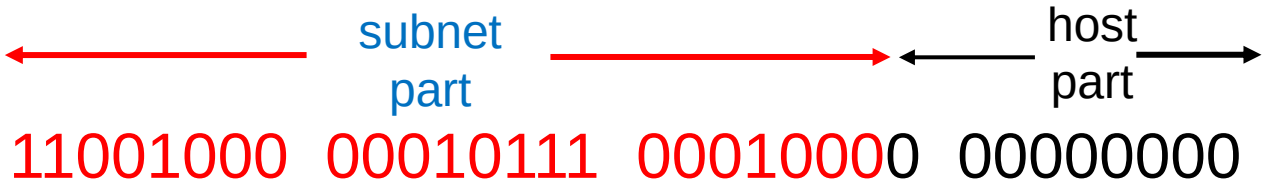


IP addressing: CIDR



CIDR: Classless InterDomain Routing (pronounced “cider”)

- CIDR generalizes the notion of subnet addressing
- subnet portion of address of arbitrary length
- address format: a.b.c.d/x, where x is # bits in subnet portion of address



200.23.16.0/23

- only these x leading prefix bits are considered by routers outside the organization’s network.

IP addresses: how to get one?

That's actually two questions:

1. Q: How does a *host* get IP address within its network (host part of address)?
2. Q: How does a *network* get IP address for itself (network part of address)

How does *host* get IP address?

1. hard-coded by sysadmin in config file (e.g., /etc/rc.config in UNIX)
2. DHCP: Dynamic Host Configuration Protocol: dynamically get address from a server
   - "plug-and-play" or zeroconf (zero-configuration) protocol

DHCP: Dynamic Host Configuration Protocol

GOAL: host *dynamically* obtains IP address from network server when it "joins" network

- can renew its lease on address in use
- allows reuse of addresses
  - only hold address while connected/on
- support for mobile users who join/leave network





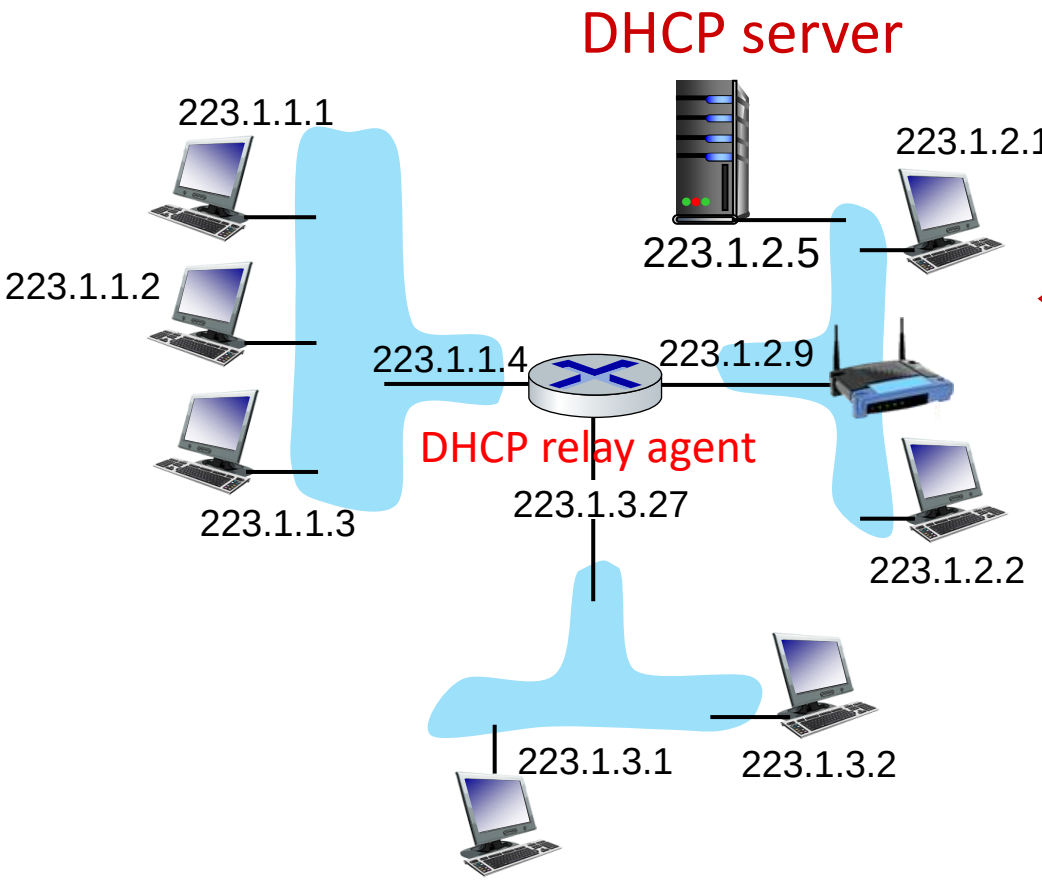
DHCP overview:

- host broadcasts DHCP discover msg [optional]
- DHCP server responds with DHCP offer msg [optional]
- host requests IP address: DHCP request msg
- DHCP server sends address: DHCP ack msg



DHCP client-server scenario

Thus DHCP is a client-server protocol



Typically, DHCP server will be co-located in router, serving all subnets to which router is attached



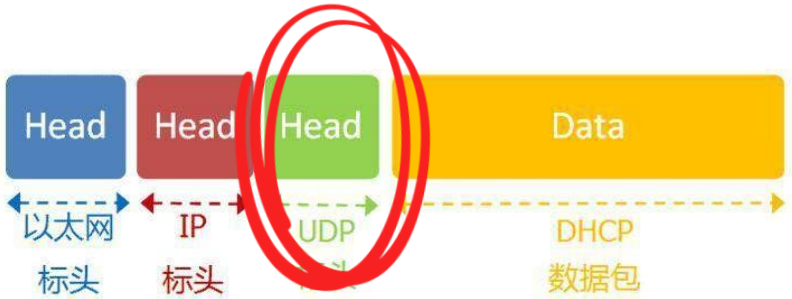
arriving DHCP client needs address in this network

- In the simplest case, each subnet will have a DHCP server.
- If no server is present on the subnet, a DHCP relay agent (typically a router) that knows the address of a DHCP server for that network is needed.

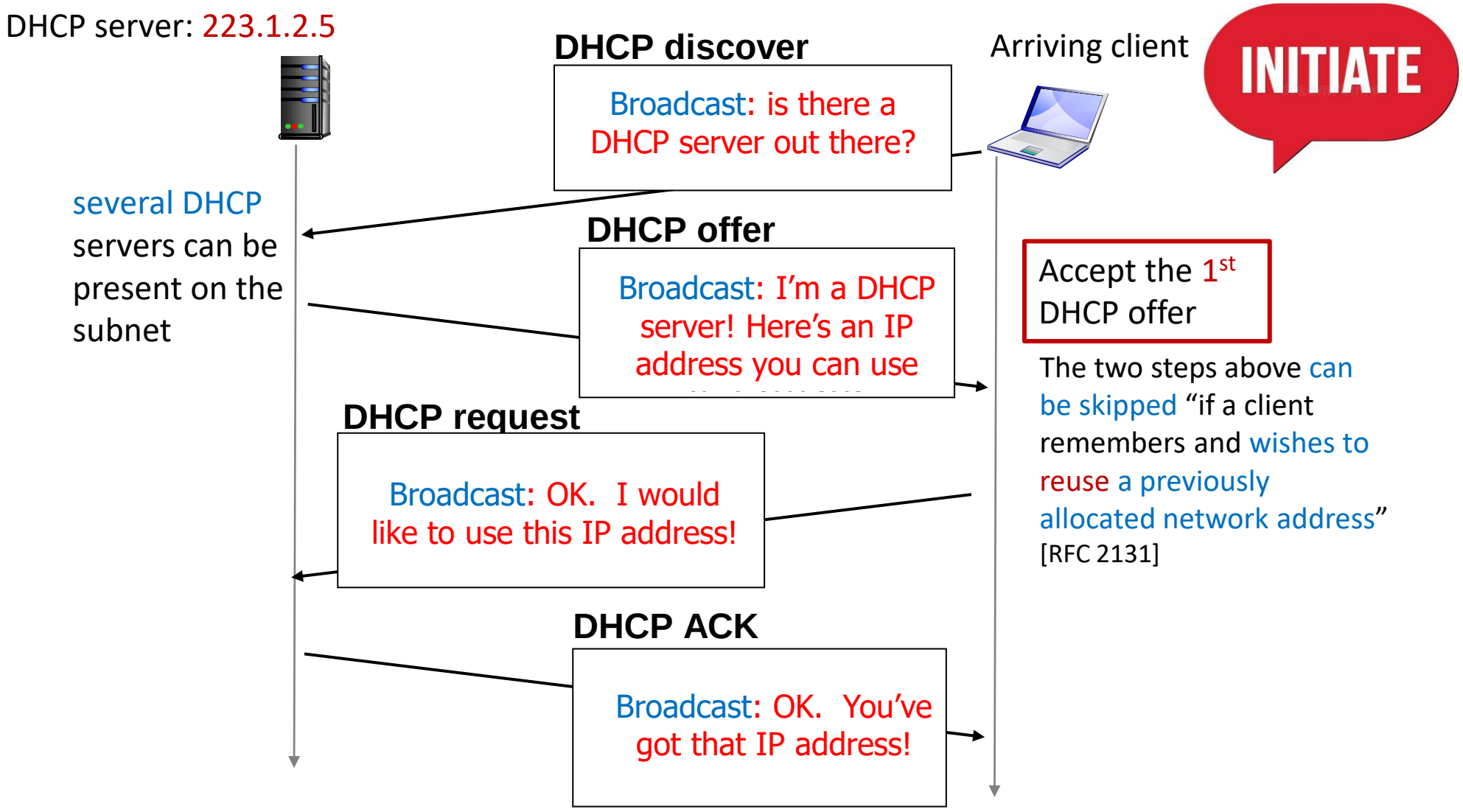
Dynamic Host Configuration Protocol (DHCP)					
Offsets	Octet	0	1	2	3
Octet	Bit	0–7	8–15	16–23	24–31
0	0	OpCode	Hardware Type	Hardware Length	Hops
4	32	Transaction ID			
8	64	Seconds Elapsed		Flags	
12	96	Client IP Address			
16	128	Your IP Address			
20	160	Server IP Address			
24	192	Gateway IP Address			
28	224	Client IP Address			
32	256	Client Hardware Address (16 bytes)			
36	288				
40	320				
44	352				
48+	384+	Server Host Name (64 bytes)			
		Boot File (128 bytes)			
		Options			

Figure 9-1: The DHCP packet structure

DHCP, an application layer protocol, is also enjoying widespread use in residential Internet access networks, enterprise networks, and in wireless LANs, where hosts join and leave the network frequently.



DHCP client-server scenario



DHCP: Wireshark output (home LAN)

request

Message type: **Boot Request (1)**
Hardware type: Ethernet
Hardware address length: 6
Hops: 0
Transaction ID: 0x6b3a11b7
Seconds elapsed: 0
Bootp flags: 0x0000 (Unicast)
Client IP address: 0.0.0.0 (0.0.0.0)
Your (client) IP address: 0.0.0.0 (0.0.0.0)
Next server IP address: 0.0.0.0 (0.0.0.0)
Relay agent IP address: 0.0.0.0 (0.0.0.0)
Client MAC address: Wistron_23:68:8a (00:16:d3:23:68:8a)
Server host name not given
Boot file name not given
Magic cookie: (OK)
Option: (t=53,l=1) **DHCP Message Type = DHCP Request**
Option: (61) Client identifier
    Length: 7; Value: 010016D323688A;
    Hardware type: Ethernet
    Client MAC address: Wistron_23:68:8a (00:16:d3:23:68:8a)
Option: (t=50,l=4) **Requested IP Address = 192.168.1.101**
Option: (t=12,l=5) **Host Name = "nomad"**
Option: (55) Parameter Request List
    Length: 11; Value: 010F03062C2E2F1F21F92B
    1 = Subnet Mask; 15 = Domain Name
    3 = Router; 6 = Domain Name Server
    44 = NetBIOS over TCP/IP Name Server
    ……

reply

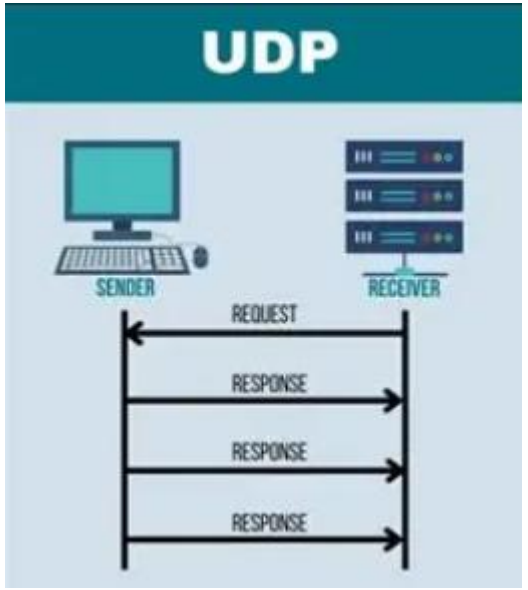
Message type: **Boot Reply (2)**
Hardware type: Ethernet
Hardware address length: 6
Hops: 0
Transaction ID: 0x6b3a11b7
Seconds elapsed: 0
Bootp flags: 0x0000 (Unicast)
Client IP address: 192.168.1.101 (192.168.1.101)
Your (client) IP address: 0.0.0.0 (0.0.0.0)
Next server IP address: 192.168.1.1 (192.168.1.1)
Relay agent IP address: 0.0.0.0 (0.0.0.0)
Client MAC address: Wistron_23:68:8a (00:16:d3:23:68:8a)
Server host name not given
Boot file name not given
Magic cookie: (OK)
Option: (t=53,l=1) DHCP Message Type = DHCP ACK
Option: (t=54,l=4) Server Identifier = 192.168.1.1
Option: (t=1,l=4) Subnet Mask = 255.255.255.0
Option: (t=3,l=4) Router = 192.168.1.1
Option: (6) Domain Name Server
    Length: 12; Value: 445747E2445749F244574092;
    IP Address: 68.87.71.226;
    IP Address: 68.87.73.242;
    IP Address: 68.87.64.146
Option: (t=15,l=20) Domain Name = "hsd1.ma.comcast.net."

DHCP: more than IP addresses

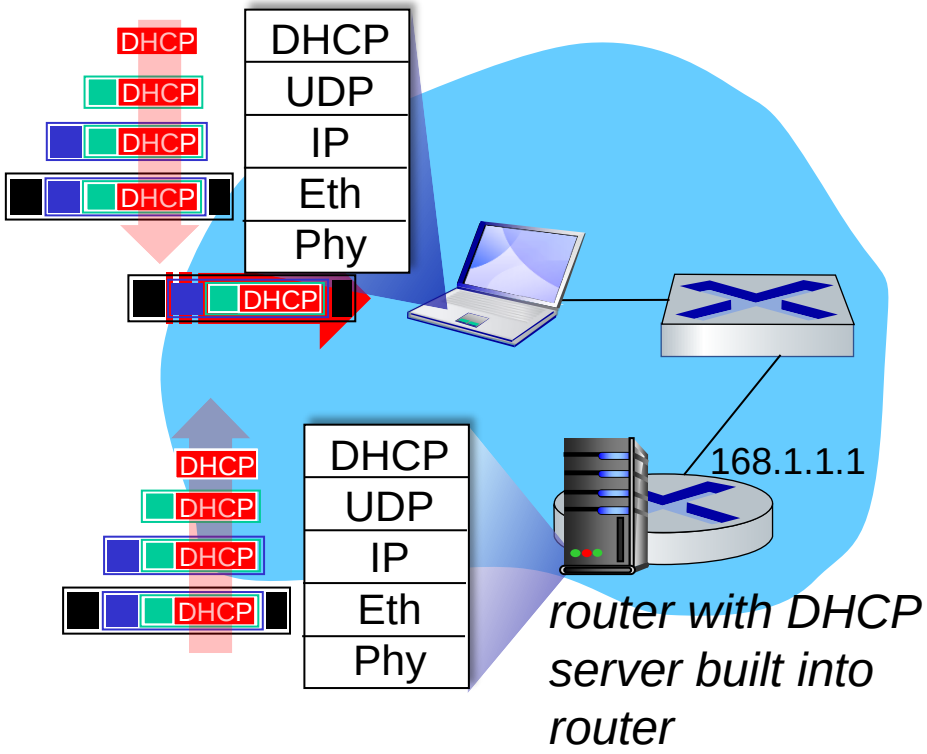
DHCP can return

1. allocated IP address (with the lease time) on subnet
2. IP address of first-hop router (the default gateway) for client
3. name and IP address of its local DNS sever
4. network mask (indicating network versus host portion of address)

- DHCP is an application-layer C/S protocol for allowing a device to automatically obtain an IP address.
  - DHCP relies on UDP as its transport layer protocol
  - It is common for the server to set the lease time to several hours or days



DHCP: example

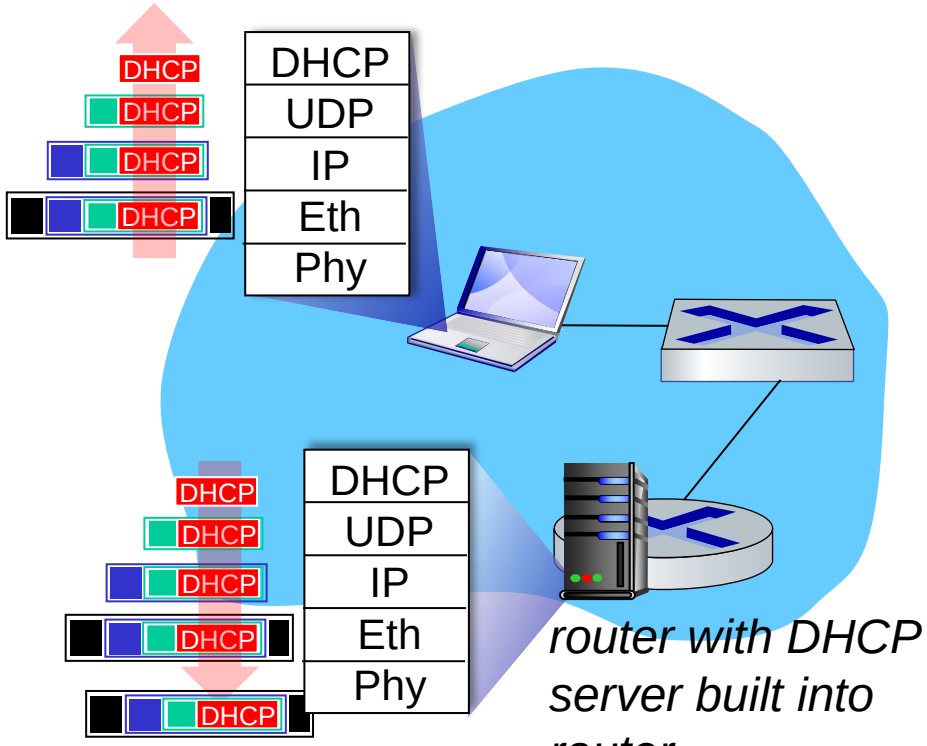


- Connecting laptop will use DHCP to get① IP address, ②address of first-hop router, ③address of DNS server.
- DHCP REQUEST message encapsulated in UDP, encapsulated in IP, encapsulated in Ethernet
- Ethernet frame broadcast

(dest: FF-FF-FF-FF-FF-FF) on LAN, received at router running DHCP server

- Ethernet demux'ed to IP demux'ed, UDP demux'ed to DHCP

DHCP: example



- DHCP server formulates DHCP ACK containing client's IP address, IP address of first-hop router for client, name & IP address of DNS server
- encapsulated DHCP server reply forwarded to client, demuxing up to DHCP at client
- client now knows its IP address, name and IP address of DNS server, IP address of its first-hop router

IP addresses: how to get one?

Q: how does *network* get subnet part of IP address?

A: gets allocated portion of its provider ISP's address space

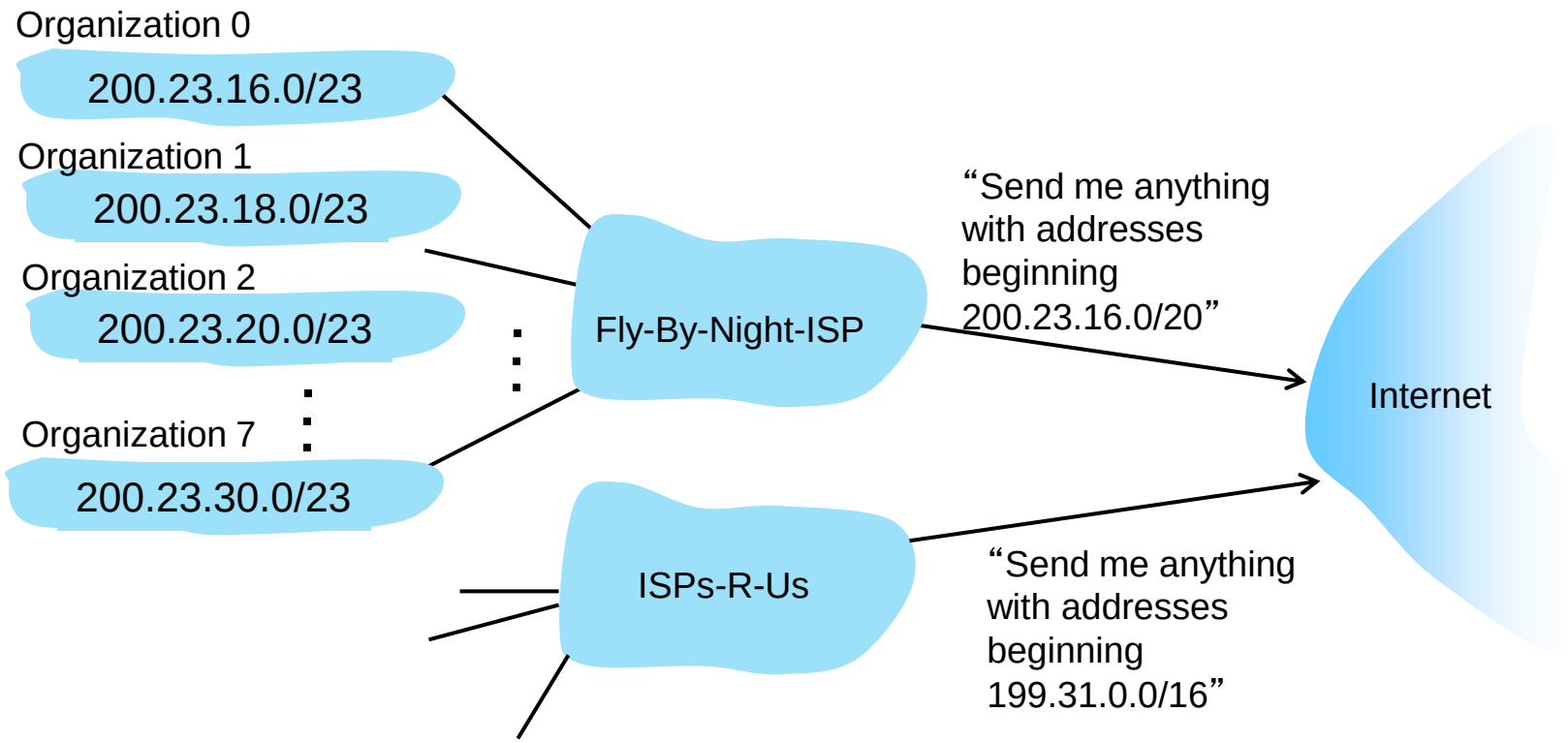
ISP's block	11001000 00010111 00010000	00000000	200.23.16.0/20

ISP can then allocate out its address space in 8 blocks:

Organization 0	11001000 00010111 00010000	00000000	200.23.16.0/23
Organization 1	11001000 00010111 00010010	00000000	200.23.18.0/23
Organization 2	11001000 00010111 00010100	00000000	200.23.20.0/23
…	……	….	….
Organization 7	11001000 00010111 00011110	00000000	200.23.30.0/23

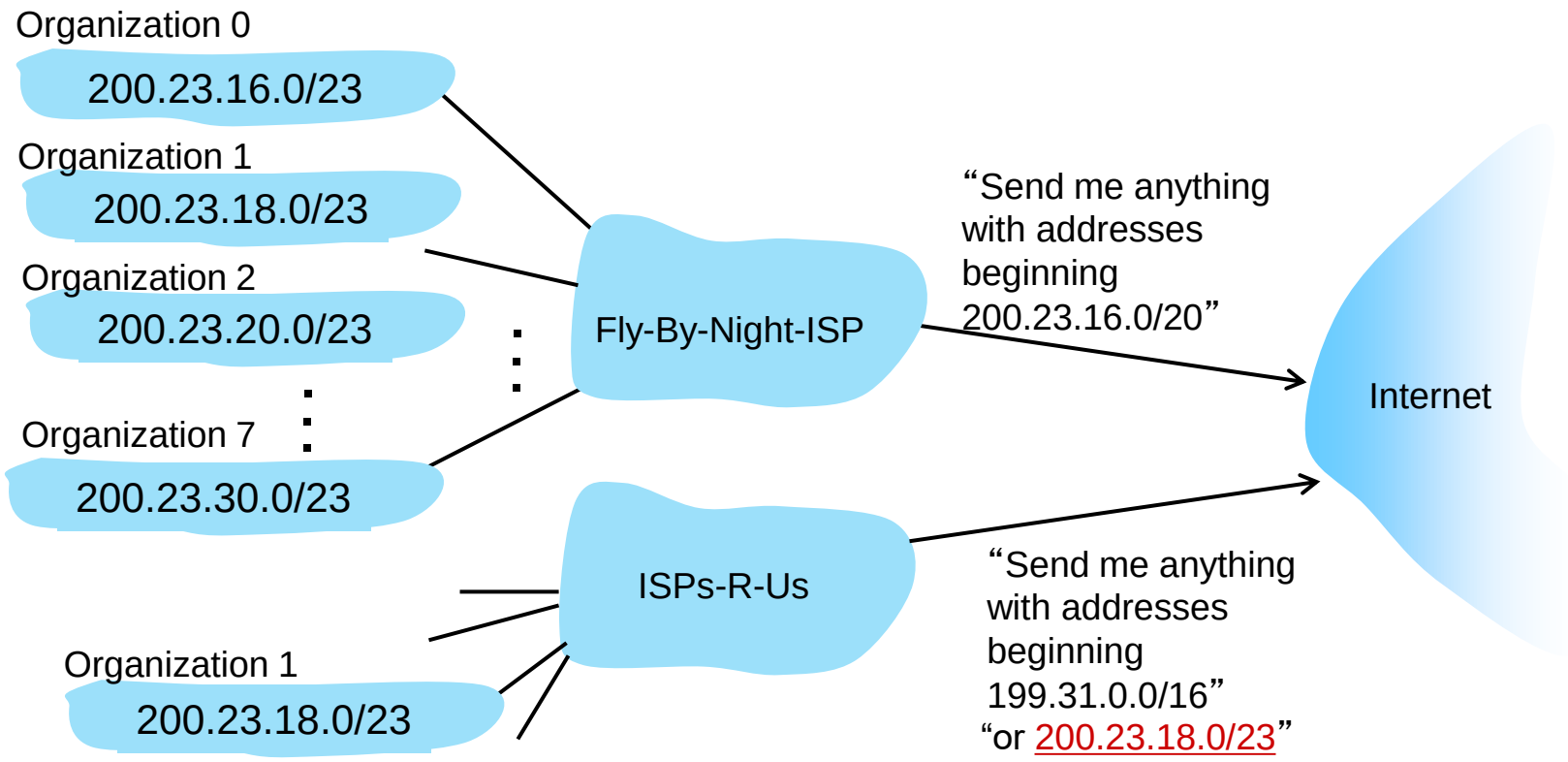
Hierarchical addressing: route aggregation

hierarchical addressing allows efficient advertisement of routing information:



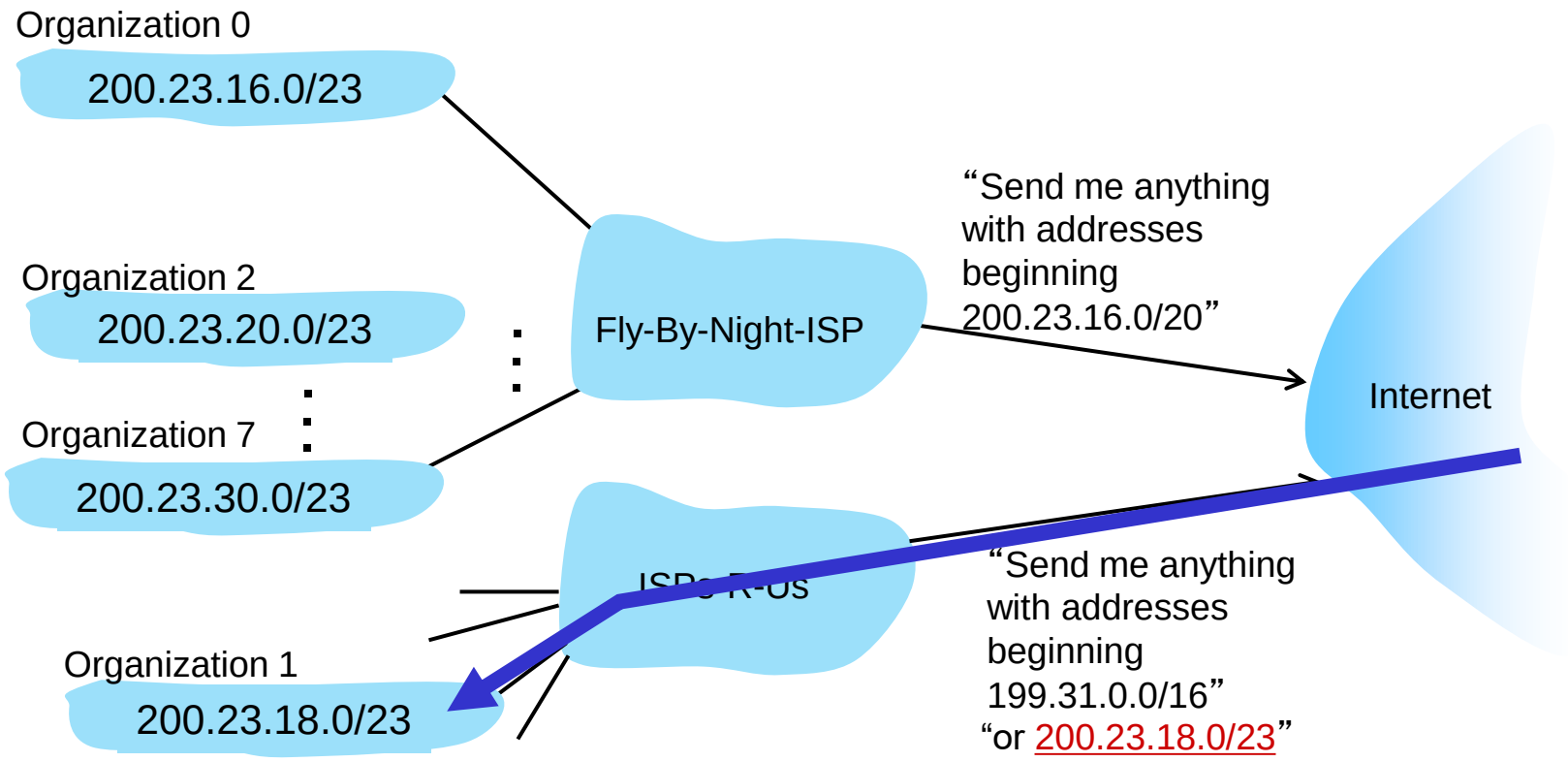
Hierarchical addressing: more specific routes

- Organization 1 moves from Fly-By-Night-ISP to ISPs-R-Us
- ISPs-R-Us now advertises a more specific route to Organization 1



Hierarchical addressing: more specific routes

- Organization 1 moves from Fly-By-Night-ISP to ISPs-R-Us
- ISPs-R-Us now advertises a more specific route to Organization 1



One Aggregation Example

- What is the network address aggregating 16 successive network addresses from 194.24.16.0/24 to 194.24.31.0/24?
- sol) If we compute the common prefix belonging to all the 16 network addresses, the aggregate address is 194.24.16.0/20

11000010 00011000 00010000	00000000	194.24.16.0/24
11000010 00011000 00011111	00000000	194.24.31.0/24
11000010 00011000 0001 0000	00000000	194.24.16.0/20

4 bits, 2^4=16 successive network addresses

IP addressing: last words ...

Q: How does an ISP get block of addresses?

A: ICANN: Internet Corporation for Assigned Names and Numbers http://www.icann.org/

- allocates IP addresses, through 5 regional registries (RRs) (who may then allocate to local registries)
- manages DNS root zone, including delegation of individual TLD (Top-Level Domains)(i.e., .com, .edu , …) management

Q: Are there enough 32-bit IP addresses?

- ICANN allocated last chunk of IPv4 addresses to RRs in 2011
- NAT (next) helps IPv4 address space exhaustion
- IPv6 has 128-bit address space

"Who the hell knew how much address space we needed?" Vint Cerf (reflecting on decision to make IPv4 address 32 bits long)

Network layer: "data plane" roadmap

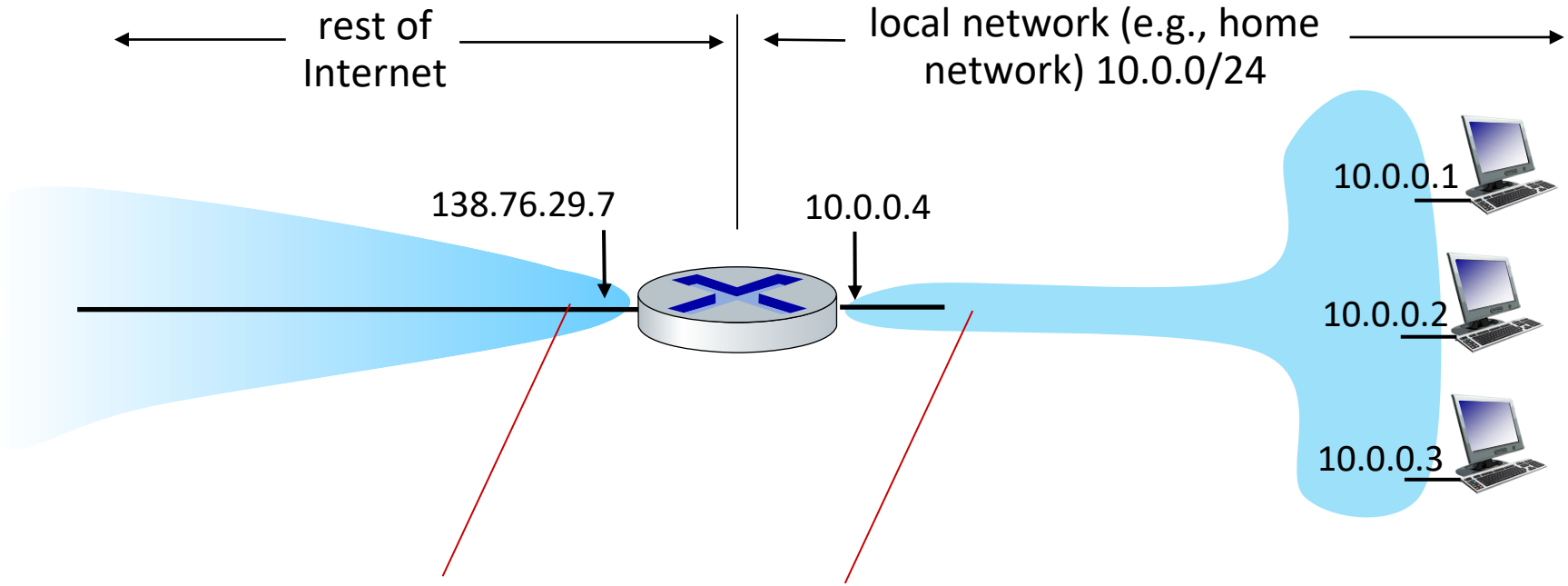
- Network layer: overview
  - data plane
  - control plane
- What's inside a router
  - input/output ports, switching
  - buffer management
  - scheduling
- IP: the Internet Protocol
  - datagram format, addressing
  - network address translation (NAT)
  - IPv6



- Generalized Forwarding
  - Match+action
  - OpenFlow
  - Middleboxes
- Internet architecture

NAT: network address translation

NAT: all devices in local network share just one IPv4 address as far as outside world is concerned



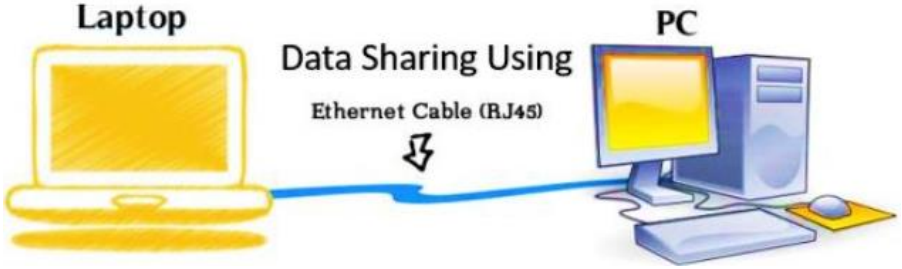
all datagrams *leaving* local network have *same* source NAT IP address: 138.76.29.7, but *different* source port numbers

datagrams with source or destination in this network have 10.0.0/24 address for source, destination (as usual)

Private Addresses /Non-routable Addresses

- CIDR blocks reserved for use within a site (private network)
- Must never appear on the Internet
- ISPs do not maintain routes

Automatic IP Assignment: If a device **doesn't get an IP from DHCP**, it automatically assigns itself a random address in the **169.254.0.0/16** range.



Link local address: 169.254.0.0/16
(169.254.0.1 - 169.254.255.254) or fe80::/10

Prefix	Lowest Address	Highest Address
10.0.0.0/8	10.0.0.0	10.255.255.255
172.16.0.0/12	172.16.0.0	172.31.255.255
192.168.0.0/16	192.168.0.0	192.168.255.255
169.254.0.0/16	169.254.0.0	169.254.255.255

NAT: network address translation

- All devices in local network have 32-bit addresses in a “private” IP address space (10/8, 172.16/12, 192.168/16 prefixes) that can only be used in local network

- **advantages:**
  - just one IP address needed from provider ISP for *all* devices
  - can change addresses of host in local network without notifying outside world
  - can change ISP without changing addresses of devices in local network
  - security: devices inside local net not directly addressable, visible by outside world

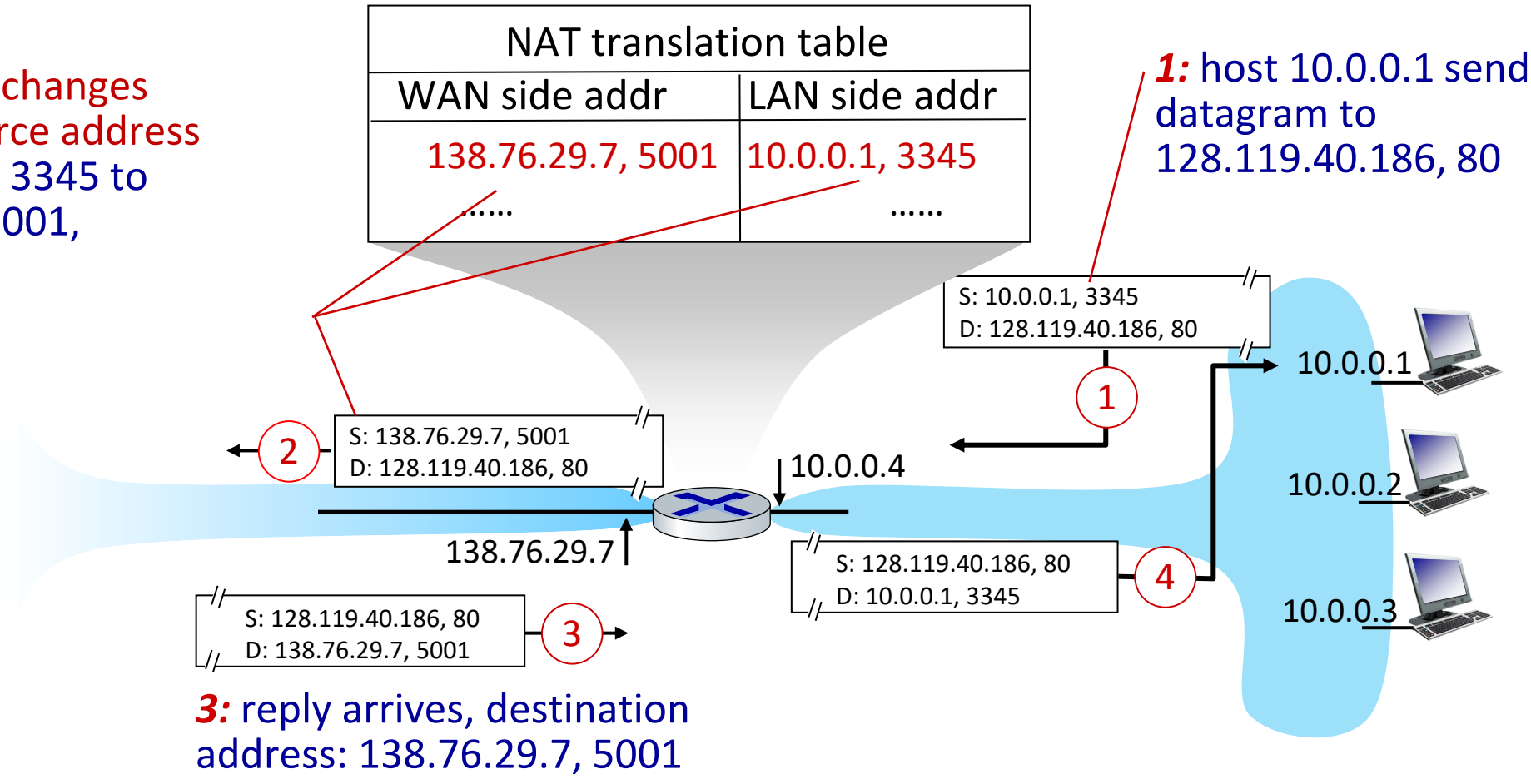
NAT: network address translation

implementation: NAT router must (transparently):

- outgoing datagrams: replace (source IP address, port #) of every outgoing datagram to (NAT IP address, new port #)
  - remote clients/servers will respond using (NAT IP address, new port #) as destination address
- remember (in NAT translation table) every (source IP address, port #) to (NAT IP address, new port #) translation pair
- incoming datagrams: replace (NAT IP address, new port #) in destination fields of every incoming datagram with corresponding (source IP address, port #) stored in NAT table

NAT: network address translation

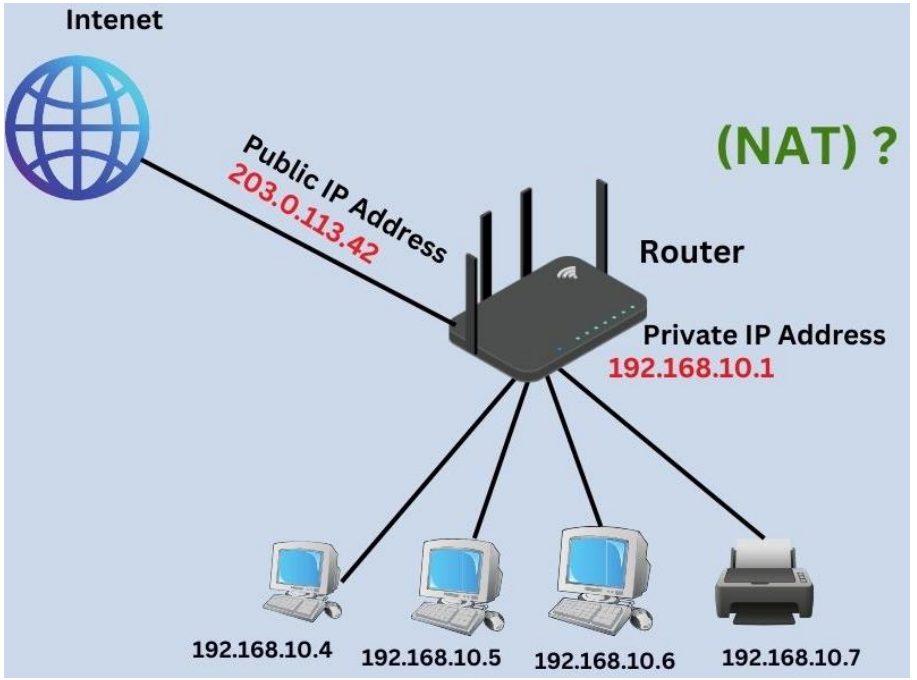
2: NAT router changes datagram source address from 10.0.0.1, 3345 to 138.76.29.7, 5001, updates table



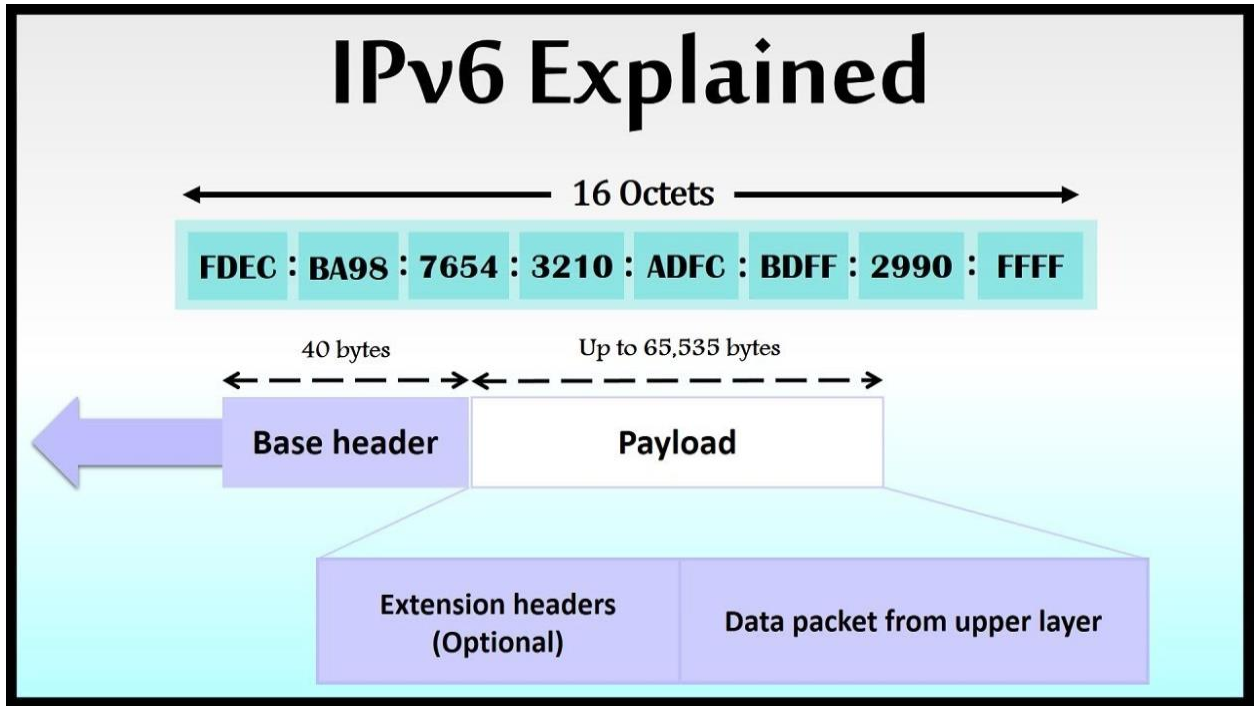
3: reply arrives, destination address: 138.76.29.7, 5001

NAT: network address translation

- NAT has been controversial:
  - routers "should" only process up to layer 3
  - address "shortage" should be solved by IPv6
  - violates end-to-end argument (port # manipulation by network-layer device)
  - NAT traversal: what if client wants to connect to server behind NAT?
- but NAT is here to stay:
  - extensively used in home and institutional nets, 4G/5G cellular nets



IPv6: motivation

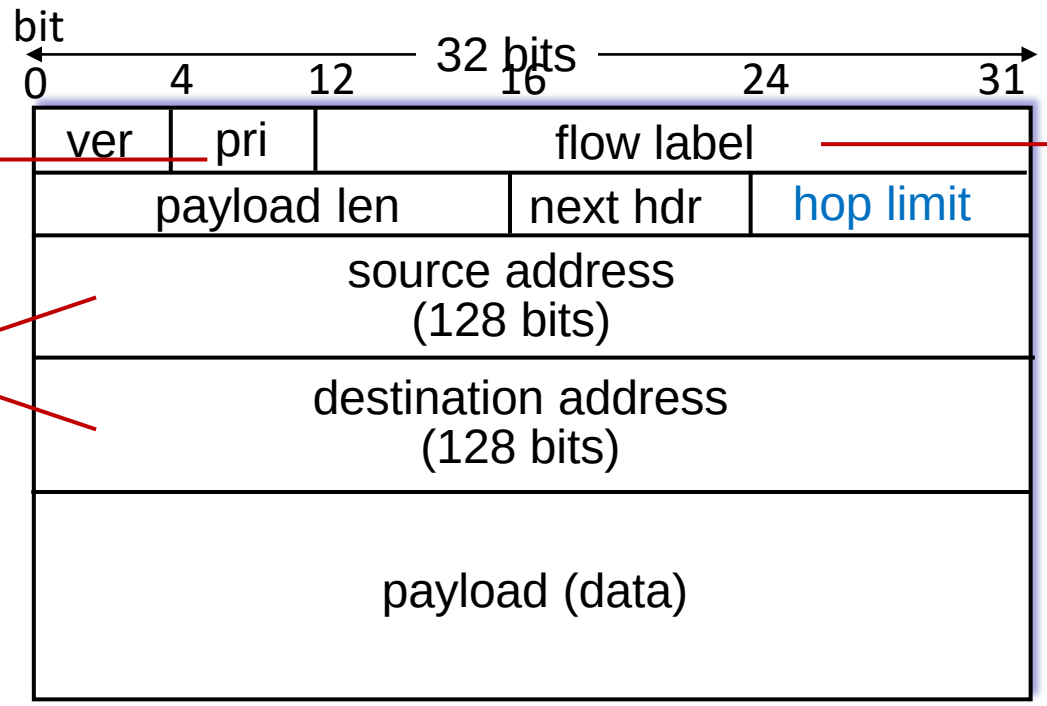


- initial motivation: 32-bit IPv4 address space would be completely allocated
- additional motivation:
  - speed processing/forwarding: 40-byte fixed length header
  - enable different network-layer treatment of "flows"

IPv6 datagram format

priority: identify priority among datagrams in flow

128-bit IPv6 addresses

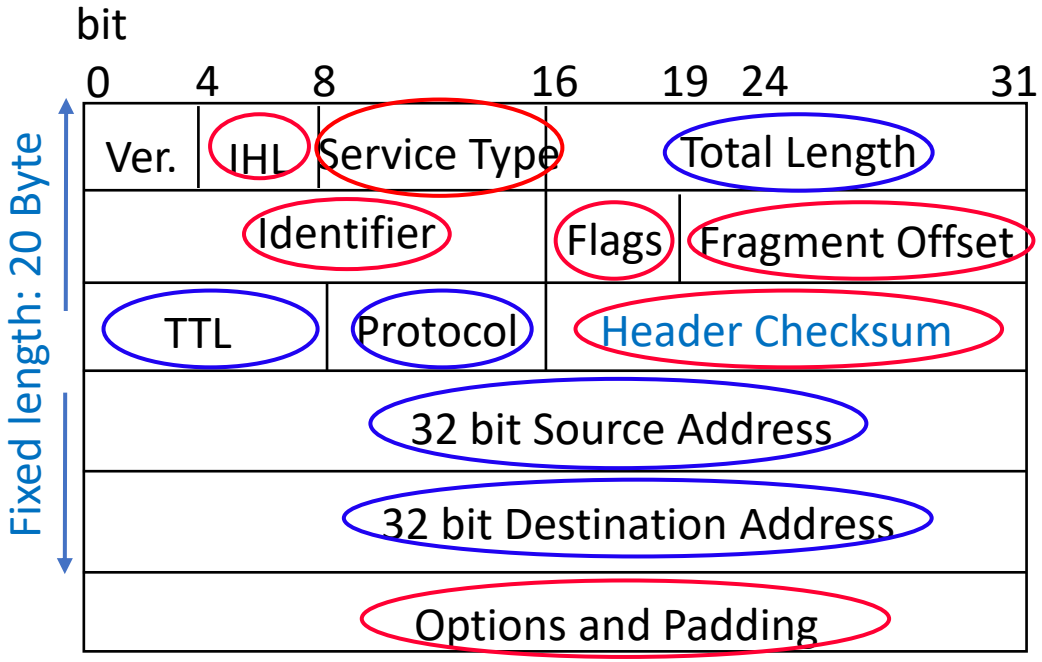


flow label: identify datagrams in same "flow." (concept of "flow" not well defined).

What's missing (compared with IPv4):

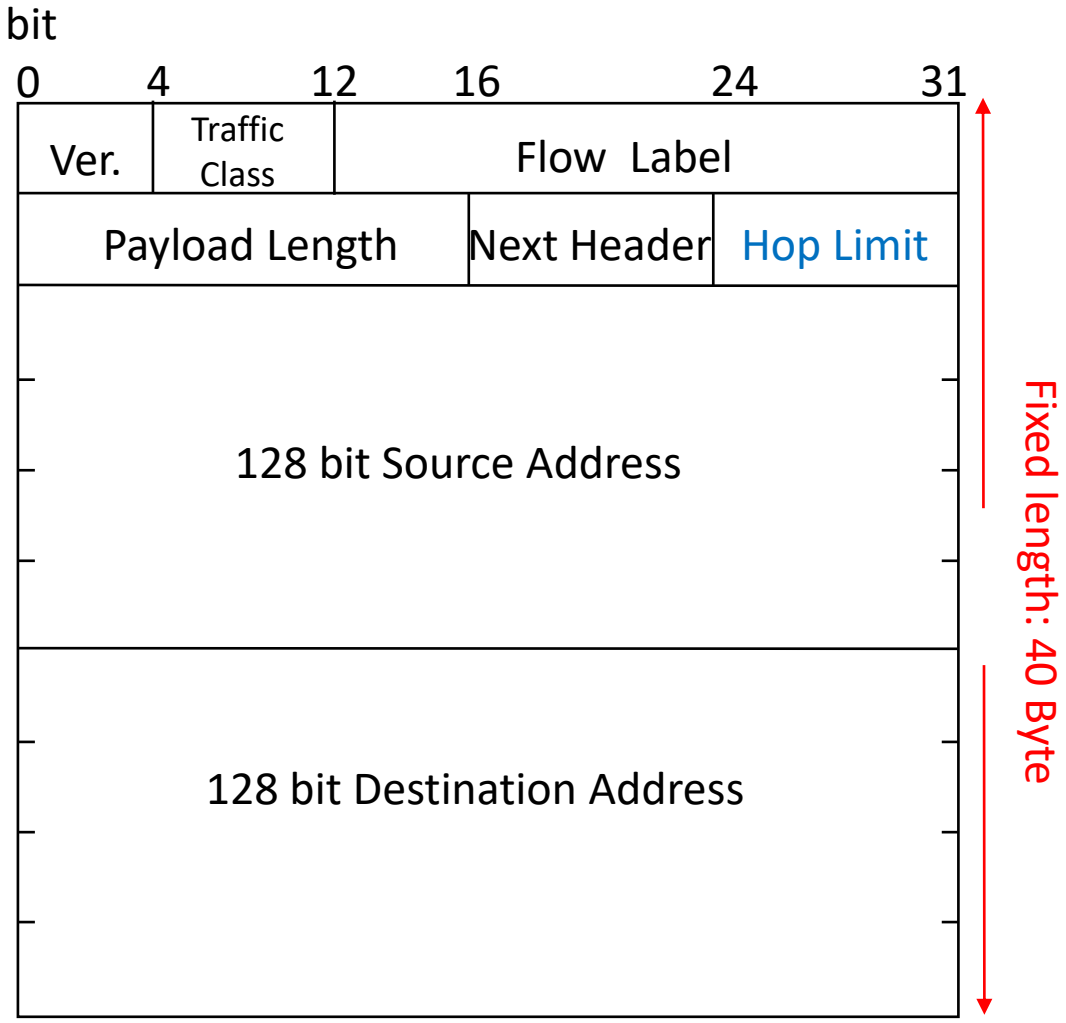
- no checksum (to speed processing at routers)
- no fragmentation/reassembly
- no options (available as upper-layer, next-header protocol at router)

IPv6 Header – Comparison with IPv4



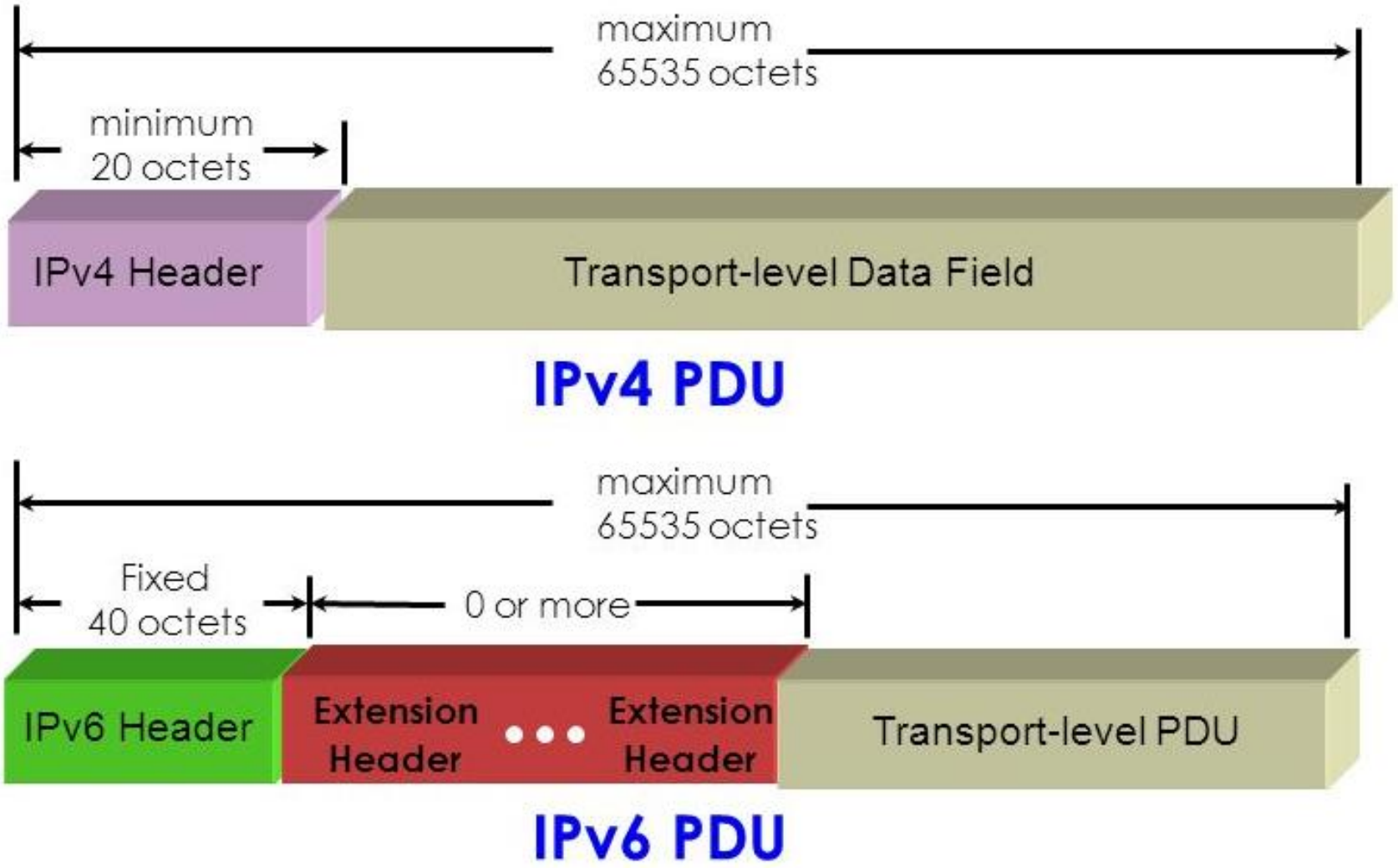
IPv4 Header
20 octets, 12 fields, incl. 3 flag bits +
fixed max number of options

Changed **Removed**



IPv6 Header
40 octets, 8 fields
+ Unlimited Chained Extension (options) Header

IPv6 vs. IPv4 Protocol Data Units (PDU)

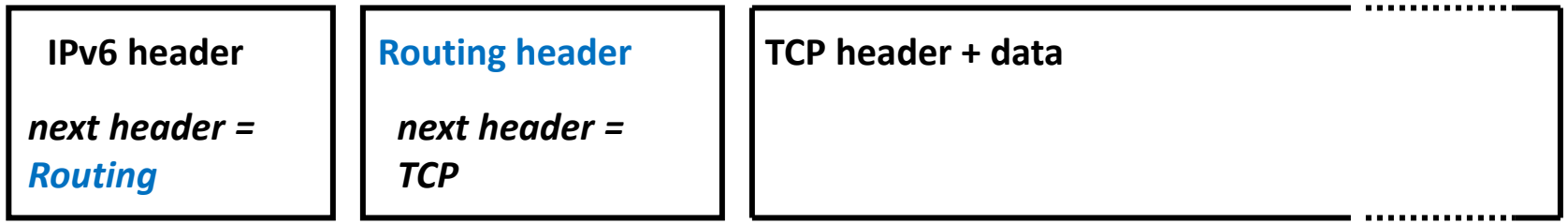


IPv6 Extension Headers

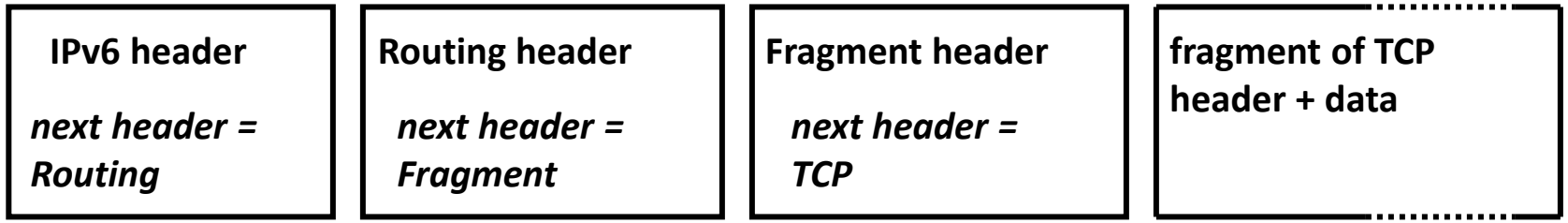
- The first field of all extension header is a 8-bit "next header" field



The next header refers to the protocol type if no optional head

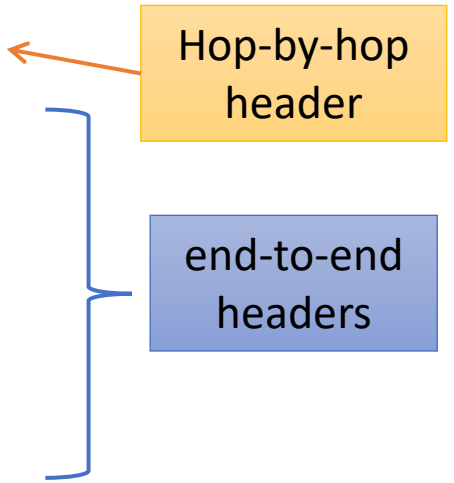


The next header specifies the type of the following header



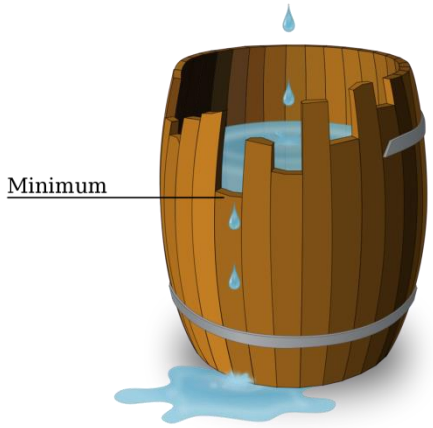
Extension Headers

- Generally processed only by the final destination, and neglected by the intermediate routers ➔ much lower overhead than IPv4 options processing
  - exception: Hop-by-Hop Options header
- Eliminated IPv4's 40-byte limit on options
  - in IPv6, limit is total packet size, or Path MTU in some cases
- Currently defined extension headers
  1. Hop-by-Hop Options
  2. Routing
  3. Fragment
  4. Authentication
  5. Encryption
  6. Destination Options



IPv6 Fragmentation and Reassembly

- Fragmentation of IPv6 datagrams is allowed and specified by an extension header.
- In IPv6, fragmentation is end-to-end, i.e., no fragmentation occurs in intermediate routers.
- The source/sender can either use the guaranteed minimum MTU of 1280 octets or perform *Path MTU Discovery* to identify the minimum MTU along the path to the destination.



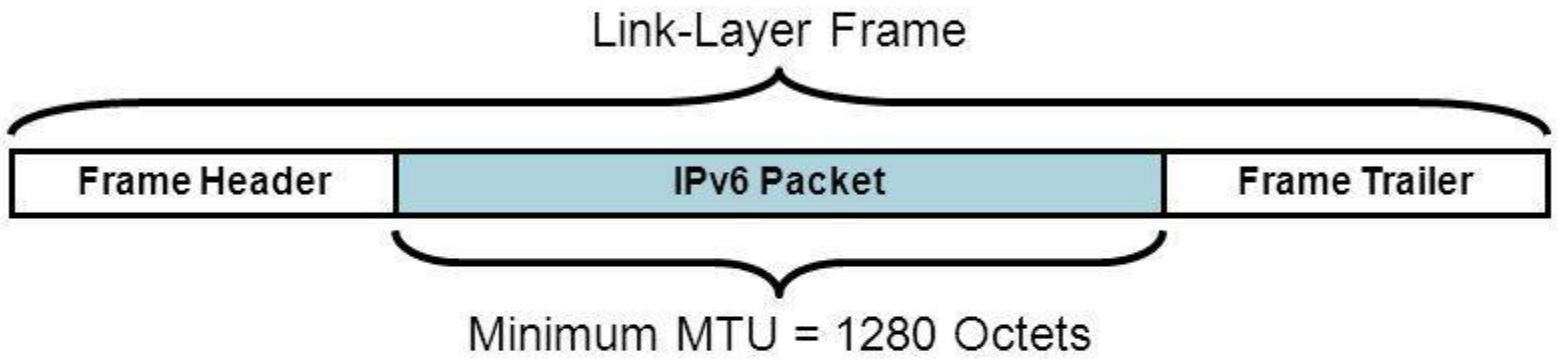


Minimum MTU

- Definitions:
  - link MTU — a link's Maximum Transmission Unit, i.e., the max IP packet size that can be transmitted over the link
  - path MTU — the minimum MTU of all the links in a path between a source and a destination

- Minimum link MTU for IPv6 is 1280 octets (versus 68 octets for IPv4)
- On links with MTU < 1280, link-specific fragmentation and reassembly must be used

Maximum Transmission Unit



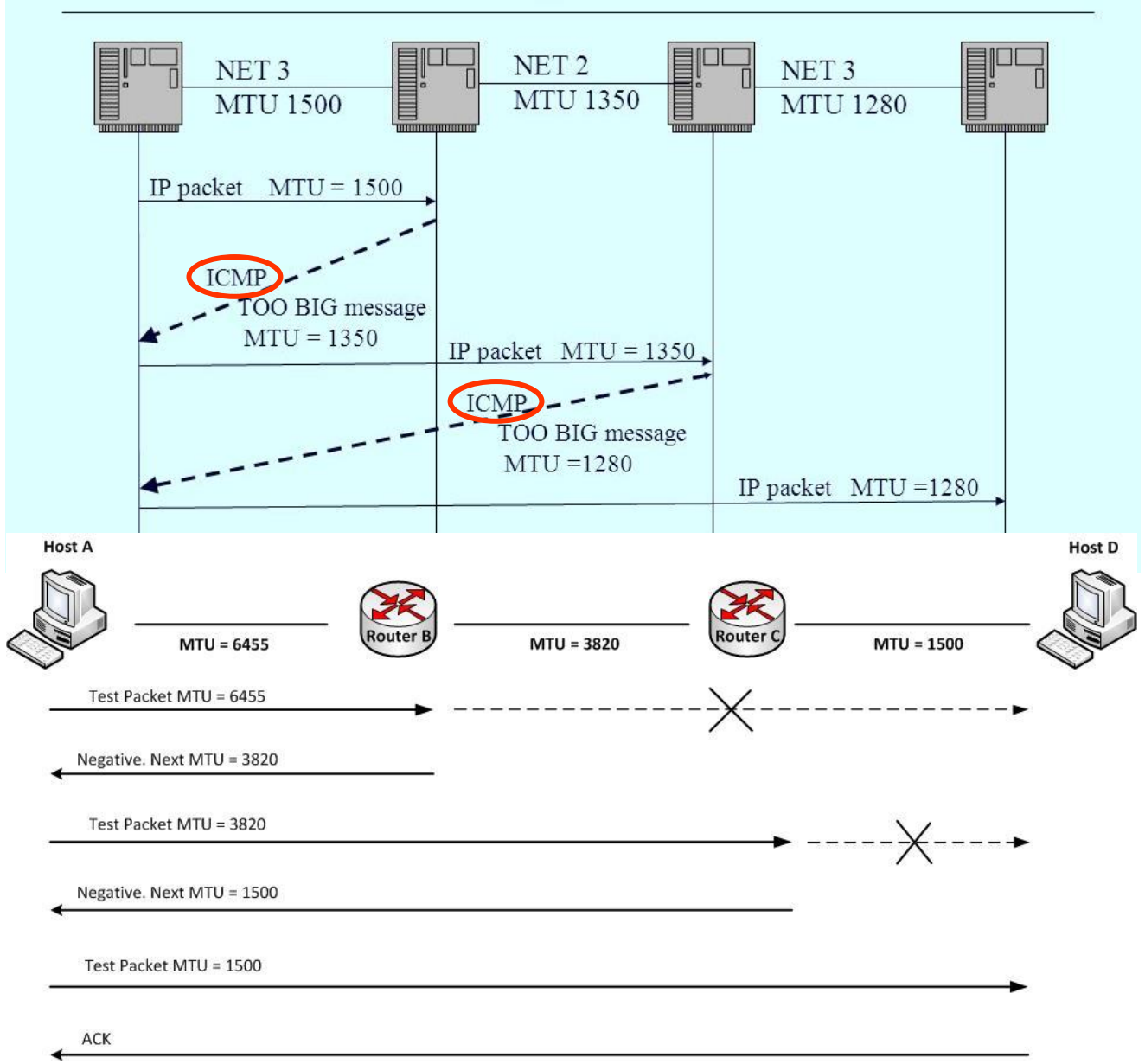
- **IPv4**
  - **MTU >= 68 octets**
- **IPv6**
  - **MTU >= 1280 octets**
  - **Path MTU used**

Path MTU Discovery (1)

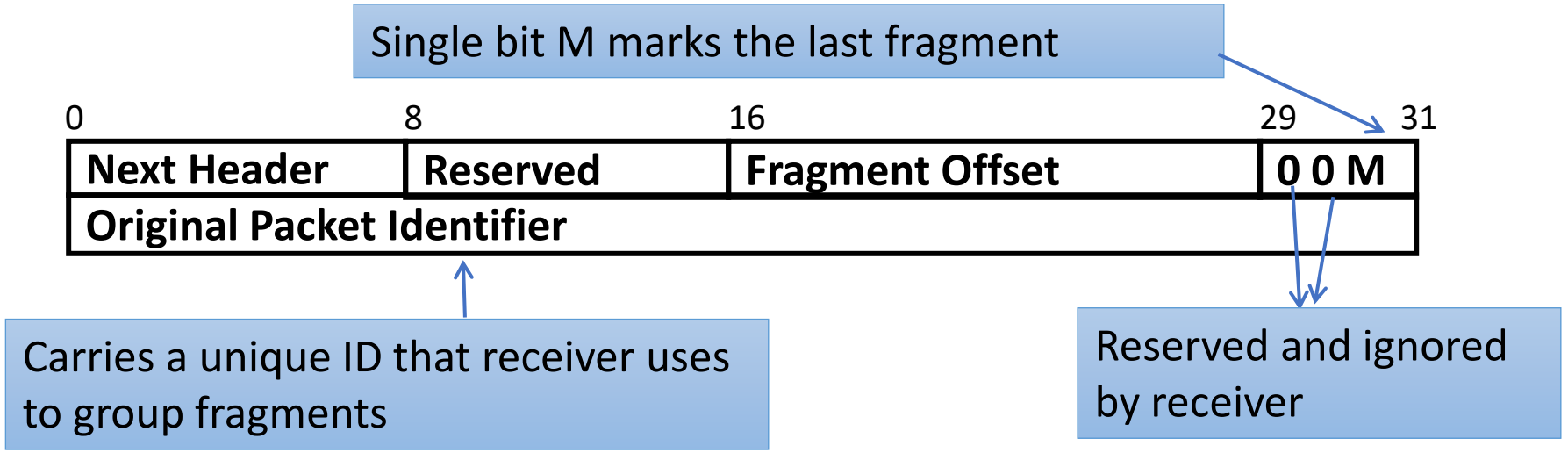
- Implementations are expected to perform path MTU discovery to send packets bigger than 1280 octets:
  - for each dest., start by assuming MTU of first-hop link
  - if a packet reaches a link in which it cannot fit, will invoke ICMP "packet too big" message to source, reporting the link's MTU; MTU is cached by source for specific destination
  - occasionally discard cached MTU to detect possible increase
- Minimal implementation can omit path MTU discovery as long as all packets kept ≤ 1280 octets
  - e.g., in a boot ROM implementation

Path MTU Discovery (2)





Fragment Extension Header

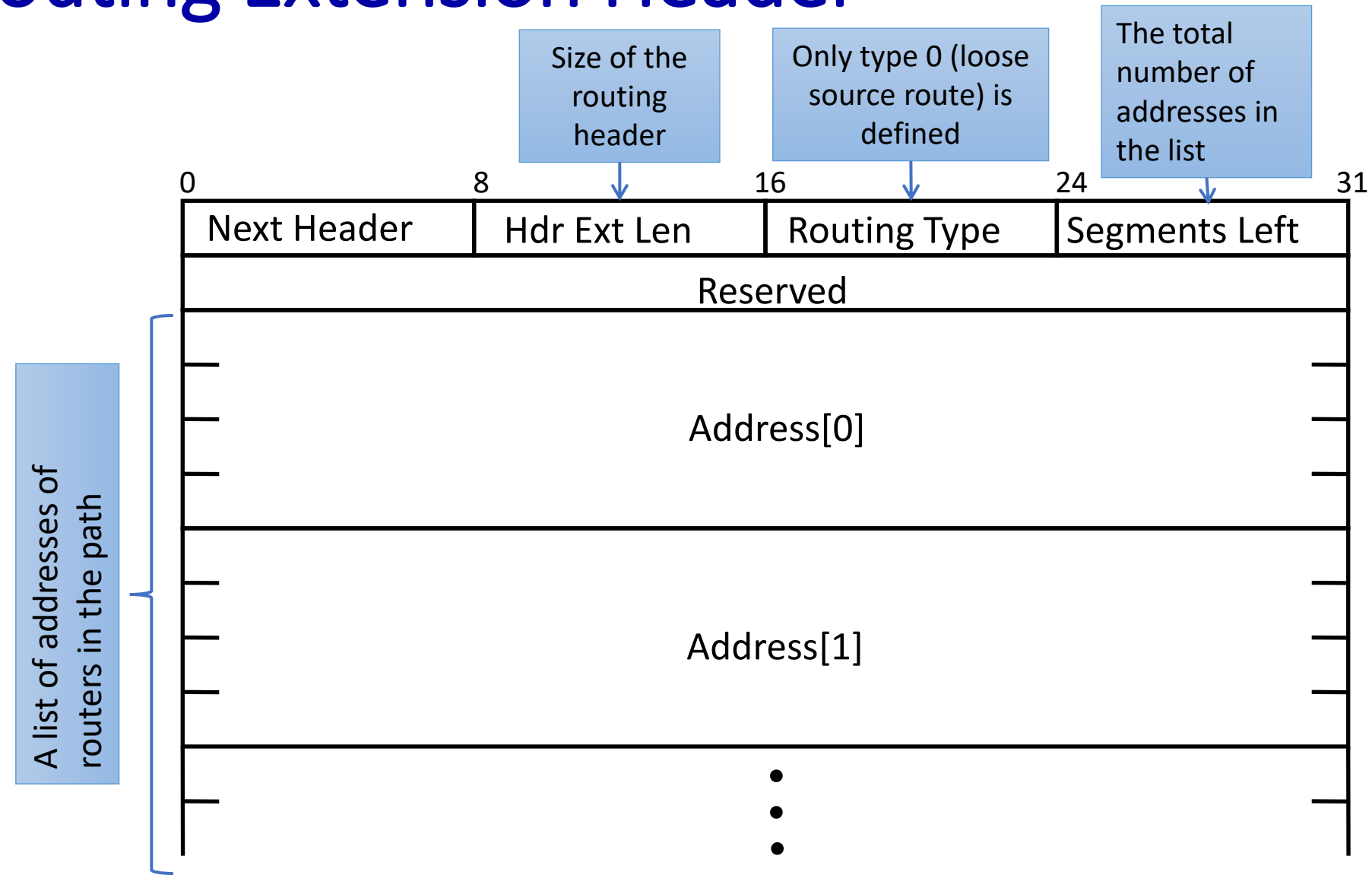


- IPv6 frag. is an end-to-end function; routers do not fragment packets en-route if too big—they send ICMP "packet too big" instead
- Each fragment must be a multiple of 8 octets
- IPv6 expands the IPv4 "Identifier" field to 32 bits to accommodate higher-speed networks.

Consequence of End-to-End Fragmentation

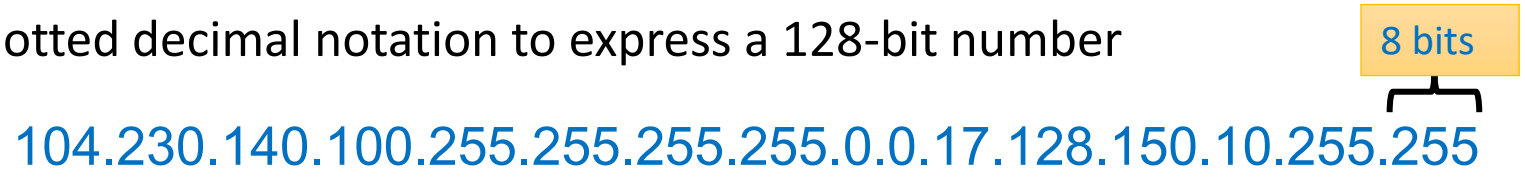
- End-to-end fragmentation alters the fundamental IPv4 assumption that routes change dynamically.
- In IPv6, routes cannot be changed as easily as those in IPv4 because a change in a route can also change the path MTU.
- When a router discovers that fragmentation is needed, it sends ICMP error message back to the source. The source performs another path MTU discovery and fragments datagrams again.

Routing Extension Header

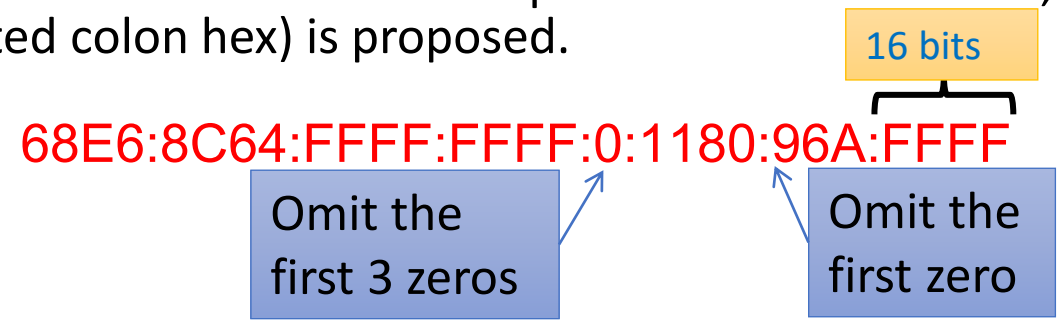


IPv6 Colon Hexadecimal Notation

- Q: How can humans read, enter and manipulate the long IPv6 addresses?
  - Binary notation is obviously untenable
  - An example using dotted decimal notation to express a 128-bit number



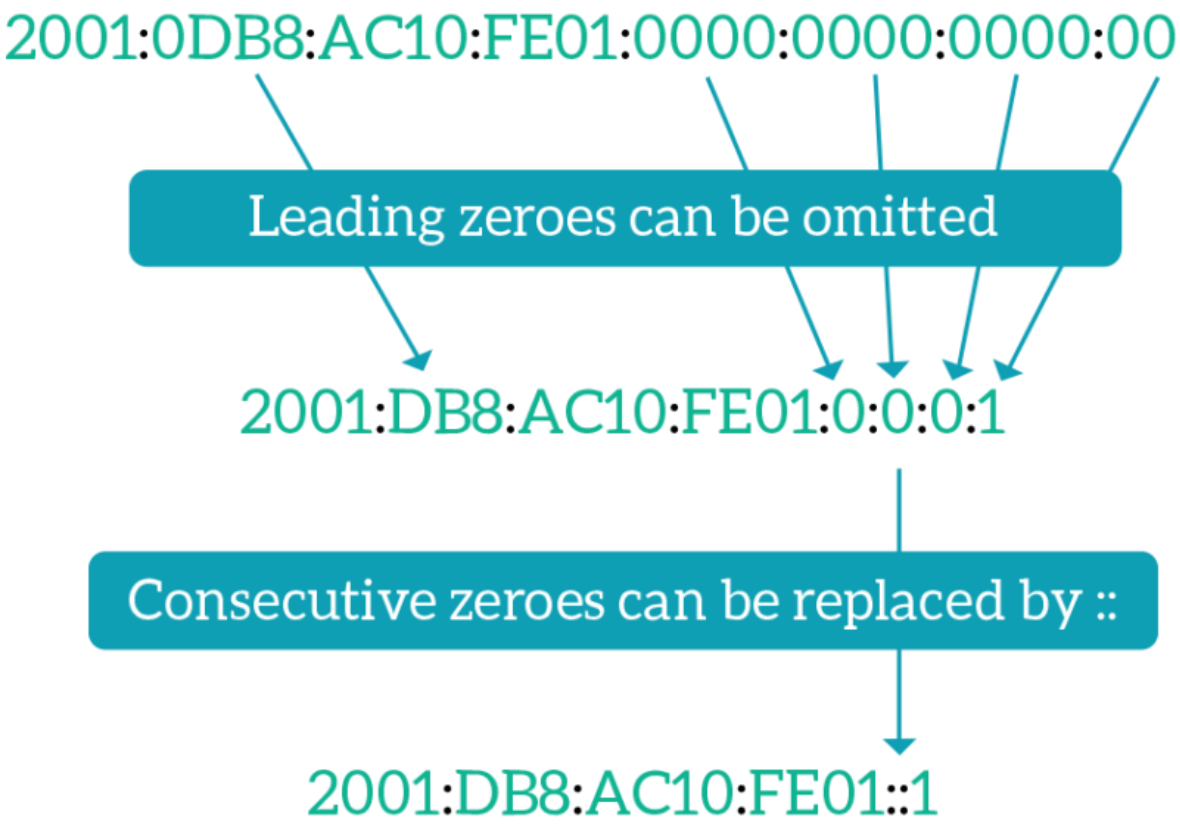
  - To help make addresses more compact and easier to enter, colon hexadecimal notation (abbreviated colon hex) is proposed.



Colon hex notation requires fewer digits and fewer separator characters than dotted decimal

Useful Techniques of Colon Hex Notation

- Zero compression (applied only once in any address)
  - FF05:0:0:0:0:0:0:B3 → FF05::B3
- Incorporation of dotted decimal suffixes (intended to be used during CIDR-like notation (specify network prefix)
  - 12AB::CD30:0:0:0:0/60 = 12AB:0:0:CD30::/60

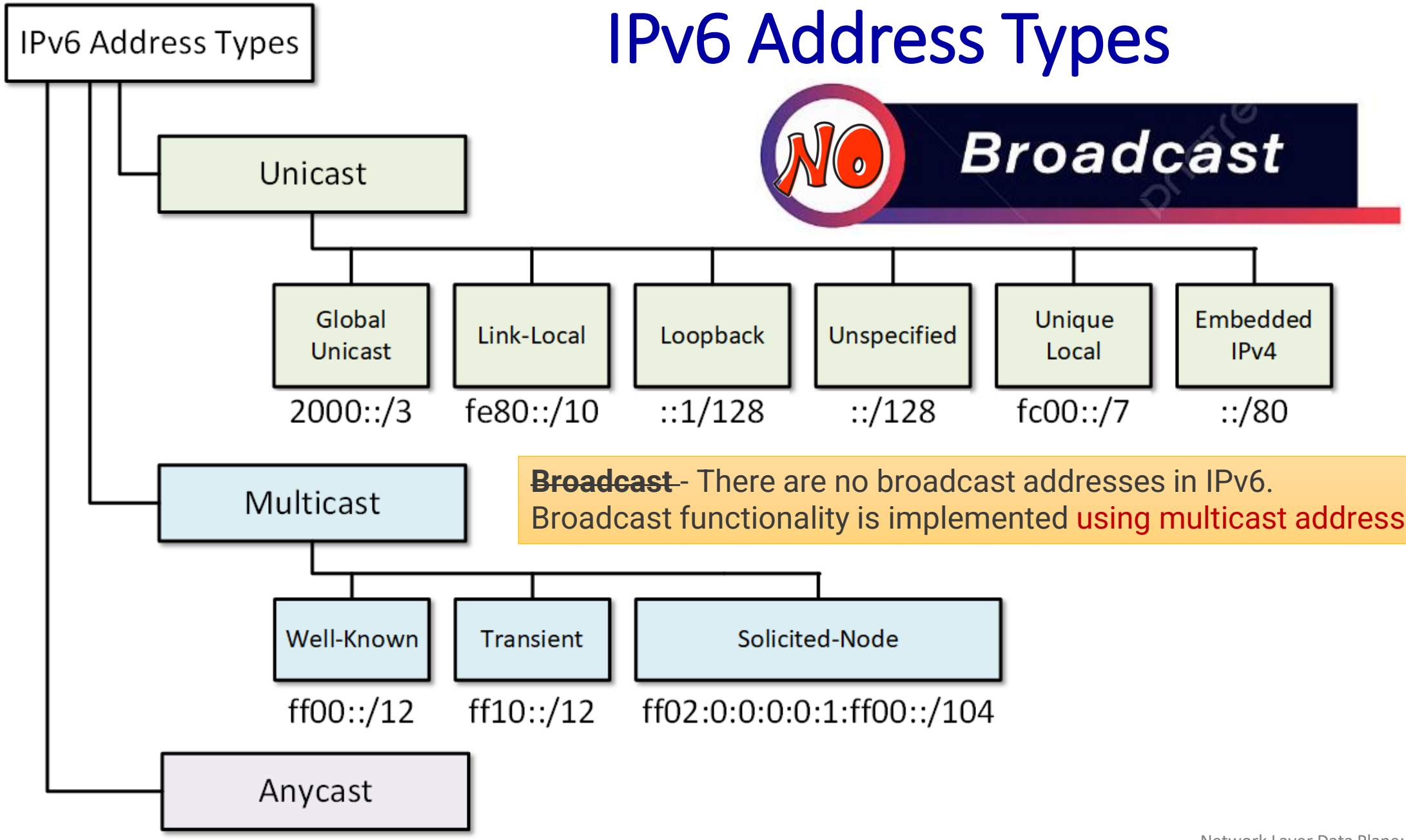


IPv6 Address Format

X:X:X:X:X:X:X:X
where X = 0000 ... FFFF (hex)

- 2001:0DB8:0000:0000:0008:8000:0000:417A
- 2001:DB8:0:0:8:8000:0:417A
- 2001:DB8::8:8000:0:417A
- 2001:DB8:0:0:8:8000::417A

IPv6 Address Types



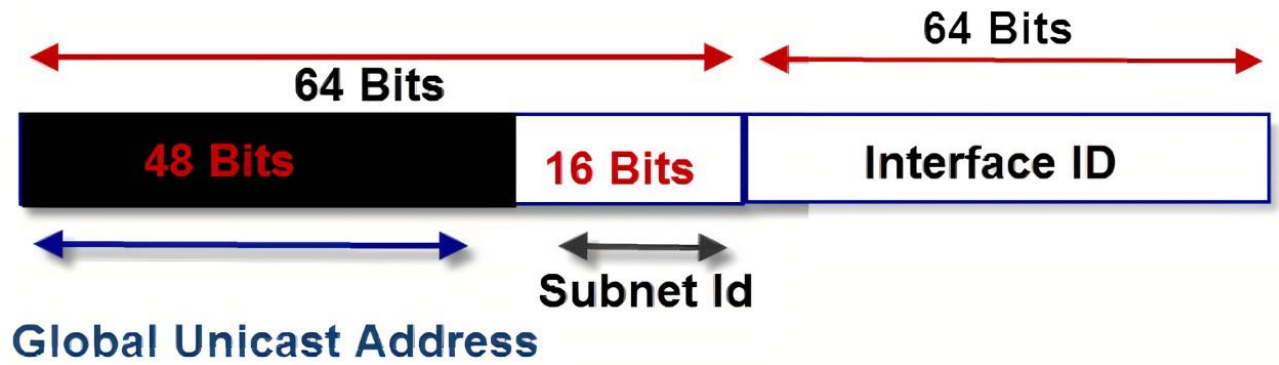
~~**Broadcast**~~ - There are no broadcast addresses in IPv6.
Broadcast functionality is implemented using multicast addresses.

Example of the IPV6 address format

FDEC : BA98 : 7654 : 3210 : ADBF : BBFF : 2922 : FFFF

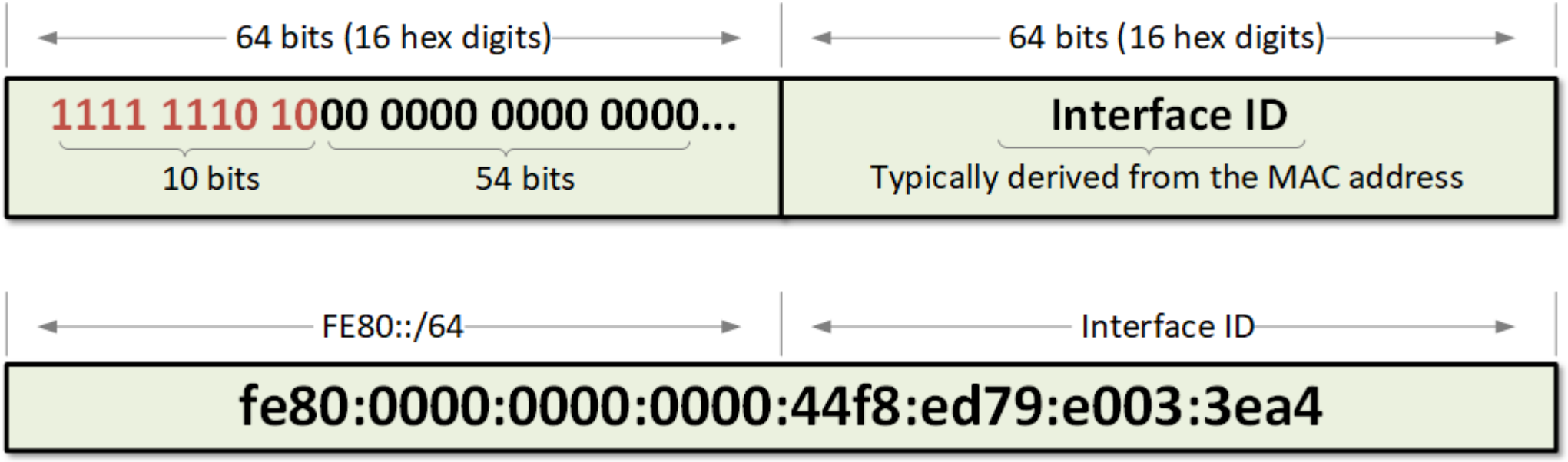
Global Prefix **Subnet** **Interface ID**

Here each block is denoted in hexadecimal digits and each block is separated by a colon.



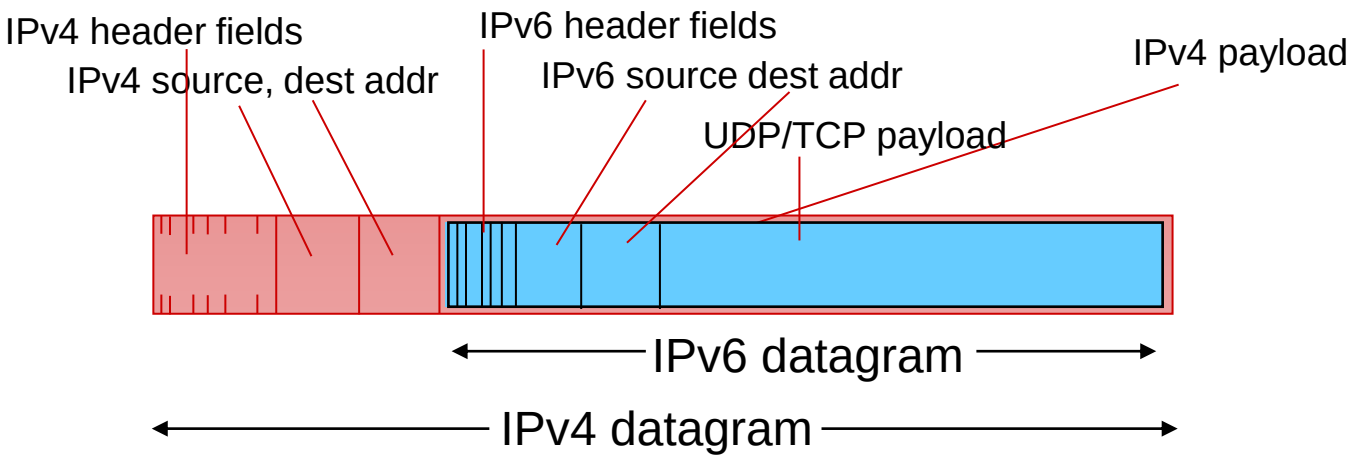
IPv6 Address Structure

Link Local Unicast IPv6 Address



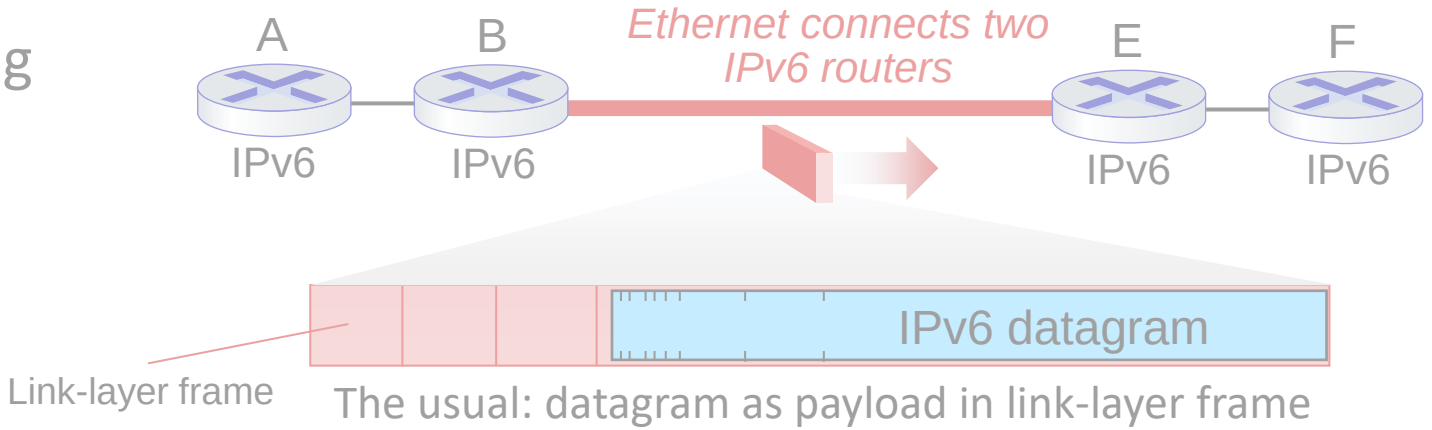
Transition from IPv4 to IPv6

- As of 2018, all five RIRs have run out of IPv4 addresses.
- not all routers can be upgraded simultaneously
  - no “flag days”
  - how will network operate with mixed IPv4 and IPv6 routers?
- tunneling: IPv6 datagram carried as *payload* in IPv4 datagram among IPv4 routers (“packet within a packet”)
  - tunneling used extensively in other contexts (4G/5G)

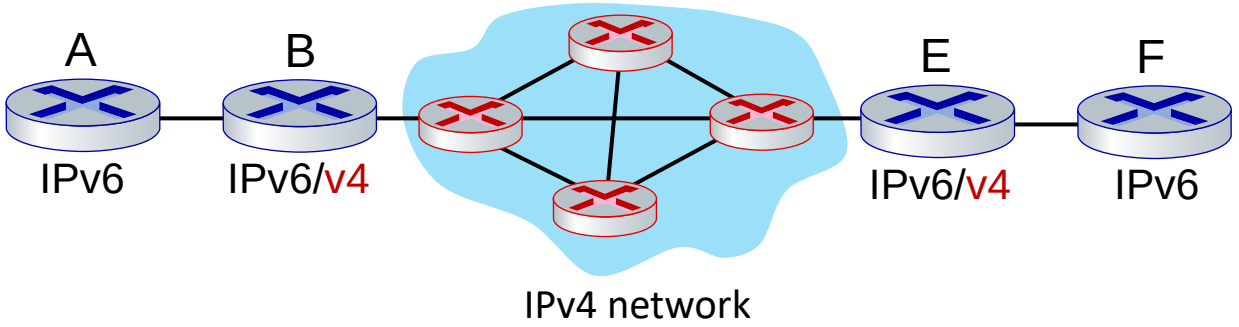


Tunneling and encapsulation

Ethernet connecting two IPv6 routers:

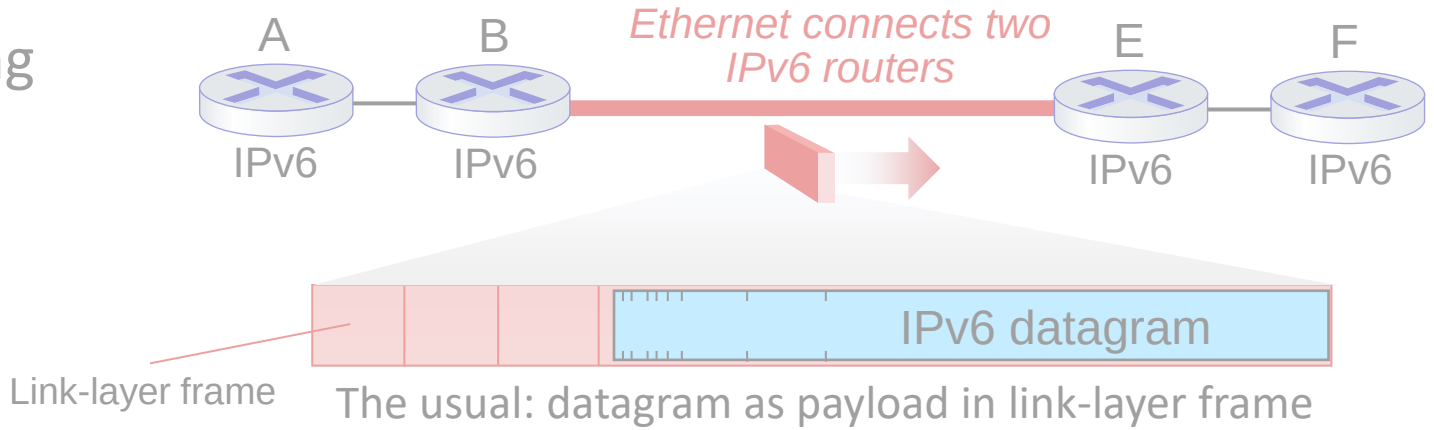


IPv4 network connecting two IPv6 routers



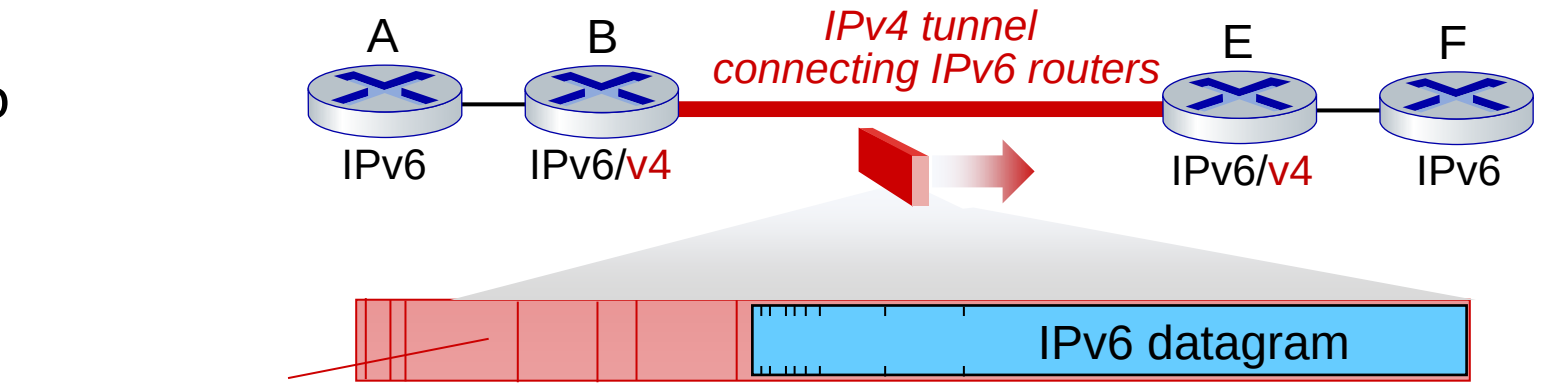
Tunneling and encapsulation

Ethernet connecting two IPv6 routers:



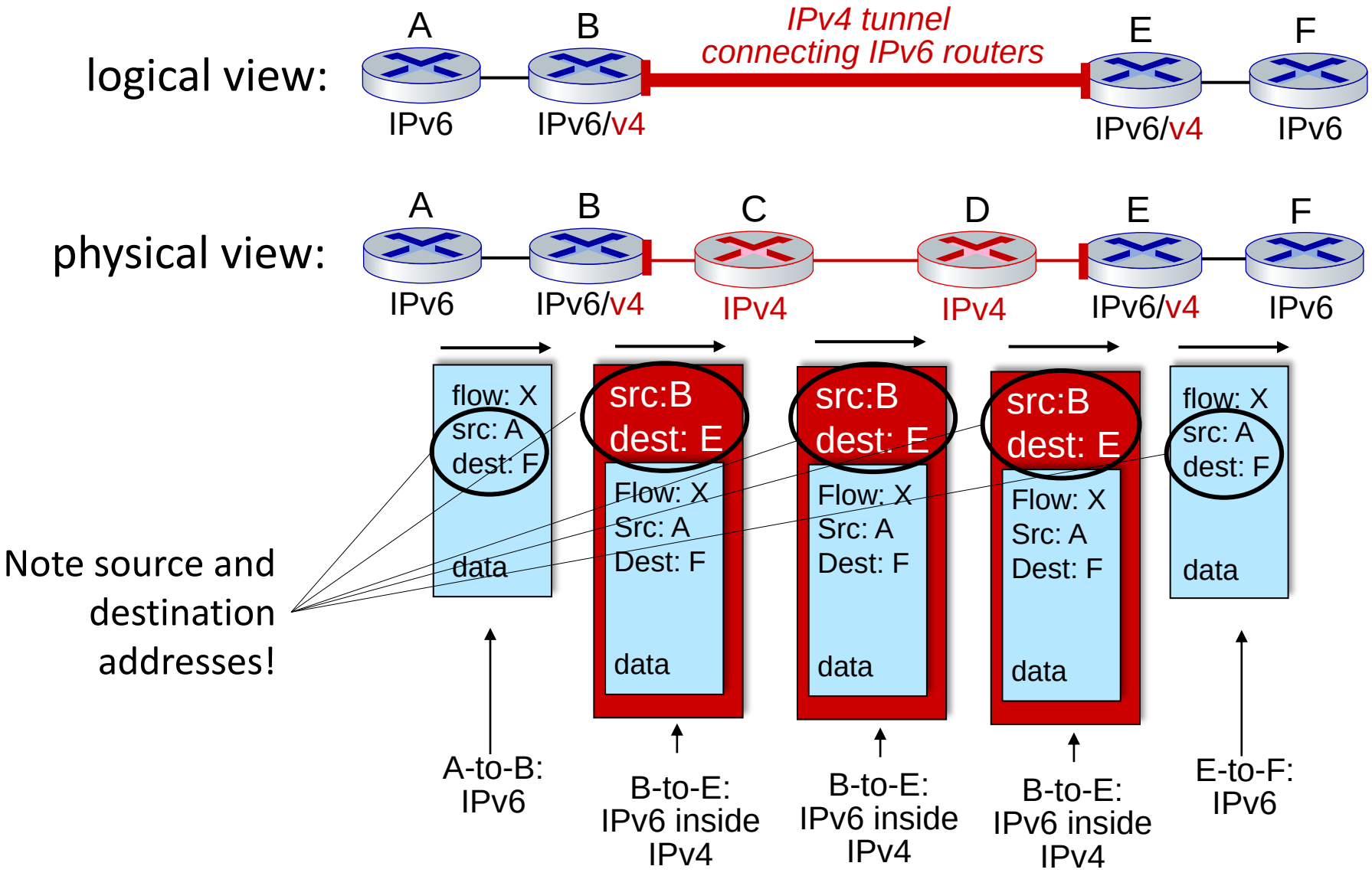
The usual: datagram as payload in link-layer frame

IPv4 tunnel connecting two IPv6 routers

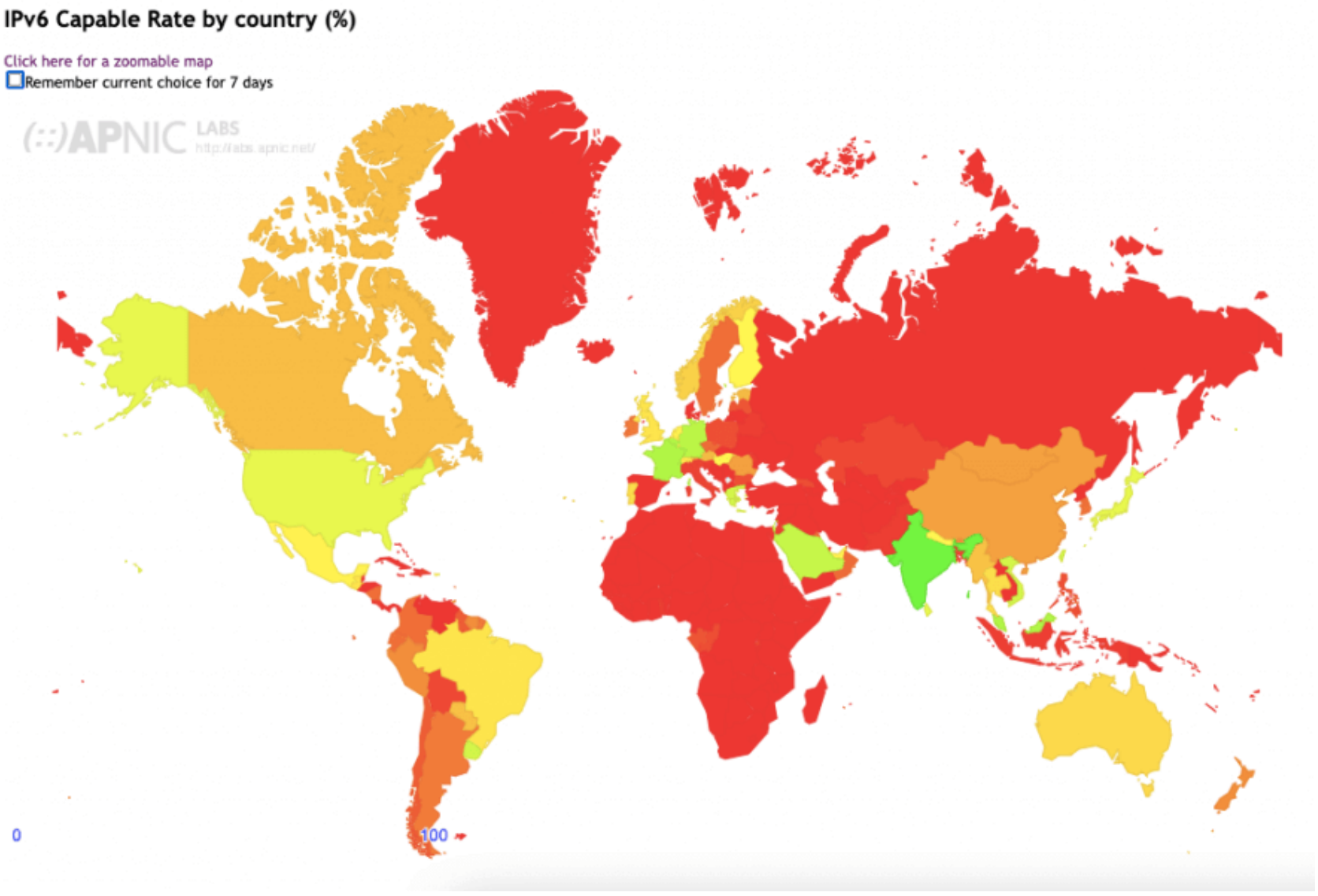


tunneling: IPv6 datagram as payload in a IPv4 datagram

Tunneling


logical view:
A
B
IPv4 tunnel
connecting IPv6 routers
E
F
IPv6
IPv6/v4
IPv6/v4
IPv6
physical view:
A
B
C
D
E
F
IPv6
IPv6/v4
IPv4
IPv4
IPv6/v4
IPv6
Note source and
destination
addresses!
flow: X
src: A
dest: F
data
src:B
dest: E
Flow: X
Src: A
Dest: F
data
src:B
dest: E
Flow: X
Src: A
Dest: F
data
src:B
dest: E
Flow: X
Src: A
Dest: F
data
flow: X
src: A
dest: F
data
A-to-B:
IPv6
B-to-E:
IPv6 inside
IPv4
B-to-E:
IPv6 inside
IPv4
B-to-E:
IPv6 inside
IPv4
E-to-F:
IPv6

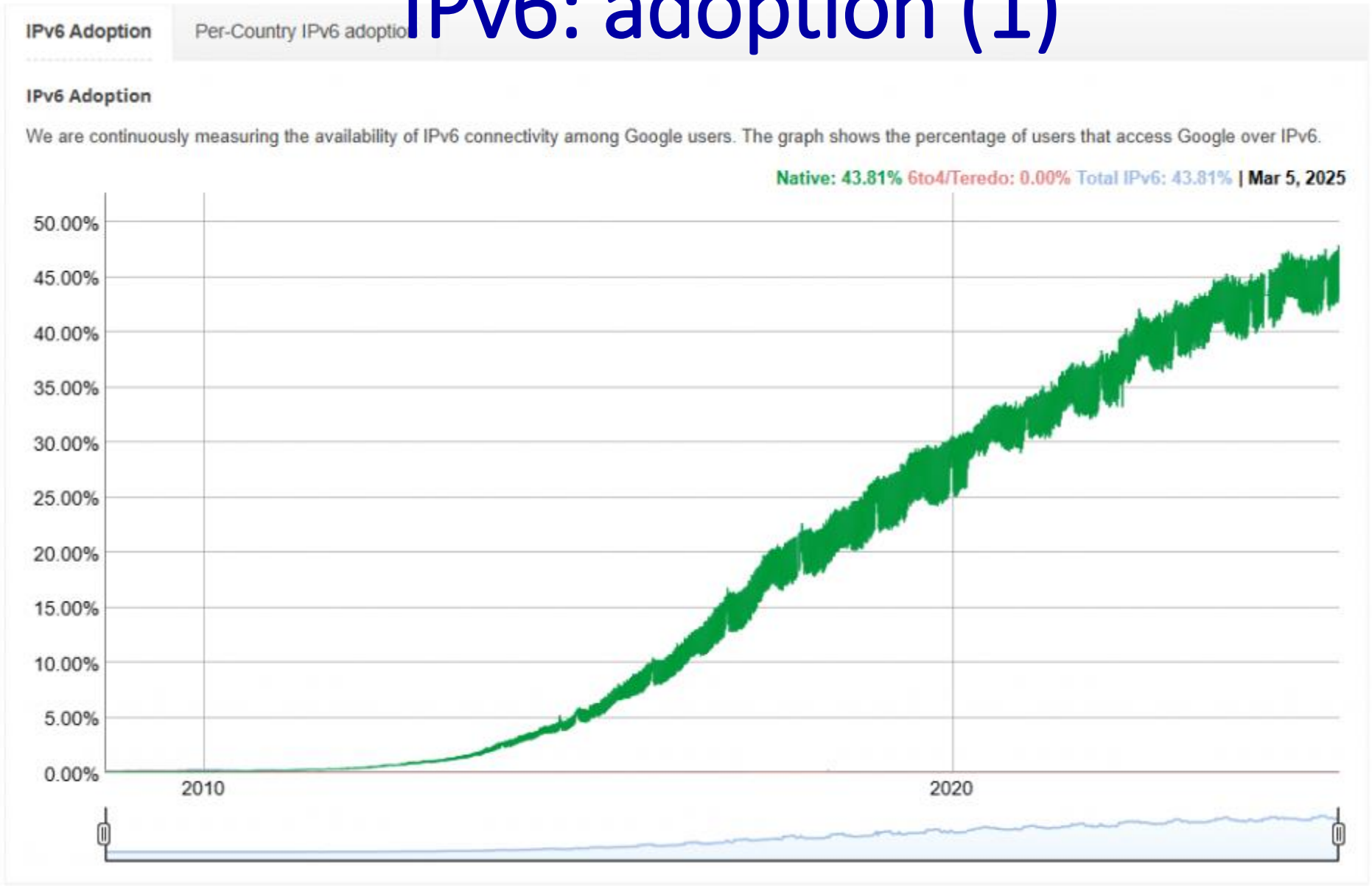
IPv6 capable rate by country (%)





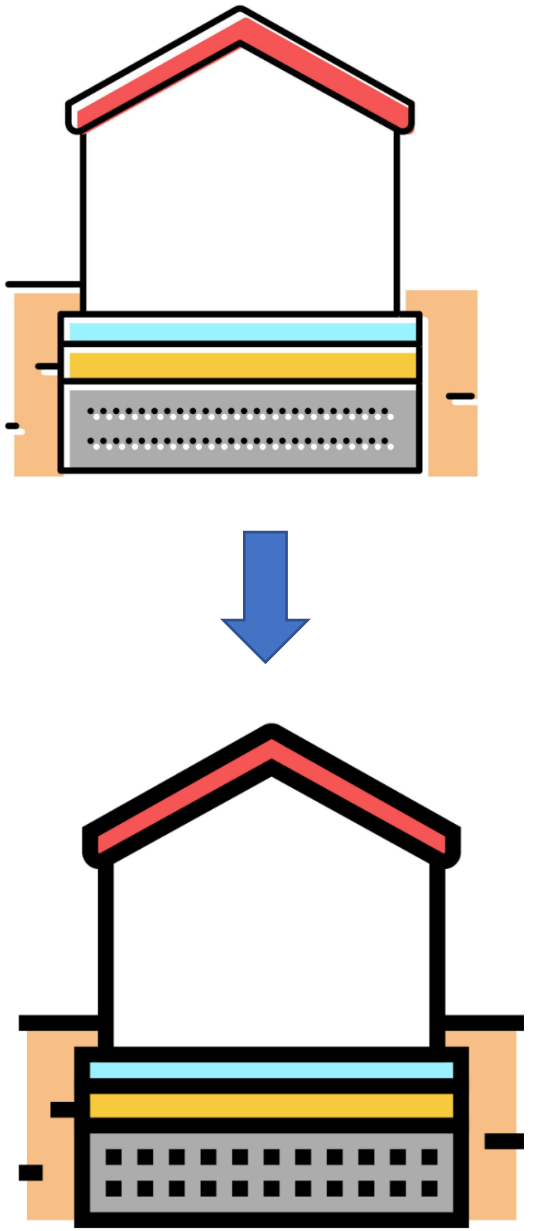
- **Green, Yellow to Light Orange Colors.** Countries with high IPv6 adoption rates have modern Internet infrastructure and have widely implemented IPv6. The leaders are clearly India, France, and Germany.
- **Dark Orange to Red.** Countries with moderate to low IPv6 adoption rates face economic constraints, lack of expertise, or need regulatory incentives.

IPv6: adoption (1)



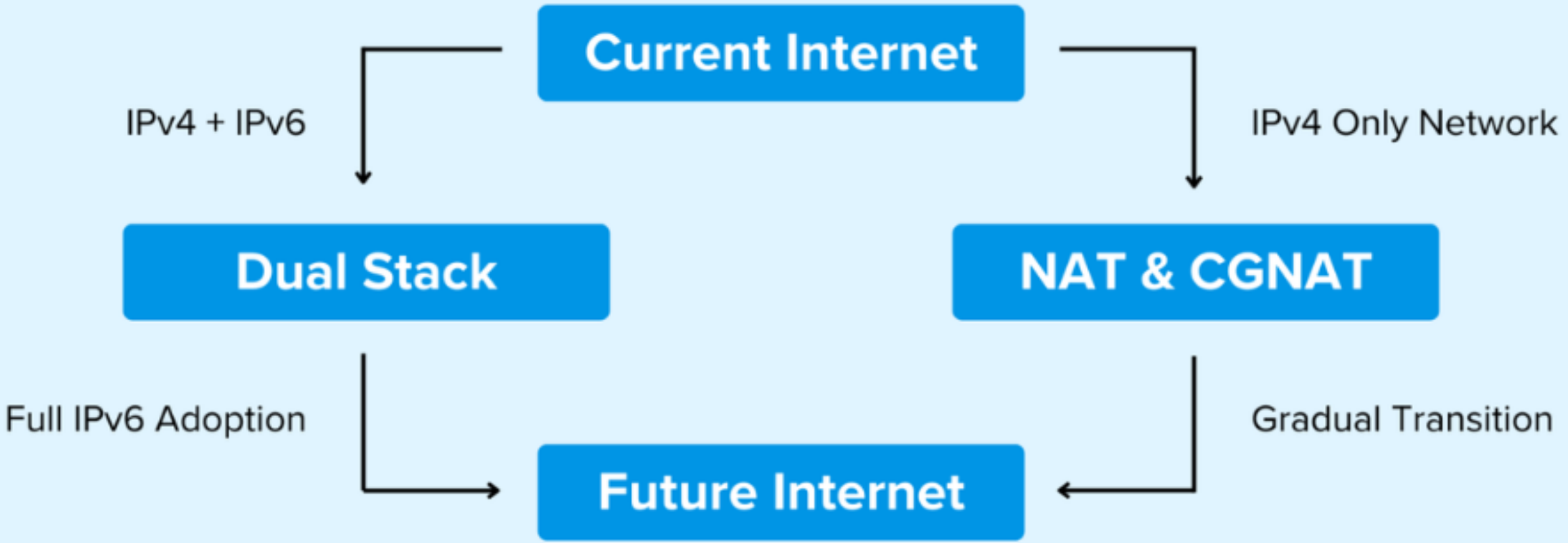
IPv6: adoption

- The IPv6 protocol was first officially published in 1995.
- June 6, 2012 was declared IPv6 Launch Day by The Internet Society
  - a day meant to officially inaugurate the beginning of the transition of the Internet to IPv6.
- Long (long!) time for deployment, use
  - 30 years and counting!
  - think of application-level changes in last 30 years: WWW, social media, streaming media, gaming, telepresence, …
  - *Why?*



IPv6 Adoption: Myths and Realities (catchpoint.com)

Initiatives to Improve IPv6 Adoption



Network layer: "data plane" roadmap

- Network layer: overview
  - data plane
  - control plane
- What's inside a router
  - input/output ports, switching
  - buffer management
  - scheduling
- IP: the Internet Protocol
  - datagram format, addressing
  - network address translation (NAT)
  - IPv6

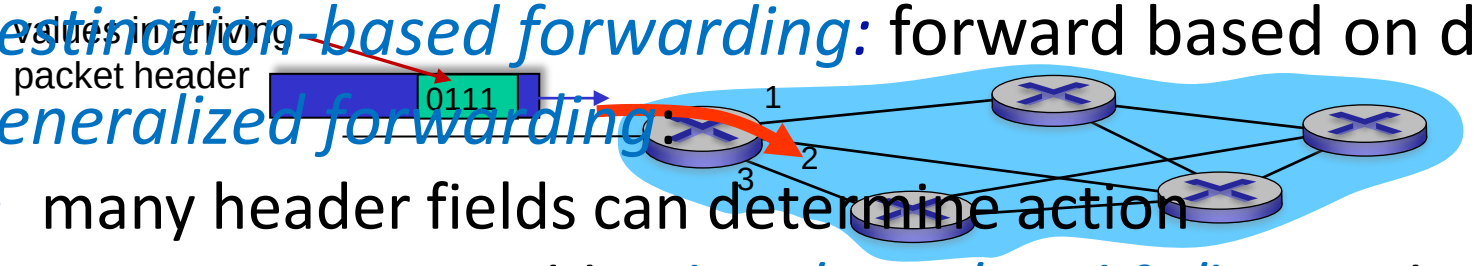


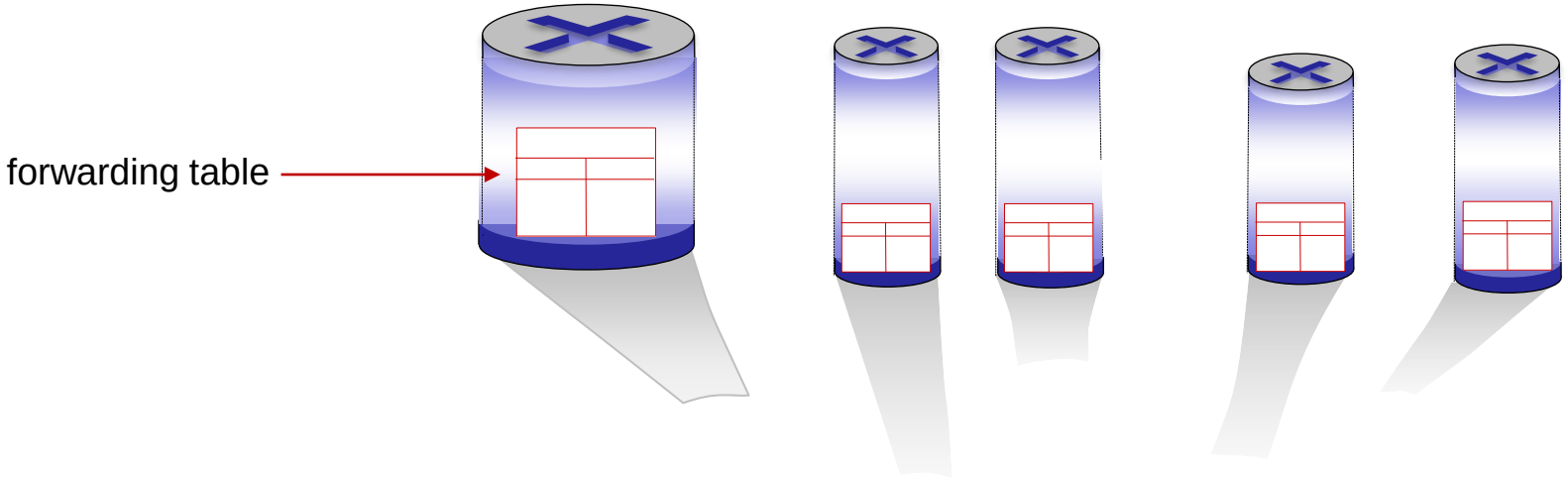
- Generalized Forwarding
  - Match+action
  - OpenFlow
  - Middleboxes
- Internet architecture

Generalized forwarding: match plus action

Review: each router contains a forwarding table

- "match plus action" abstraction: match bits in arriving packet, take action
  - *destination-based forwarding:* forward based on dest. IP address
  - *generalized forwarding:*
    - many header fields can determine action
    - many action possible: drop/copy/modify/log packet

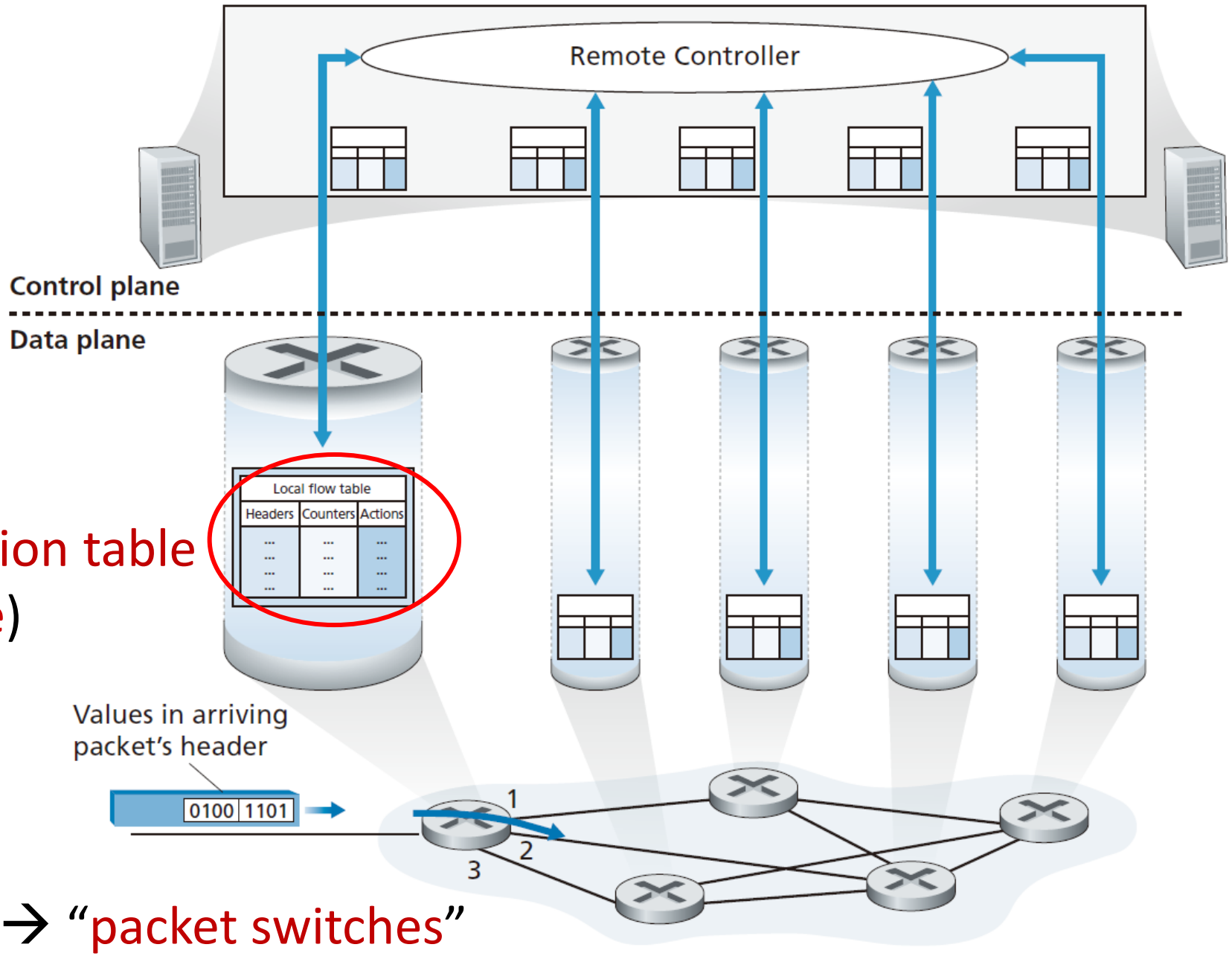




Generalized forwarding:

- Each packet switch contains a match-plus-action table
  - that is computed and distributed by a remote controller.

Match-plus-action table
(aka: flow table)

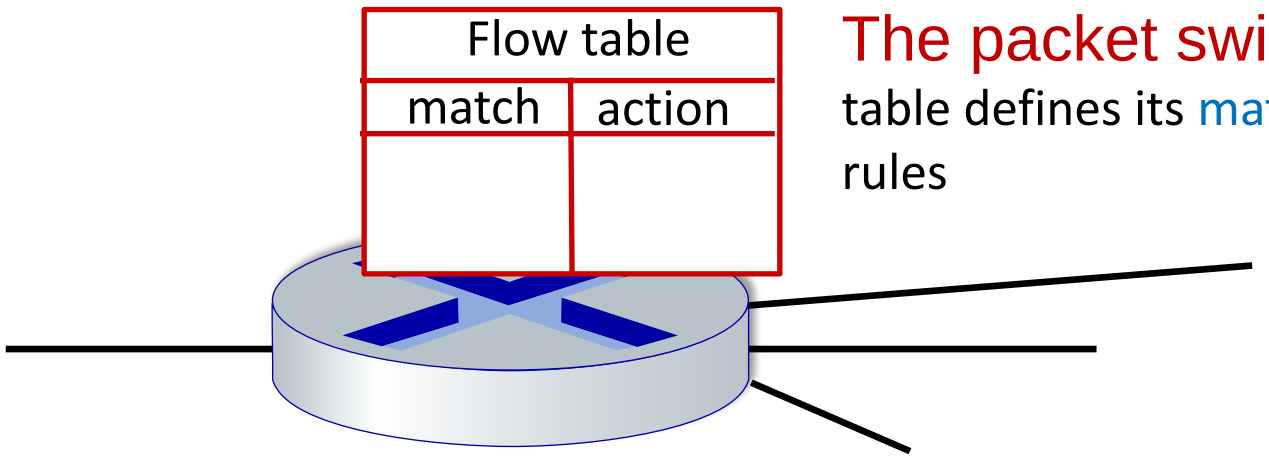


The forwarding devices → "packet switches"

Flow table abstraction

- flow: defined by header field values (in link-, network-, transport-layer fields)
- generalized forwarding: simple packet-handling rules
  - match: pattern values in packet header fields
  - actions: for matched packet: drop, forward, modify matched packet or send matched packet to controller
  - priority: disambiguate overlapping patterns
  - counters: #bytes and #packets

rather than layer 3 "routers" or layer 2 "switches."



The packet switch's flow table defines its match + action rules

the flow table is essentially an API:
the abstraction through which an individual packet switch's behavior can be programmed

Flow table abstraction

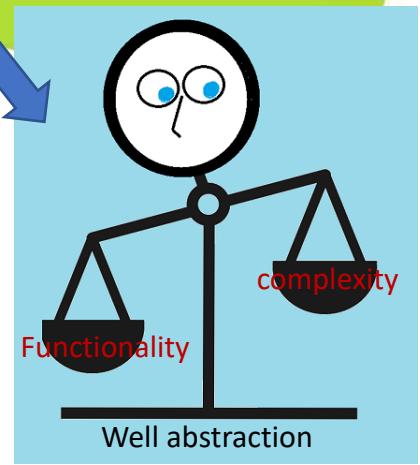
- flow: defined by header fields
- generalized forwarding: simple packet-handling rules
  - match: pattern values in packet header fields
  - actions: for matched packet: drop, forward, modify matched packet or send matched packet to controller
  - priority: disambiguate overlapping patterns
  - counters: #bytes and #packets



OpenFlow 1.0: flow table entries

An OpenFlow-enabled device
= a router (layer-3 device)
= a switch (layer-2 device)

Not all fields (e.g., TTL) in an IP header can be matched



OpenFlow's match abstraction for *three* layers of protocol headers

Match	Action	Stats

Packet + byte counters

1. Forward packet to port(s)
2. Drop packet
3. Modify fields in header(s)
4. Encapsulate and forward to controller

Header fields to match:

Ingress Port	Src MAC	Dst MAC	Eth Type	VLAN ID	VLAN Pri	IP Src	IP Dst	IP Proto	IP ToS	TCP/UDP Src Port	TCP/UDP Dst Port

Link layer | Network layer | Transport layer

OpenFlow: examples

Destination-based forwarding:

Switch Port	MAC src	MAC dst	Eth type	VLAN ID	VLAN Pri	IP Src	IP Dst	IP Prot	IP ToS	TCP s-port	TCP d-port	Action
*	*	*	*	*	*	*	51.6.0.8	*	*	*	*	port6

IP datagrams destined to IP address 51.6.0.8 should be forwarded to router output port 6

Firewall:

Switch Port	MAC src	MAC dst	Eth type	VLAN ID	VLAN Pri	IP Src	IP Dst	IP Prot	IP ToS	TCP s-port	TCP d-port	Action
*	*	*	*	*	*	*	*	*	*	*	22	drop

Block (do not forward) all datagrams destined to TCP port 22 (ssh port #)

Switch Port	MAC src	MAC dst	Eth type	VLAN ID	VLAN Pri	IP Src	IP Dst	IP Prot	IP ToS	TCP s-port	TCP d-port	Action
*	*	*	*	*	*	128.119.1.1	*	*	*	*	*	drop

Block (do not forward) all datagrams sent by host 128.119.1.1

OpenFlow: examples

Layer 2 destination-based forwarding:

Switch Port	MAC src	MAC dst	Eth type	VLAN ID	VLAN Pri	IP Src	IP Dst	IP Prot	IP ToS	TCP s-port	TCP d-port	Action
*	*	22:A7:23:11:E1:02	*	*	*	*	*	*	*	*	*	port3

layer 2 frames with destination MAC address 22:A7:23:11:E1:02 should be forwarded to output port 3

OpenFlow abstraction

- Match + action: abstraction unifies different kinds of devices

Router

- ***match:*** longest destination IP prefix
- ***action:*** forward out a link

Switch

- ***match:*** destination MAC address
- ***action:*** forward or flood

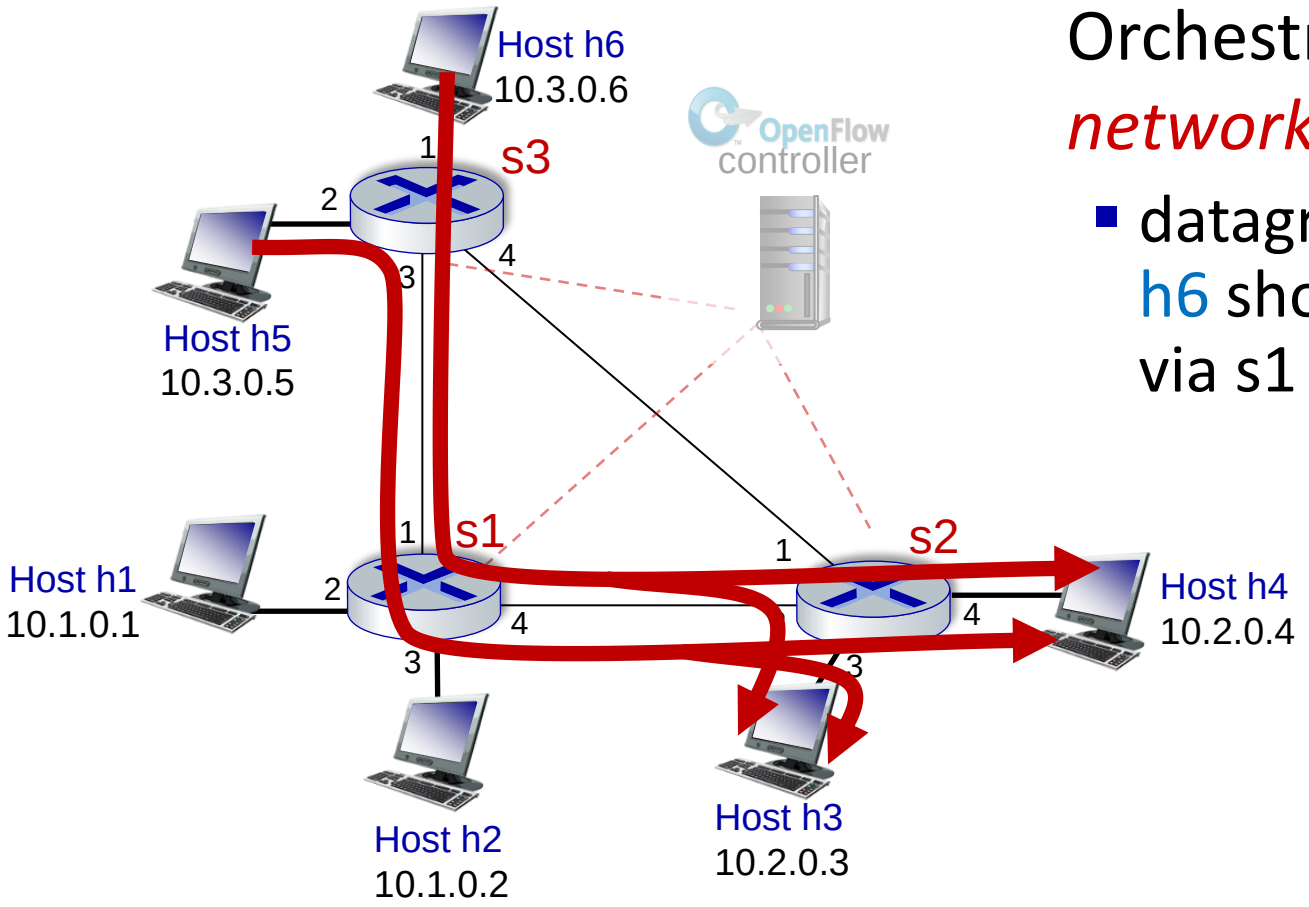
Firewall

- ***match*****:** IP addresses and TCP/UDP port numbers
- ***action:*** permit or deny

NAT

- ***match:*** IP address and port
- ***action:*** rewrite address and port

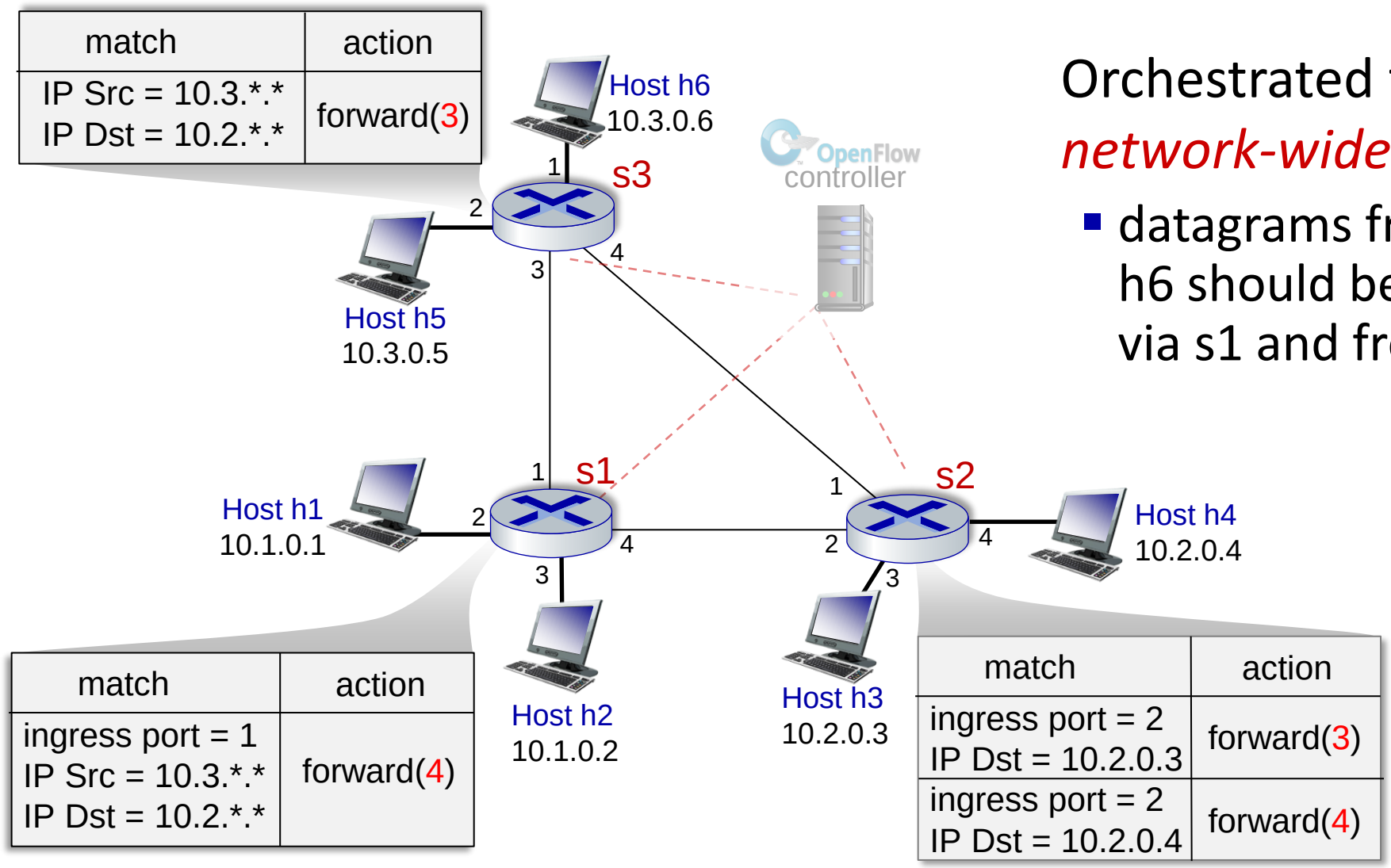
OpenFlow example 1: simple forwarding



Orchestrated tables can create *network-wide* behavior, e.g.,:

- datagrams from hosts h5 and h6 should be sent to h3 or h4, via s1 and from there to s2

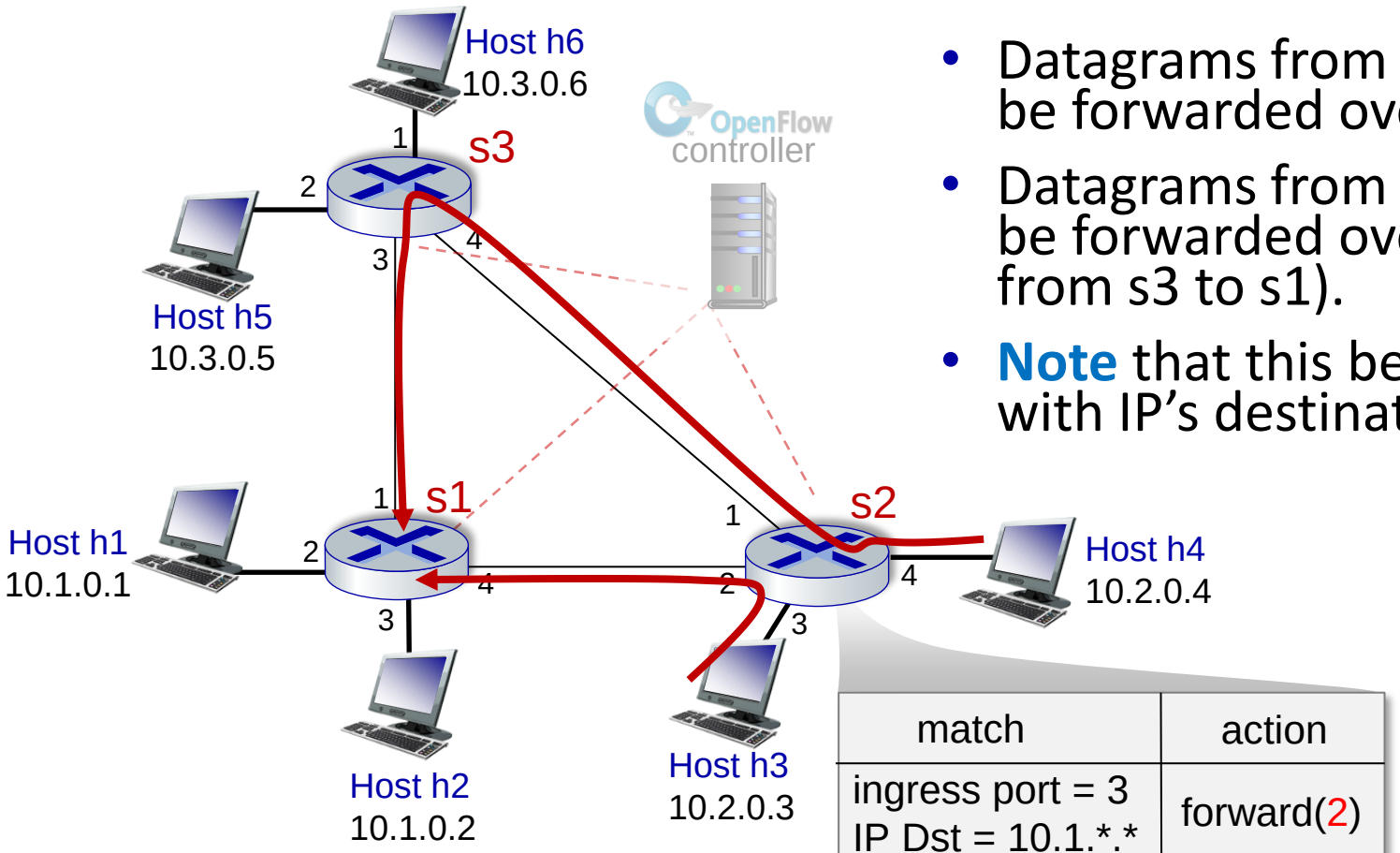
OpenFlow example 1: simple forwarding



Orchestrated tables can create *network-wide* behavior, e.g.,:

- datagrams from hosts h5 and h6 should be sent to h3 or h4, via s1 and from there to s2

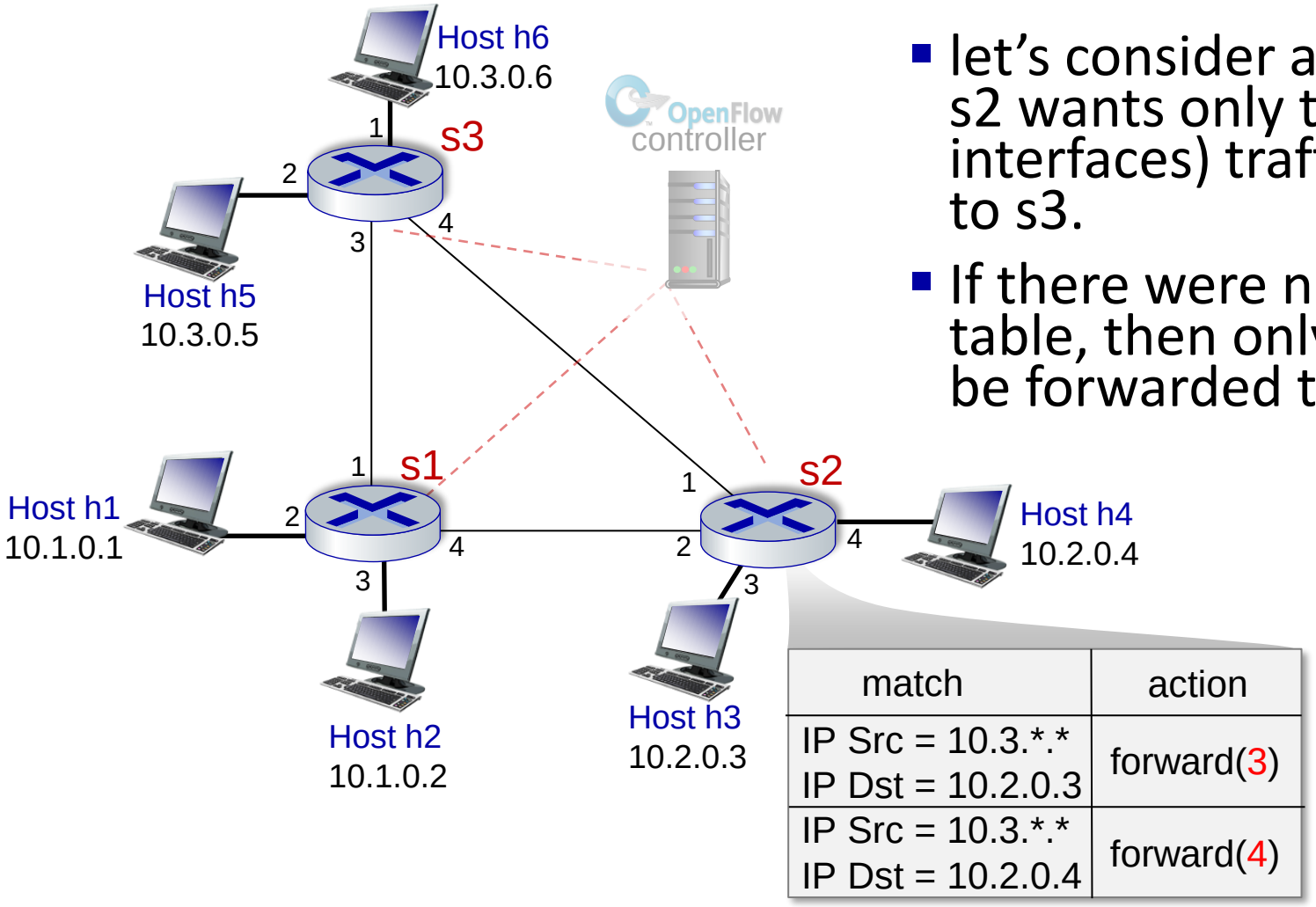
OpenFlow example 2: load balancing



- Datagrams from h3 destined to 10.1.*.* are to be forwarded over the direct link s2 → s1
- Datagrams from h4 destined to 10.1.*.* are to be forwarded over the link s2 → s3 (and then from s3 to s1).
- **Note** that this behavior couldn't be achieved with IP's destination-based forwarding.

match	action
ingress port = 3 IP Dst = 10.1.*.*	forward(2)
ingress port = 4 IP Dst = 10.1.*.*	forward(1)

OpenFlow example 3: Firewalling



match	action
IP Src = 10.3.*.* IP Dst = 10.2.0.3	forward(3)
IP Src = 10.3.*.* IP Dst = 10.2.0.4	forward(4)

- let's consider a firewall scenario in which s2 wants only to receive (on any of its interfaces) traffic sent from hosts attached to s3.
- If there were no other entries in s2's flow table, then only traffic from 10.3.*.* would be forwarded to the hosts attached to s2.

Generalized forwarding: summary

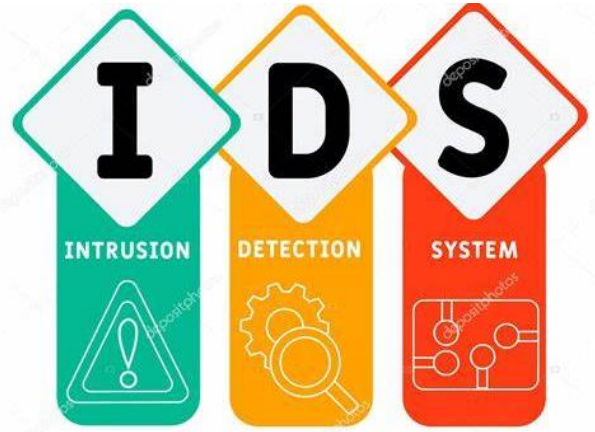
- "match plus action" abstraction: match bits in arriving packet header(s) in any layers, take action
  - matching over many fields (link-, network-, transport-layer)
  - local actions: drop, forward, modify, or send matched packet to controller
  - "program" *network-wide* behaviors
- simple form of "network programmability"
  - programmable, per-packet "processing"
  - *historical roots:* active networking
  - *today:* more generalized programming

Middleboxes

Middlebox (RFC 3234)

"any intermediary box performing functions apart from normal, standard functions of an IP router on the data path between a source host and destination host"

Three Types Of Services Performed By Middleboxes

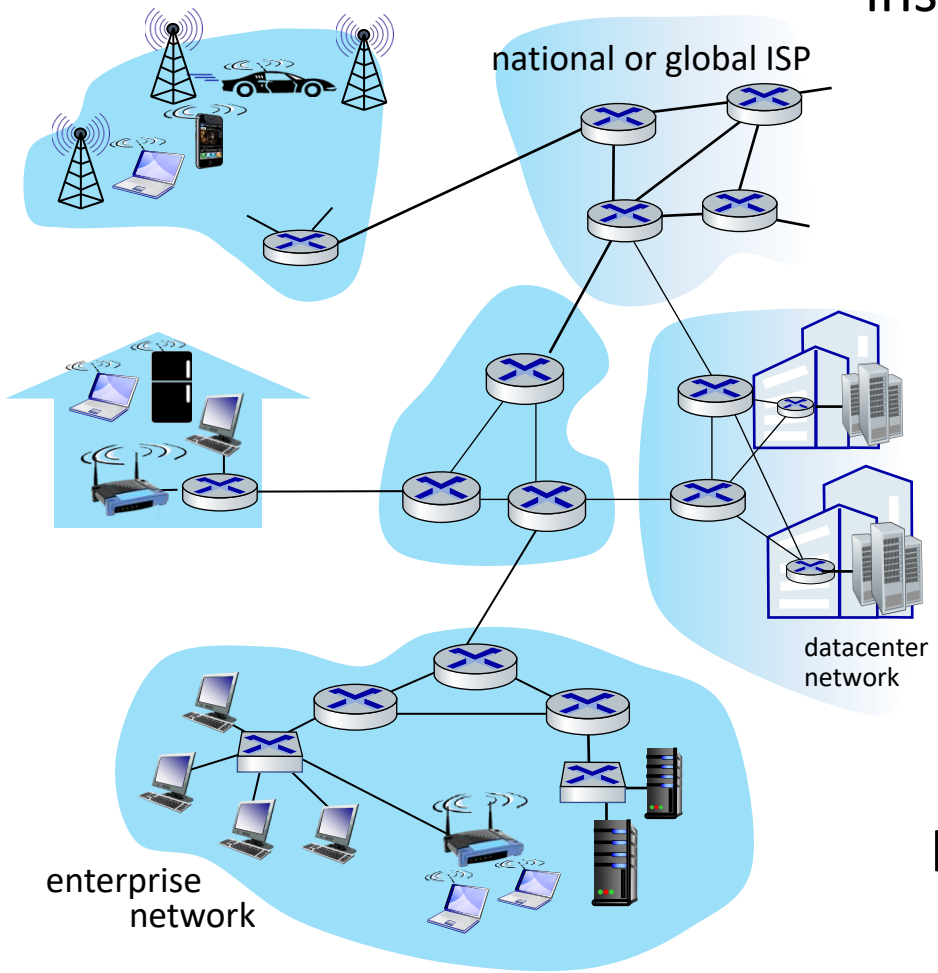


- *NAT Translation*.
  - NAT boxes implement private network addressing,
  - rewriting datagram header IP addresses and port numbers.
- *Security Services*.
  - Firewalls block traffic based on header-field values or redirect packets for additional processing, such as deep packet inspection (DPI), Intrusion Detection Systems (IDS).
- *Performance Enhancement*.
  - These middleboxes perform services such as compression, content caching, and load balancing of service requests (e.g., an HTTP request, or a search engine query) to one of a set of servers that can provide the desired service.

Middleboxes everywhere!

NAT: home, cellular, institutional

Application-specific: service providers, institutional, CDN



Firewalls, IDS: corporate, institutional, service providers, ISPs

Load balancers: corporate, service provider, data center, mobile nets

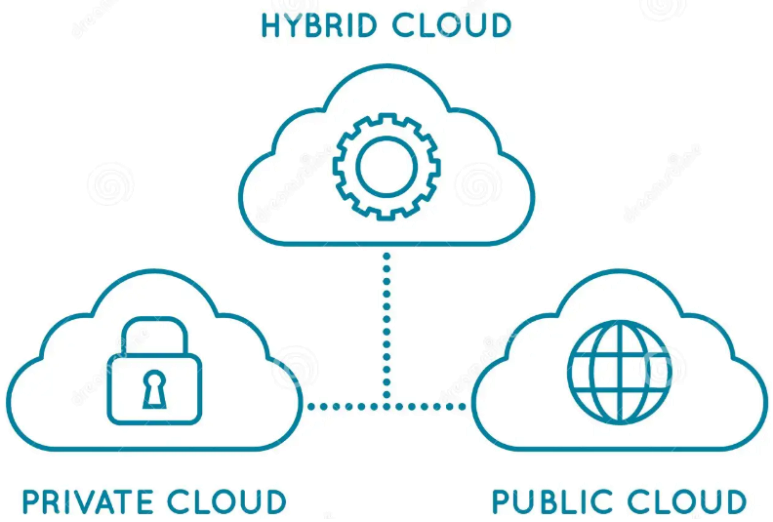
Caches: service provider, mobile, CDNs

Middleboxes

proprietary (closed) hardware solutions



- "whitebox" hardware implementing open API
  - programmable local actions via match + action
  - move towards innovation/differentiation in software
- SDN: (logically) centralized control and configuration management often in private/public cloud
  - network functions virtualization (NFV): programmable services over whitebox networking, computation, storage
- To outsource middlebox functionality to the cloud



Network layer: “data plane” roadmap

- Network layer: overview
  - data plane
  - control plane
- What’s inside a router
  - input/output ports, switching
  - buffer management
  - scheduling
- IP: the Internet Protocol
  - datagram format, addressing
  - network address translation (NAT)
  - IPv6



- Generalized Forwarding
  - Match+action
  - OpenFlow
  - Middleboxes
- Internet architecture

Internet architecture

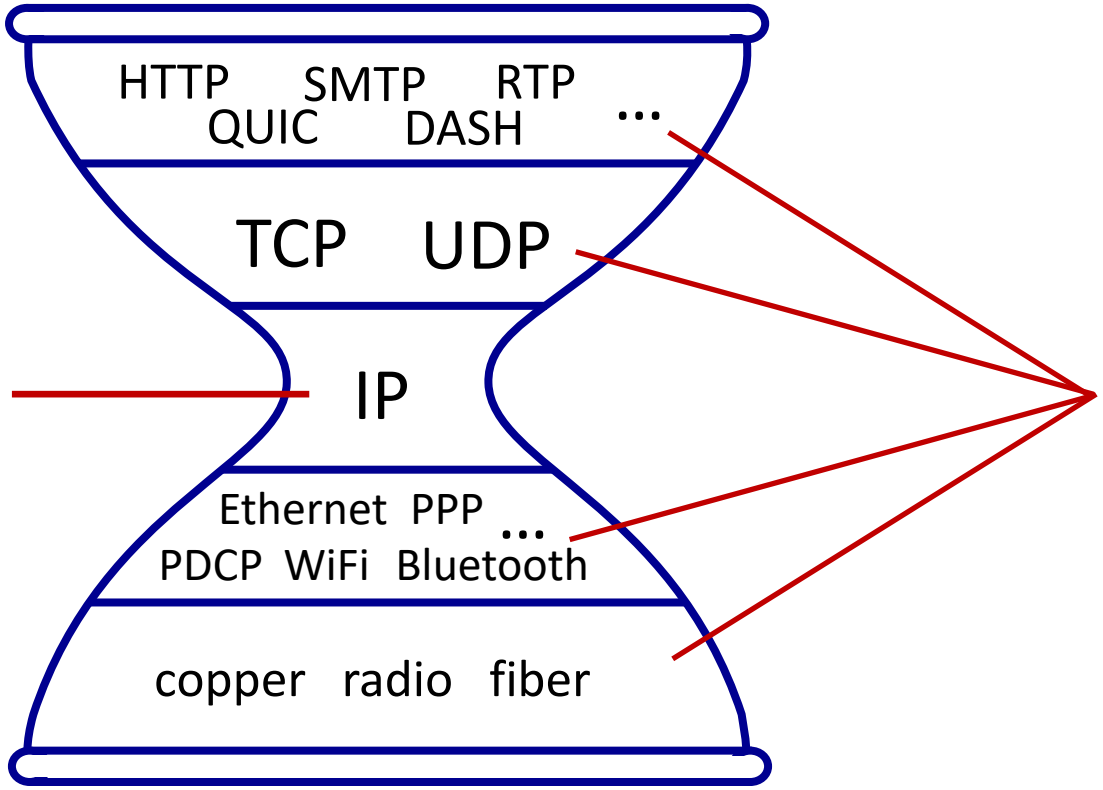


- For many years, the Internet architecture had a clear separation between the network layer and the transport/application layers.
  - the network layer consisted of routers, operating within the network *core*
  - the transport and application layers were implemented in hosts operating at the network *edge*.
- Today's middleboxes clearly violate this separation
  - A **NAT** box: sitting between a router and host, rewrites network-layer IP addresses and transport-layer port numbers;
  - An in-network **firewall:** blocks suspect datagrams using application-layer (e.g., HTTP), transport-layer, and network-layer header fields;
  - **E-mail security gateways:** injected between the e-mail sender (whether malicious or not) and the intended e-mail receiver, filtering application-layer e-mail messages based on whitelisted/blacklisted IP addresses as well as e-mail message content.

The IP hourglass

Internet's "thin waist":

- *one* network layer protocol: IP
- *must* be implemented by every (billions) of Internet-connected devices

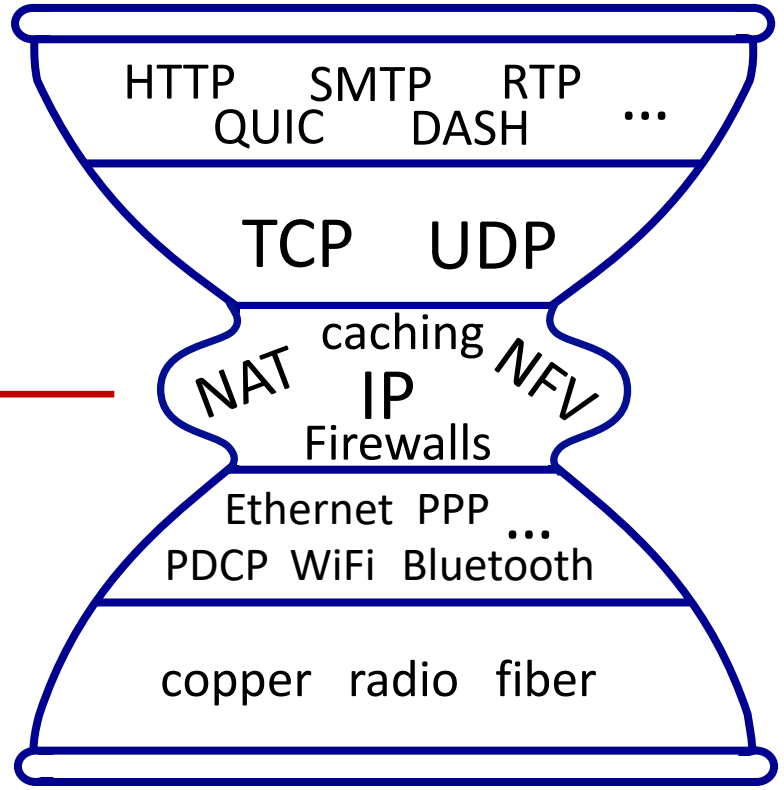


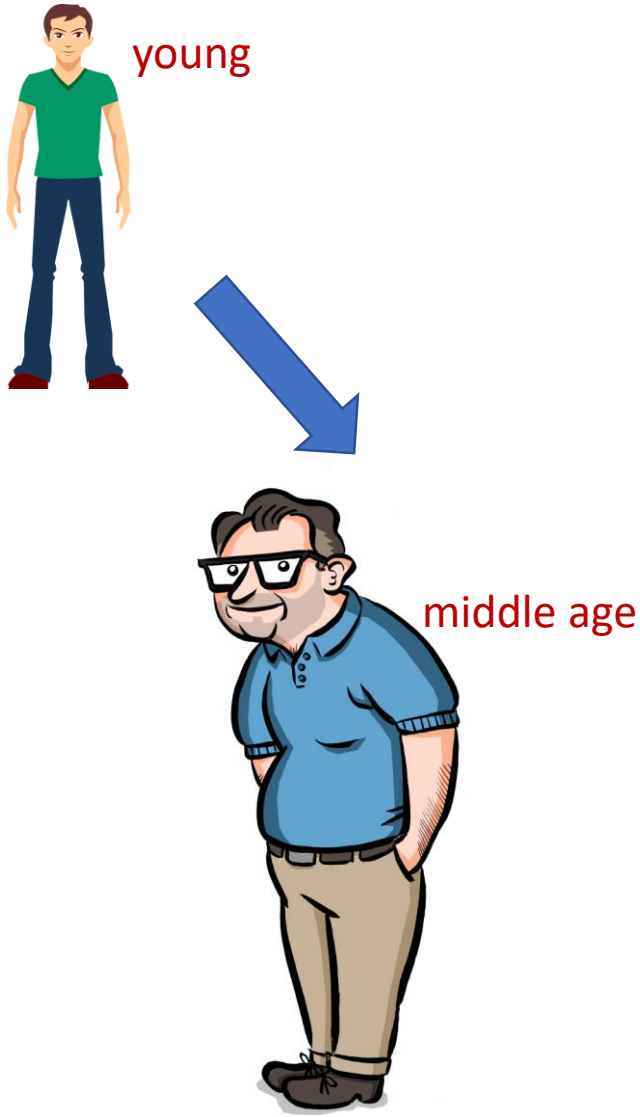
many protocols in physical, link, transport, and application layers

The IP hourglass, at middle age

Internet's middle age "love handles"?

- middleboxes, operating inside the network





Architectural Principles of the Internet

RFC 1958

"Many members of the Internet community would argue that there is no architecture, but only a tradition, which was not written down for the first 25 years (or at least not by the IAB). However, in very general terms, the community believes that the goal is connectivity, the tool is the Internet Protocol, and the intelligence is end to end rather than hidden in the network."

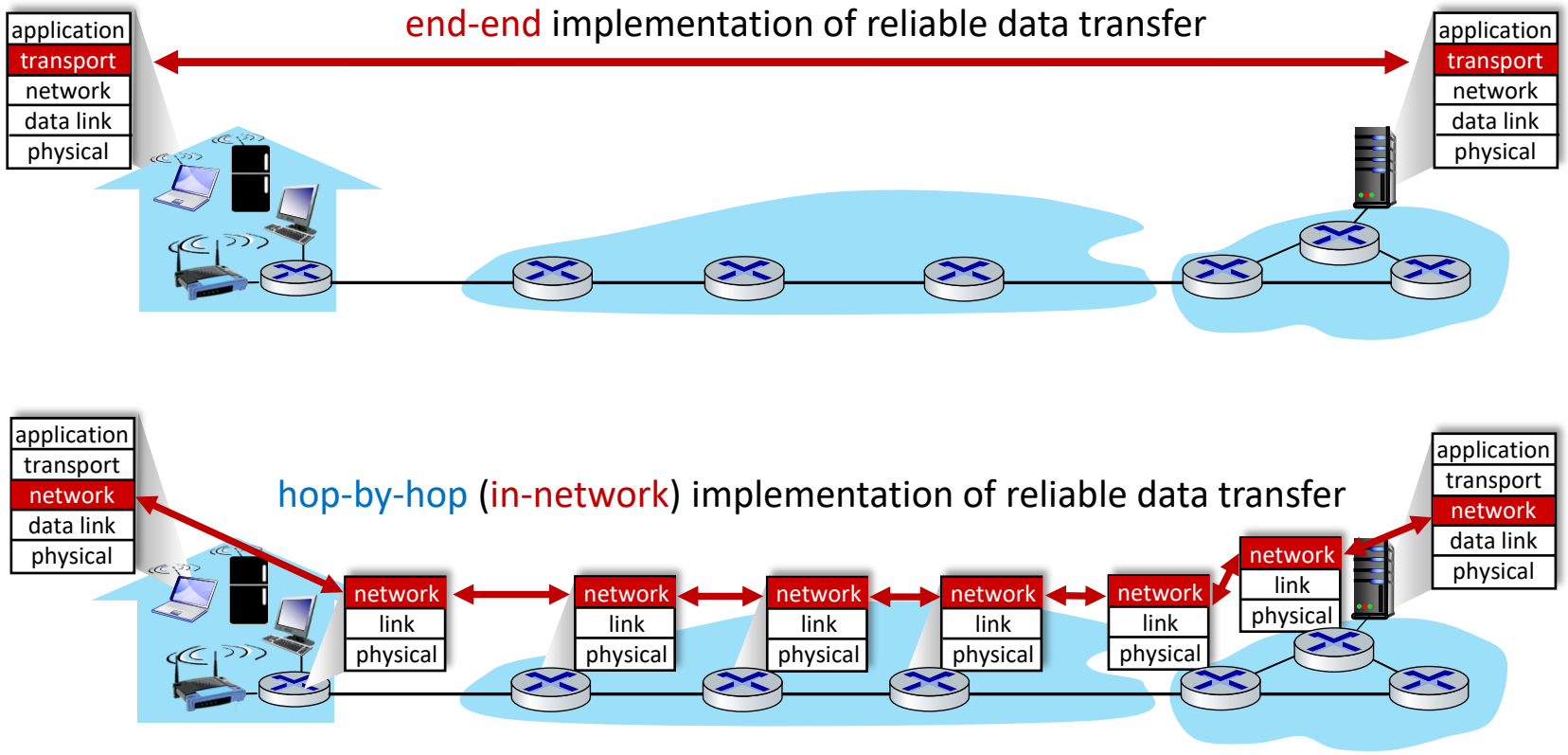
Three cornerstone beliefs:

- simple connectivity
- IP protocol: that narrow waist
- intelligence, complexity at network edge



The end-end argument

- some network functionality (e.g., reliable data transfer, congestion) can be implemented in network, or at network edge

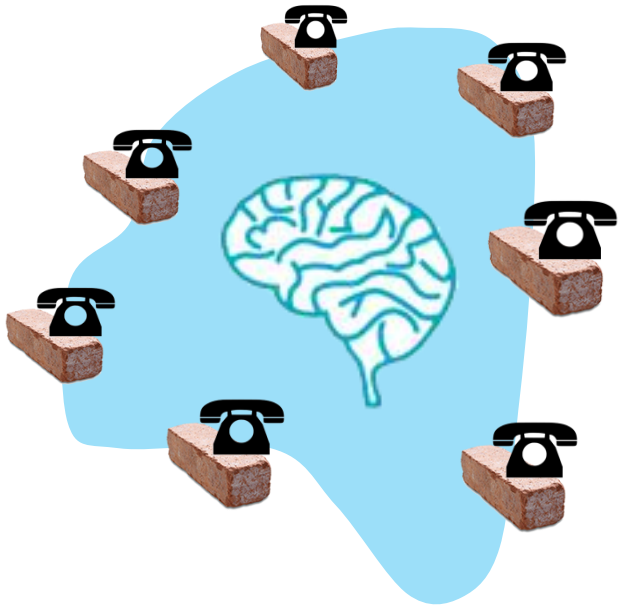


The end-end argument

- some network functionality (e.g., reliable data transfer, congestion) can be implemented in network, or at network edge

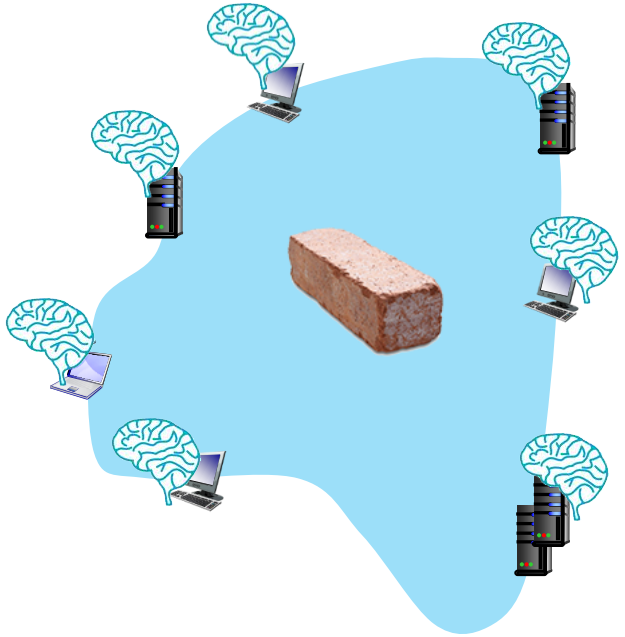
> "The function in question can completely and correctly be implemented only with the knowledge and help of the application standing at the end points of the communication system. Therefore, providing that questioned function as a feature of the communication system itself is not possible. (Sometimes an incomplete version of the function provided by the communication system may be useful as a performance enhancement.)
>
> We call this line of reasoning against low-level function implementation the "end-to-end argument."
>
> Saltzer, Reed, Clark 1984

Where's the intelligence?



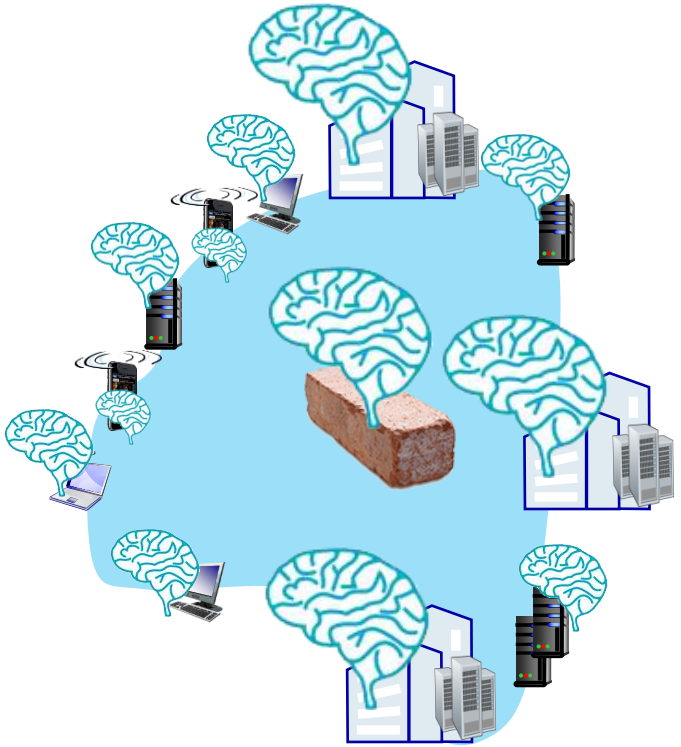
20th century phone net:

- intelligence/computing at network switches



Internet (pre-2005)

- intelligence, computing at edge



Internet (post-2005)

- programmable network devices
- intelligence, computing, massive application-level infrastructure at edge

Chapter 4: done!

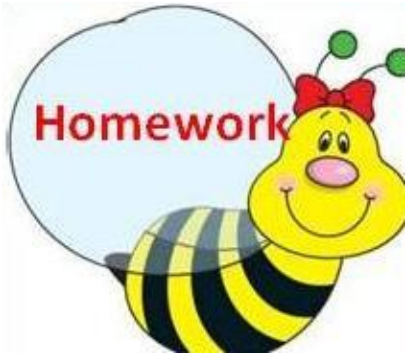
- Network layer: overview
- What's inside a router
- IP: the Internet Protocol
- Generalized Forwarding, SDN
- Middleboxes



Question: how are forwarding tables (destination-based forwarding) or flow tables (generalized forwarding) computed?

Answer: by the control plane (next unit)

Homework



1. Can packet queueing occur at input ports or output ports? What would be the consequences? How to solve the problems?
2. The possible modes in the switching via interconnection network.
3. Give some examples of the present routers for switching via bus and via interconnection network, respectively.
4. Why is IPv6 adoption so slow?
5. Compare and contrast the IPv4 and the IPv6 header fields. Please give the benefits of IPv6 when compared with IPv4.
6. Introduction of the most popular OpenFlow version at present.
7. It has been said that when IPv6 tunnels through IPv4 routers, IPv6 treats the IPv4 tunnels as link-layer protocols. Do you agree with this statement? Why or why not?

Wireshark LAB

